

Tesina di Maturita'

Anno Scolastico 1992-93

**Fochi Michele
&
Amatulli Davide**

Classe 5^aA Sez. Informatica

Introduzione

I programmi per la tesina di maturità sono in tutto 3:

- 1) T.I.P. (Text Image Processor v1.0)
- 2) G.I.P. (Graphic Image Processor v1.0)
- 3) SPRITE (Sprite Manager v5.0)

I primi servono per disegnare una schermata (in modalità testo oppure grafica) per poi convertirla in un file che contenga istruzioni in un certo linguaggio, quindi per facilitare la creazione di maschere o di disegni grafici.

I linguaggi supportati da T.I.P. sono:

- 1) Turbo Pascal
- 2) Turbo C
- 3) Turbo Assembler
- 4) Data Base
- 5) DOS (Ansi.sys)
- 6) Basic

mentre per G.I.P. sono disponibili:

- 1) Turbo Pascal
- 2) Turbo C
- 3) Turbo Assembler

Il terzo programma è stato scritto per consentire una più facile creazione di sprite (la parola deriva da **spirit byte**, cioè oggetti in movimento). Uno sprite è quindi un oggetto disegnato in modo bit-mapped, pixel per pixel, quindi dettagliatamente. Una volta terminato il disegno questo sarà convertito (nel solo linguaggio Turbo Pascal), e sarà possibile visualizzarlo sul video con l'istruzione PutImage.

Tutti i programmi sono stati scritti da FOCHI MICHELE utilizzando il seguente materiale:

- Turbo Pascal v7.0, per la scrittura del programma IP (TIP, GIP e SPRITE)
- Turbo C v1.0, per la scrittura dei programmi in C
- Borland C++ v3.0, per la scrittura dei programmi in C
- Turbo Assembler v3.0, per la scrittura dei programmi in TASM
- Data Base III e IV, per la scrittura dei programmi in DB
- GW-Basic v2.02, per la scrittura dei programmi in BASIC

Questa documentazione, scritta da FOCHI MICHELE e AMATULLI DAVIDE, è stato utilizzato Microsoft Word v2.0 per Windows. Sono stati utilizzati i seguenti testi/manuali:

- Guida completa al Turbo Pascal (McGraw Hill)

- Guida completa al Turbo C (McGraw Hill)
- Il turbo C++ v3.0 (manuale originale)
- Turbo Assembler (manuale originale)
- Grafica in C (Jackson)
- Grafica avanzata in C (Jackson)
- GW-Basic (manuale originale)
- Data Base IV (manuale originale)
- Manuale della stampante compatibile IBM
- Manuale del mouse compatibile MICROSOFT
- Manuale del DR-DOS v6.0
- Manuale dell' MS-DOS v5.0 e v6.0

L' occupazione dei programmi sul disco è la seguente:

- TIP v1.0	32.200 righe, 26 units,	2 MBytes
- GIP v1.0	25.000 righe, 29 units,	2,4 MBytes
- SPRITE v5.0	14.300 righe, 14 units,	1,5 MBytes
- Presentazione	1.100 righe, 11 units,	200 KBytes
- Questa documentazione	258 pagine,	16 MBytes
- Totale		22,1 MBytes

Text Image Processor (T.I.P.)

Questo programma è utile per chi deve creare delle schermate (a colori o no) in bianco e nero, per poi inserirle nel proprio programma, che può essere in batch (utilizzando la conversione in sequenze escape ANSI), in Turbo Pascal, in Turbo C, in Turbo Assembler, in Data Base oppure in Basic. Per modificare il disegno ci sono inoltre una serie di strumenti 'avanzati' (vedi il menu 'Opzioni avanzate') come ad esempio lo spostamento dello schermo, lo spostamento del solo colore di primo piano o carattere o sfondo (o combinati fra loro), ecc.

Verrà data in seguito una breve descrizione del programma e dei suoi menu, visualizzando anche le schermate che dovrebbero apparire all' utente per facilitarne la comprensione.

Appena eseguito il file TIP.EXE si avrà una schermata iniziale vuota, come la seguente:

μ §

Come si può notare, nella parte bassa dello schermo sono indicati il numero di pagina (da 1 a 10, più la ClipBoard), quello di riga (da 1 a 24), quello di colonna (da 1 a 80), il colore (sfondo e primo piano) scelto, qualche informazione (F1 o ESCAPE), l' ultimo carattere digitato (in questo caso è lo spazio), un carattere (codice ASCII 21) che dice se il disegno è stato modificato o no (in questo caso sì), lo stato i NUMLOCK (N), CAPSLOCK (C), SCROLLLOCK (S), INSERT (I), LEFTSHIFT (freccia a sinistra), RIGHTSHIFT (freccia a destra), ALT (!) e CTRL (^). Per ultimo è visualizzata l' ora del sistema, con i due punti (:) lampeggianti ogni mezzo secondo.

Il cursore si dovrebbe trovare nella posizione iniziale, cioè nella prima riga e prima colonna.

Premendo il tasto ESCAPE si apre il menu principale, che è il seguente:

μ §

Dal quale è possibile effettuare tutte le scelte e impartire i comandi al programma.

Sottomenu: CARATTERE ASCII

Questo menu visualizza tutto il set dei 256 caratteri ASCII e ne permette la scelta da parte dell'utente, sia con i tasti di movimento, sia con il mouse, sia digitando il tasto stesso. La finestra di selezione che apparirà sarà simile alla seguente:

μ §

Sono presenti anche altre informazioni, come il codice ASCII nelle diverse conversioni binario, decimale, ottale ed esadecimale.

Sottomenu: CORNICI

E' molto simile al sottomenu precedente, con la differenza che in questo verranno visualizzati solo quei caratteri ASCII che fanno parte di un tipo di cornice. Sul video si vedrà la finestra:

μ §

Sottomenu: TRACCIA CORNICI

Questo strumento è molto comodo quando occorre fare dei riquadri, delle cornici non regolari, delle linee per lo schermo, ecc., in quanto offre la possibilità di disegnare, muovendosi con il cursore, i caratteri grafici precedentemente visti oppure un carattere ASCII in particolare. Ad ogni intersezione verranno congiunti, se possibile, i caratteri incontrati con quello scritto sul video.

Dal menu:

μ §

è possibile scegliere il tipo di cornice (oppure un carattere qualsiasi) o ripristinare il movimento del cursore (con 'Nessuna cornice').

Sottomenu: COLORI

Per modificare il colore di visualizzazione, scegliere questa opzione dal menu principale. Verranno stampati sul video i colori disponibili:

μ §

Scegliere 'ACCETTA ED ESCI' per chiudere la finestra. Per modificare lo stato del lampeggio è sufficiente spostarsi su 'LAMPEGGIO' e premere lo spazio.

Sottomenu: SCEGLI IMMAGINE

Una cosa molto comoda del programma è la possibilità di editing di ben 10 (più la ClipBoard) finestre, semplicemente premendo i tasti ALT-1, ALT-2, ..., ALT-0, ALT-C. In alternativa, si può scegliere questa opzione dal menu principale; verrà visualizzato lo stato, il nome e il tipo di tutte le immagini, e verrà data la possibilità di scegliere l' area di lavoro:

μ §

Sottomenu: BLOCCHI

Lavorare con i blocchi è molto utile. Sono a disposizione le seguenti opzioni:

μ §

Le novità rispetto agli altri programmi commerciali sono la possibilità di leggere un blocco dal disco (sia in un file maschera sia in un file di testo) e di salvarlo, di invertire i caratteri (orizzontali, verticali o solo i colori) di un blocco, di contornarlo, di cancellarlo o riempirlo. Il blocco definito verrà memorizzato nella ClipBoard (l' undicesima finestra), utilizzata da molti comandi di questo programma (anche da quelli del sottomenu SCHERMO).

Per la definizione di un blocco si possono utilizzare sia il mouse sia i tasti ('1' per lo spigolo in alto a sinistra, '2' per quello in basso a destra); il contorno evrrà segnato da frecce che appaiono e scompaiono (ripristinando il disegno) per visualizzare anche i caratteri che vengono utilizzati dal contorno. Per un esempio di definizione di un blocco:

μ §

Sottomenu: SCHERMO

Questo menu è analogo a quello BLOCCHI, con la differenza che ora le operazioni agiscono sull' intero schermo e non su un blocco in particolare. Il menu che apparirà è il seguente:

μ §

Sottomenu: FILES

Offre la possibilità di leggere un file (sia maschera sia testo), salvarlo, cambiare la directory corrente, visualizzare una directory, eseguire un comando DOS, eseguire una SHELL DOS.

La finestra è:

μ §

Per l' I/O su disco di un file è stata utilizzata la stessa proedura (InputFile), che attende il nome di un file visualizzando la seguente finestra di dialogo:

μ §

dove è possibile muoversi con i tasti TAB e SHIFT-TAB tra le due finestre interne (nome file e lista disco), effettuare una ricerca rapida scrivendo le iniziali del nome da trovare nella finestra delle directory (lista), e molte altre cose; per maggiori informazioni premere il tasto F1.

Allo stesso modo è stata scritta una procedura che modifica la directory, che ha il seguente aspetto:

μ §

Sottomenu: CONFIGURAZIONE

Questo menu è utile per modificare particolari configurazioni, quali la velocità del mouse, la velocità di apparizione di menu, finestre, ecc., i colori dello schermo, i bordi e le frecce dei blocchi e delle finestre, ecc.

Il menu si presenta così:

μ §

Per modificare i colori è sufficiente posizionarsi sul campo scelto (vedi figura sotto), premere RETURN e, dalla finestra dei colori che apparirà (vedi opzione 'COLORI' del menu principale) scegliere il colore desiderato. Ci sono molti colori, per cui è consigliabile, se si intende modificarli, salvarli nel file di configurazione (vedi 'Salva configurazione' più avanti).

Modifica dei colori:

μ §

Per quanto riguarda la modifica delle velocità (ritardi di apparizione), che si presenta con la seguente finestra:

μ §

è sufficiente utilizzare i tasti di movimento del cursore (su e giù per cambiare campo, destra e sinistra per modificare il valore del campo).

E' possibile modificare anche il tipo di cornici delle finestre, il tipo di bordo del blocco evidenziato o delle sue frecce oppure la velocità di movimento del mouse (se installato). Le finestre sono tutte visualizzate di seguito.

Modifica cornici delle finestre e del bordo del blocco evidenziato:

μ §

Modifica delle frecce del blocco evidenziato:

μ §

Modifica della velocità del mouse:

μ §

Modifica delle 'Opzioni avanzate':

μ §

Descrivo ora cosa significano i campi della finestra:

Blocca ForeGround	mantiene inalterato il colore di foreground
Blocca BackGround	mantiene inalterato il colore di background
Blocca Carattere	mantiene inalterato il carattere
Return a Capo	SI = premendo RETURN si va a capo della riga seguente NO = premendo RETURN si visualizza l' ultimo carattere digitato
Inverti Caratteri X	nell' invertire un blocco o uno chermo, modifica quei caratteri che hanno un corrispondente carattere rovesciato (es. 'V', '()', '<>', le cornici, ecc.)
Inverti Caratteri Y	nell' invertire un blocco o uno chermo, modifica quei caratteri che hanno un corrispondente carattere rovesciato (es. 'V', 'v^', 'MW', le cornici, ecc.)
Abilita il Suono	abilita (SI) o disabilita (NO) il suono dello speaker del PC
Cursore Normale	se SI il cursore è un blocco pieno; se NO il cursore è formato da due linee perpendicolari (una orizzontale e l' altra verticale) che si incontrano nella posizione (riga e colonna) corrispondente del cursore.

Per leggere il file TIP.CFG che contiene le configurazioni sul disco, scegliere 'Leggi configurazione'; per salvarle su disco, e quindi sovrascrivere il file, scegliere 'Salva configurazione'.

Sottomenu: CONVERSIONE

E' possibile convertire il file in diversi linguaggi e con notevoli varianti:

μ §

Sotttomenu: HELP

Verrà aperta una finestra che conterrà le schermate di aiuto per l' utente, leggendole dal file TIP.HLP. E' possibile scegliere un argomento specifico, utilizzare la ricerca veloce per cercarne uno, visualizzare l' indice degli argomenti, la schermata precedente, ed altro. Vedere l' argomento 'Help generale' per maggiori informazioni.

Il video apparirà così:

μ §

Sottomenu: USCITA

Chiede all' utente se desidera uscire dal programma e lo informa delle immagini che sono state modificate, dopo l' ultimo salvataggio.

Tasti Speciali

Se si tengono premuti per alcuni secondi i tasti speciali SHIFT, CTRL oppure ALT,

compariranno delle finestre che informano l'utente sui tasti di scelta rapida disponibili (ad esempio, CTRL-L per leggere un file, SHIFT-F1 per l'indice degli argomenti, ALT-1 per modificare la prima pagina, ecc.).

Se si tiene premuto il tasto ALT compare la seguente finestra:

Se si tiene premuto CTRL compare:

Mentre se si tiene premuto SHIFT:

Per modificare il ritardo di apparizione, modificare il valore del campo **Menu Speciali**, sottomenu Configurazione/ Modifica Velocità del menu principale (o, come mostrato in figura, SHIFT-F3).

Graphic Image Processor (G.I.P.)

Quando si presenta la necessità di creare disegni o presentazioni personalizzate, il programma G.I.P. diventa molto utile, e risparmia al programmatore il noioso e lungo lavoro di calcolare le coordinate delle figure disegnate una ad una per vedere se è corretta oppure no. E' un normale paint per DOS, simile più ad un CAD, in quanto non si può lavorare con le immagini che normalmente si vedono nei video giochi o con immagini digitalizzate, per il semplice motivo che i linguaggi supportati non sono in grado di gestirle (cioè non con poche istruzioni come un rettangolo). La schermata che comparirà appena eseguito il programma è la seguente:

µ §

Se si ha familiarità con l'ambiente grafico WINDOWS si noteranno molte somiglianze: pulsanti (icone) tridimensionali con effetto ombra, barre di scorrimento, ecc.

All'interno di questo programma è possibile utilizzare sia il mouse sia la tastiera, ma è consigliato quest'ultimo e, dato che la tastiera non si utilizza ovunque, all'inizio dell'esecuzione è controllata l'installazione del dispositivo.

Per farlo funzionare correttamente è necessario avere una scheda grafica VGA 640x480, un mouse, il driver del mouse caricato, e un DOS v3.1 o superiore.

E' presente un aiuto in linea per sapere qualche informazione o per dare suggerimenti a chi non conosce l'ambiente G.I.P., semplicemente premendo una delle combinazioni di tasti seguenti:

- 1) <F1> Aiuto generale
- 2) <Shift> + <F1> Indice degli argomenti
- 3) <Control> + <F1> Aiuto specifico
- 4) <Alt> + <F1> Schermata di aiuto precedente

Le schermate di aiuto sono memorizzate nel file 'GIP.HLP'.

§

1 2 3 4 5 6 7 8 9 10 11 12 13

Saranno di seguito descritte le icone presenti nella parte laterale sinistra dello schermo:

1) Icona di scelta/modifica degli oggetti
 permette di scegliere un oggetto nella finestra di disegno e, se selezionato, di modificarne le dimensioni (rettangolo, cerchio, ellisse, linea, ecc.); se la figura è un testo ne permette la modifica della riga.

2) Icona di operazioni sul/dal disco
 si aprirà una finestra per la scelta del tipo di operazione da effettuarsi:

Nell' ordine, le operazioni sono:

- a) Leggi file immagine (*.GIP)
- b) Salva file immagine (*.GIP)
- c) Leggi file palette colori (*.PAL)
- d) Salva file palette colori (*.PAL)
- e) Nuovo disegno
- f) Stampa immagine
- g) Esci dal programma

La finestra per la lettura, scrittura, conversione, ecc., che interessano file su disco è del tipo:

μ §

3) Icona tavolozza dei colori (palette)
 Permette di modificare i colori del programma, personalizzandoli.
 La finestra che comparirà è la seguente:

dove: OK serve per accettare la configurazione,
 ESCI serve per uscire senza modificare i colori
 RESET serve per riportare il colore selezionato ai valori precedenti
 RESET ALL serve per riportare tutti i colori ai valori precedenti

configurazione DEFAULT serve per ripristinare il colore selezionato (secondo le hardware),
configurazioni DEFAULT ALL serve per ripristinare tutti i colori (secondo le hardware).

4) Icona movimento del puntatore
disegnati Permette di muovere il puntatore dell' oggetto, per scorrere la lista degli oggetti (dal primo all' ultimo). Le scelte possibili sono:

- Vai al primo oggetto
- Vai all' oggetto precedente
- Vai all' oggetto successivo
- Vai all' ultimo oggetto
- Cancella l' oggetto selezionato

i tasti Lo spostamento verso l' oggetto precedente e successivo può essere fatta anche con <TAB> e <Shift> + <TAB>, e la cancellazione con il tasto <CANCEL> o <DELETE>.

5) Icona disegno libero
Sceglie il disegno libero come strumento di disegno (punti).

6) Icona operazione blocchi
Effettua le operazioni principale con un blocco, che sono:

- Memorizzazione del blocco
- Richiamo del blocco memorizzato
- Spostamento del blocco
- Copia del blocco

7) Icona conversione del disegno
Converte l' immagine in memoria in un file di testo che contiene le istruzioni in:

- Turbo C
- Turbo Pascal
- Turbo Assembler

modificabile direttamente dall' utente.

8) Icona aiuto
Richiama la schermata di aiuto generale.

9) Icona font

Sceglie alcuni parametri per la visualizzazione di una stringa di testo:

- Tipo di font DefaultFont
- Tipo di font TriplexFont
- Tipo di font SmallFont
- Tipo di font GothicFont
-
- Giustificazione orizzontale sinistra
- Giustificazione orizzontale centrata
- Giustificazione orizzontale destra
- Giustificazione verticale alto
- Giustificazione verticale centrata
- Giustificazione verticale basso
- Dimensione personalizzata del font
- Direzione orizzontale
- Direzione verticale

Una volta scelte queste cose (non è obbligatorio però), è sufficiente premere il pulsante sinistro del mouse alle coordinate in cui si desidera visualizzare la riga di testo (è solo indicativo, può essere modificata in subito dopo) e apparirà una finestra simile alla seguente:

dove è possibile scegliere tutti i parametri disponibili e in più la stringa di testo da stampare sul video. Selezionando il pulsante OK si potrà muovere un rettangolo all'interno della finestra di disegno: il contorno del rettangolo corrisponde all'area che la riga digitata precedentemente occuperà realmente nel disegno. Premendo il pulsante sinistro del mouse verrà posizionato il testo.

10) Icona scelta strumenti

Permette di scegliere uno strumento tra quelli disponibili:

- Cerchio vuoto
- Cerchio pieno
- Rettangolo vuoto
- Rettangolo pieno
- Cubo vuoto
- Cubo pieno
- Poligono vuoto
- Poligono pieno

- Linea
- Settore circolare vuoto
- Settore circolare pieno
- Arco vuoto
- Arco vuoto + corda
- Riempie un' area delimitata dal colore di primo piano
- Ridisegna l' immagine
- Mantiene il centro (punto di inizio) delle figure fisso
- Mantiene il centro (punto di inizio) delle figure variante
- Decide il colore di sfondo.

11) Icona scelta retino

Cambia il retino corrente, selezionabile tra uno dei seguenti:

- EmptyFill
- SolidFill
- LineFill
- LtSlashFill
- SlashFill
- BkSlashFill,
- LtBkSlashFill
- HatchFill
- XHatchFill,
- InterleaveFill
- WideDotFill
- CloseDotFill
- UserFill

quella Il retino USER è quello definito dall' utente (cioè personalizzato), e, scegliendo icona appaierà un' altra finestra, simile alla seguente:

in cui è possibile cambiare a piacere la configurazione di riempimento, utilizzando sempre il colore di sfondo.

12) Icona scelta linea

Permette di scegliere il tipo di linea da utilizzare, selezionabile tra i seguenti:

- Linea continua sottile (SolidLn, Thickness=1)
- Linea continua spessa (SolidLn, Thickness=3)
- Linea puntata sottile (DottedLn, Thickness=1)
- Linea puntata spessa (DottedLn, Thickness=3)

- Linea tratteggiata sottile (CenterLn, Thickness=1)
- Linea tratteggiata spessa (CenterLn, Thickness=3)
- Linea tratto-punto sottile (DashedLn, Thickness=1)
- Linea tratto-punto spessa (DashedLn, Thickness=3)
- Linea definita dall' utente (UserLn)

Vale lo stesso discorso di prima: se si sceglie il tipo di linea USER, sarà possibile personalizzare la linea; comparirà la finestra:

Selezionare OK per confermare, ESCI per annullare.

13) Icona orologio

Appariranno due orologi sul video: il primo analogico, con 4 lancette (ore, minuti, secondi e centesimi di secondi); il secondo digitale con ore e minuti. Un esempio è il seguente:

Nella parte bassa sono presenti i 16 pulsanti per il colore di sfondo e gli altri 16 per il colore di primo piano. Vicino si trova lo spazio destinato alle coordinate grafiche (x e y), i colori selezionati e l' ultima operazione/disegno effettuata/scelto. Questo si può vedere dalla seguente figura:

Sprite Manager (SPRITE)

Se si vuole che il programma lavori in modalità grafica (quello che accade normalmente oggi) occorre prima o poi scontrarsi con il problema di disegnare piccoli oggetti, utilizzati la maggior parte delle volte per renderlo più presentabile. In Pascal esiste una procedura, la PutImage, relativamente veloce, che visualizza sul video una certa immagine precedentemente salvata. Se però non è stata salvata in un buffer di memoria, in quanto ci vorrebbe molto tempo per calcolare esattamente le coordinate, la procedura non serve a nulla. Il programma Sprite Manager è stato scritto appositamente per tale scopo: dopo aver disegnato il proprio sprite sul video, verrà creato, a richiesta, un file che conterrà una marea di numeri che, correttamente sistemati, daranno l' immagine voluta. La tecnica con la quale vengono ricavati tali numeri è abbastanza complessa, basata sulla rappresentazione a piani di bit.

Sprite Manager non ha finestre di aiuto, in quanto è, come accennato, solo un programma secondario, che nei progetti iniziali non rientrava negli obiettivi della tesina.

Per fare un esempio, è stato molto utile per creare tutte le icone, i pulsanti ed ogni oggetto che

richiedeva quelle speciali caratteristiche del programma G.I.P. dello stesso Sprite Manager (con la prima versione - 1.0 - in modalità testo).

Prima di descrivere l' ambiente e tutti i vari comandi disponibili, passo alla descrizione delle istruzioni principali per utilizzare uno sprite all' interno di un altro programma.

Eseguire Sprite Manager, digitando:

C:\SPRITE> SPRITE <Invio>

Se tutto è corretto dovrebbe apparire la schermata del programma, simile a questa:

μ §

Gli errori che si possono verificare sono 2: non avere una VGA, risoluzione 640x480, 16 colori, e per questo problema non ci sono soluzioni; oppure non avere un mouse o non aver caricato il driver relativo (acquistarne uno o caricare il driver per continuare).

Come si può benissimo vedere, la maggior parte dello schermo è occupata dal disegno dello sprite in dimensioni ingrandite (la scala minima è 2:1, mentre la massima è 419:1). La parte in basso a destra serve per lo sprite in dimensioni reali (scala 1:1). L' editing è permesso sia sulla prima sia sulla seconda indifferentemente.

In alto a destra ci sono le icone (o pulsanti) che permettono di eseguire tutti i comandi del programma. Sul video appariranno così:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

17 18 19 20 21 22 23 24 25 26 27 28

Passo ora alla descrizione di ciascun pulsante:

1) ... 16) Icone colore

Scegliere un colore fra i 16 disponibili

17) Icona nuovo sprite

Permette di pulire l' area di lavoro, cancellando lo sprite in memoria (ne viene chiesta la conferma).

18) Icona lettura di uno sprite dal disco

Alla pressione di questo pulsante apparirà la seguente finestra:

L' da dove sarà possibile inserire il nome del file da leggere (o sceglierlo dalla lista).
estensione di default è '.SPR', ma può essere qualunque. Se il file

selezionato contiene dati corretti, verrà visualizzato lo sprite nella finestra in basso a destra (dimensioni reali, 1:1), in modo da consentirne una rapida ricerca (in caso contrario verrà visualizzata la scritta 'No Sprite').

19) Icona uscita dal programma

Esce dal programma e ritorna al DOS.

20) Icona scrittura di uno sprite sul disco

Salva su un file (*.SPR) lo sprite in memoria. La finestra che apparirà è molto simile a quella vista per la lettura da disco.

21) Icona aumento del fattore di scala

Serve per aumentare le dimensioni dello sprite visualizzato nella finestra più grande. La dimensione massima è 419:1.

22) Icona diminuzione del fattore di scala

Serve per diminuire le dimensioni dello sprite visualizzato nella finestra più grande. La dimensione minima è 2:1.

23) Scelta del tipo di griglia

Ci possono essere tre tipi di griglia:

- Griglia normale (linee)
- Griglia per punti
- Nessuna griglia

24) Icona scelta del tipo di strumento di disegno

Sono disponibili diversi strumenti di disegno:

- Cerchio vuoto
- Cerchio pieno
- Poligono vuoto
- Poligono pieno
- Rettangolo vuoto
- Rettangolo pieno
- Disegno libero
- Testo

25) Icona scelta del tipo di linea

I tipi di linea disponibili sono i seguenti:

- Linea continua sottile (SolidLn, Thickness=1)
- Linea continua spessa (SolidLn, Thickness=3)
- Linea puntata sottile (DottedLn, Thickness=1)
- Linea puntata spessa (DottedLn, Thickness=3)
- Linea tratto-punto sottile (DashedLn, Thickness=1)
- Linea tratto-punto spessa (DashedLn, Thickness=3)
- Linea tratteggiata sottile (CenterLn, Thickness=1)
- Linea tratteggiata spessa (CenterLn, Thickness=3)

26) Scelta del tipo di retino

I retini sono quelli standard del Turbo Pascal, e cioè:

- EmptyFill
- SolidFill
- LineFill
- LtSlashFill
- SlashFill
- BkSlashFill,
- LtBkSlashFill
- HatchFill
- XHatchFill,
- InterleaveFill
- WideDotFill
- CloseDotFill

27) Icona informazioni generali sul sistema

Le informazioni sono riferite all' ora, alla memoria, al disco, ecc., e verranno visualizzate in una finestra simile alla seguente:

28) Icona conversione dello sprite in Turbo Pascal

Per convertire lo sprite scegliere un file (la finestra è simile a quella per la lettura di uno sprite) o digitarne il nome.

Non ci sono grosse difficoltà nell' utilizzare questo programma, per cui credo siano sufficienti le poche spiegazioni date.

Il Turbo C

Nelle prossime pagine verranno date informazioni, talvolta solo sommarie, di programmazione in

Nozioni Basilari Sul C

Il C è nato come linguaggio di medio livello ossia il C è meno potente, più difficile da usare o meno sviluppato dei linguaggi ad alto livello, come BASIC e PASCAL. Esso è considerato a medio livello perché unisce elementi dei linguaggi ad alto livello con le funzionalità dell' assembler. Inizialmente il C è stato impiegato per la programmazione di sistema. Un programma di sistema fa parte di un' ampia classe di programmi che forniscono una porzione del sistema operativo del computer o dei suoi programmi di supporto. Per esempio sono generalmente considerati di sistema i seguenti programmi:

- sistemi operativi
- assembleri
- compilatori
- editors
- gestori di basi di dati

In seguito alla crescente popolarità che il C ha ottenuto, molti programmatori hanno iniziato a farne uso per qualsiasi tipo di impegno, grazie alla sua compatibilità ed efficienza.

Per capire meglio il funzionamento del C occorre mettere a fuoco la differenza che sussiste tra compilatore e interprete. I compilatori e gli interpreti sono semplicemente dei programmi sofisticati che opera sul codice sorgente. Un interprete legge il codice sorgente del programma, una linea per volta, ed esegue le operazioni specificate in quella linea. Un compilatore legge l' intero programma e lo converte in un codice oggetto, che è una traduzione del codice sorgente in un formato che il computer può eseguire direttamente. Dopo la compilazione, il programma scritto in sorgente non ha più significato per il fine dell' esecuzione. Se si fa uso di un interprete, il sorgente deve essere disponibile ogni volta che si intende eseguire il programma.

La compilazione richiede una certa quantità di tempo in più rispetto all' interpretazione, ma il tempo risparmiato nell' esecuzione di un programma compilato è ben superiore. In generale i compilatori C eseguono un numero limitato di verifiche di errore durante l'esecuzione, come ad esempio il controllo sui limiti degli array o il controllo di compatibilità tra tipi degli argomenti. La maggior parte dei controlli è quindi lasciata alla responsabilità del programmatore, per non ridurre la velocità di esecuzione. Di conseguenza sono i programmatori che devono decidere l' opportunità o meno di tali controlli.

Il C non può essere considerato come un linguaggio strutturato a blocchi perché non permette di dichiarare sottoprogrammi in maniera annidata. La caratteristica che lo distingue è la possibilità di definire in modo preciso regole di variabilità dei dati e del codice necessarie all' esecuzione di un compito specifico rispetto al resto del programma. Un modo per definire queste regole e quindi creare moduli di programma è quello di utilizzare sottoprogrammi autosufficienti.

In C i programmi vengono ricostruiti tramite le funzioni, per mezzo delle quali è possibile scomporre il programma in piccoli blocchi, ottenendo così programmi più modulari. Un' ulteriore possibilità di modulazione del codice viene offerta dal C per mezzo dei blocchi di codice, che

sono blocchi di istruzioni logicamente connesse che vengono trattate come un'unità. In C, un blocco si ottiene racchiudendo delle istruzioni tra parentesi graffe aperte e chiuse.

Struttura Di Un Programma

Va sempre tenuto presente che le parole chiavi del C vanno scritte solo in minuscolo. Tutti i programmi in C sono costituiti da una o più funzioni. L'unica funzione che deve sempre essere chiamata è il **main()**, ed è la prima funzione a cui viene ceduto il controllo quando inizia l'esecuzione di un programma.

La **main()**, di solito, non esegue alcuna elaborazione ma si limita a chiamare le funzioni più opportune per eseguire il compito richiesto; **f1()**, **f2()**, ..., **fn()** rappresentano funzioni definite dall'utente. In teoria sarebbe possibile scrivere un intero programma C costituito unicamente da istruzioni definite automaticamente dal compilatore. In realtà capita raramente in quanto la definizione formale del linguaggio non prevede istruzione di I/O. Pertanto, per svolgere tali operazioni, è inevitabile fare ricorso a funzioni predefinite dalla libreria del Turbo C.

Il Turbo C possiede una libreria standard che fornisce le funzioni opportune per l'esecuzione delle operazioni più comuni. Quando nel codice si chiama una funzione che non fa parte del programma che si sta scrivendo, questa, grazie al linker, viene cercata nelle funzioni di libreria.

Schema di come si imposta generalmente un programma in linguaggio C:

```
main()
{
variabili locali
sequenza di istruzioni
}

f1()
{
variabili locali
sequenza di istruzioni
}

f2()
{
variabili locali
sequenza di istruzioni
}

fn()
{
variabili locali
sequenza di istruzioni
}
```

}

Tipi Di Dati

In C esistono cinque tipi di dati primari: carattere, intero, reale, doppia precisione e indefinito. Le parole chiave utilizzate per dichiarare rispettivamente variabili di questi tipi sono **char**, **int**, **float**, **double** e **void**.

TIPO	N° BIT OCCUPATI	INTERVALLO
char	8	da 0 a 255
int	16	da -32768 a 32767
float	32	da 3.4E-38 a 3.4E+38
double	64	da 1.7E-308 a 1.7E+308
void	0	indefinito

Le variabili di tipo **char** sono impiegate per contenere valori per cui sono sufficienti 8 bit, come ad esempio la lettere ASCII "A","B","C",ecc. Le variabili di tipo **int** possono contenere numeri che non richiedono una parte frazionaria. Le variabili di tipo **float** e di tipo **double** memorizzano numeri reali. Il tipo **void** è stato introdotto per usi differenti: il principale è che consente di dichiarare esplicitamente che una funzione non restituisce alcun valore. In Turbo C sono presenti anche tipi di dati strutturati scalari come campi di bit: tipi enumerativi e tipi definiti dall'utente. I modificatori sono impiegati per adattare con maggiore precisione i tipi di dati fondamentali alle esigenze del programmatore.

I modificatori possibili possono essere: **signed**, **long**, **unsigned**, **short**.

E' possibile applicare ai modificatori **signed**, **long**, **short**, **unsigned** tipi primari intero e carattere, ma non è possibile applicare **long** al tipo **double**. Sebbene sia ammesso il tipo variante **signed integer**, si tratta di una dichiarazione ridondante, perché il tipo primario intero lo prevede già il segno per default.

TIPO	N° BIT OCCUPATI	INTERVALLO
char	8	da -128 a 127
unsigned char	8	da 0 a 255
signed char	8	da -128 a 127
int	16	da -32768 a 32767
unsigned int	16	da 0 a 65535
signed int	16	da -32768 a 32767
short int	16	da -32768 a 32767
unsigned short int	16	da 0 a 65535
signed short int	16	da -32768 a 32767
long int	32	da -2147483648 a 2147483647
signed long int	32	da -2147483648 a 2147483647
unsigned long int	32	da 0 a 4294967295
float	32	da 3.4E-38 a 3.4E+38
double	64	da 1.7E-308 a 1.7E+308
long double	64	da 1.7E-308 a 1.7E+308

Il Turbo C possiede due varianti che vengono impiegate per controllare il modo in cui avvengono l'accesso e la modifica delle variabili. Il nome di queste varianti è **const**. Quando a una variabile viene applicata alla variante **const**, è possibile assegnarle un valore iniziale, che non può essere modificato per tutta la durata della esecuzione del programma. Per esempio

```
const float versione = 6.0
```

crea una variabile **float** chiamata versione che non può essere alternata dal programma.

Variabili Locali E Variabili Globali

Le variabili dichiarate all'interno di una funzione vengono dette locali. Esse possono essere utilizzate solo all'interno di istruzioni contenute nello stesso blocco dove la variabili sono state dichiarate. In altre parole non sono accessibili all'esterno del blocco di definizione. Esempio:

```
funz1()
{
  int x
  x=10
}
```

```
funz2()
{
  int x
  x=40
}
```

Dove entrambe le funzioni funz1, funz2 dichiarano una variabile di nome X; tuttavia non esista una relazione tra le due variabili, perchè sono dichiarate all'interno di due blocchi differenti. Le variabili globali sono note nell'intero programma e possono essere utilizzate in qualsiasi punto del codice. E' buona pratica di programmazione che le variabili globali siano dichiarate in cima al programma, anche se è possibile dichiararle in qualsiasi punto dalla prima occorrenza, purché sia all'esterno di tutte le funzioni.

Esempio di dichiarazioni di variabili locali e globali:

```
int conta; /* conta è globale */
main()
{
  conta=100;
  funz1()
}

funz1()
```

```

{
    int temp;
    temp == conta;
    funz2()
    printf("Conta vale %d, conta ) ; /* stampa 100 */

funz2()
{
    int conta ;
    for (conta =1 ; conta<10; conta++)
        printf(".")
}

```

Da un esame del programma appare che sia main() che funz() utilizzano la variabile conta, senza averla dichiarate, sfruttando il fatto che si tratta di una variabile globale. La funzione funz2() ha invece dichiarato una variabile di nome conta. Quando funz2() usa il nome conta, accede alla propria variabile locale e non alla omonima globale. In caso di omonimia tra una variabile globale e una locale tutti i riferimenti fatti di una funzione si riferiscono alla variabile locale. Il Turbo C riserva una apposita area di memoria per le variabili globali.

Operatori Matematici

Gli operatori aritmetici del C, sono: +, -, *, /. Gli operatori possono essere applicati alla maggior parte dei tipi di dati. Applicando l'operazione divisione a un intero o a un carattere viene troncato il resto: per esempio 10/3 dà come risultato 3, qualora si tratti di una divisione intera.

L'operatore % restituisce il resto di una divisione intera, ma non può essere usato su tipi float o double.

OPERAZIONE	AZIONE
+	Addizione
-	Sottrazione
*	Moltiplicazione
/	Divisione
%	Resto della divisione
--	Decremento
++	Incremento

Altri tipi di operatori sono le quantità logiche vero e falso che restituiscono come risultato 1 in corrispondenza di vero e 0 in corrispondenza di falso. Qui sotto è mostrata una tabella degli operatori logici e relazionali.

OPERATORI RELAZIONARI

OPERATORE	AZIONE
>	Maggiore

>=	Maggiore o uguale
<	Minore
<=	Minore o uguale
==	Uguale
!=	Diverso

OPERATORI LOGICI

OPERATORE	AZIONE
&&	AND
	OR
!	NOT

Conversioni Di Tipo Nelle Espressioni

Quando in una espressione sono utilizzati dati di tipi differenti, il compilatore converte tutti nello stesso tipo, e in particolare nel tipo che occupa più memoria, operazione per operazione, come descritto dalle regole seguenti:

- 1) Tutti i char e gli short int sono convertiti in int.
Tutti i float sono convertiti in double.
- 2) Per tutte le coppie di operandi si applica la seguente sequenza di operazioni: se uno degli operandi è long double, l' altro operando è convertito in long double. Se uno degli operandi è double, l' altro operando è convertito in double. Se uno degli operandi è long, l' altro è convertito in long. Se uno degli operandi è unsigned, l' altro è convertito in unsigned.

Dopo che il compilatore ha applicato queste regole di conversione, ogni coppia di operandi risulta dello stesso tipo, che sono anche il tipo del risultato. Si noti che la seconda regola è costituita da più condizioni che vanno applicate in sequenza. Nell' esempio, prima il computer converte **car** in int e **f** in

double. Poi il risultato di char viene convertito in double, essendo il risultato di float di tipo double. A questo punto il risultato è double, essendo questo il tipo di entrambi gli operandi.

```
char car;
int i;
float f;
double d;
totale = ( car / i ) + ( f * d ) - ( f + i )
```

Direttive Al Compilatore

Il codice di un programma scritto in C può includere alcune istruzioni al compilatore. Queste istruzioni sono dette direttive del preprocessore. Come definito nella proposta dello standard

ANSI, il preprocessore del C ammette le seguenti direttive:

DIRETTIVA	FUNZIONE
<code>#else</code>	{ direttive per la compilazione condizionale }
<code>#elif</code>	{ direttive per la compilazione condizionale }
<code>#if</code>	{ direttive per la compilazione condizionale }
<code>#ifdef</code>	{ direttive per la compilazione condizionale }
<code>#ifndef</code>	{ direttive per la compilazione condizionale }
<code>#define</code>	Definisce un identificatore e una stringa.
<code>#undef</code>	Elimina la definizione di un nome di macro definita dalla direttiva
<code>#define</code>	
<code>#error</code>	Forza il Turbo C a interrompere la compilazione.
<code>#pragma</code>	Da al compilatore determinate istruzioni.
<code>#include</code>	Chiede al compilatore di includere un determinato file che contiene
la	direttiva <code>#include</code> .

Funzioni Video

Il compilatore C è dotato di una compilatore libreria di funzioni grafiche che permettono la visualizzazione di grafici e diagrammi. Le funzioni video sono simili alle corrispondenti routine di Turbo Pascal. Se si conoscono approfonditamente tutti gli aspetti relativi alle modalità video disponibili sul PC e le operazioni di creazione e gestione delle finestre, il Turbo C fornisce un esteso sistema per output su schermo per i personal computer. I prototipi e l'informazione di intestazione per le funzioni di output di testo su schermo si trovano in **conio.h**. Invece i prototipi e le funzioni relative alle funzioni grafiche si trovano in **graphics.h**.

Il sistema grafico richiede che la libreria **graphics.lib** sia collegata con il vostro programma. Se state usando la versione a linea di comando, dovete necessariamente includere il nome della libreria nella linea di comando. Il PC è dotato di un adattatore video che può essere un MonoChrome Display Adapter (MDA) per la semplice visualizzazione del testo, può essere in grado di visualizzare immagini grafiche, come ad esempio un adattore Color/Graphics Adapter (CGA), un Enhanced Graphics (EGA) un Video Graphics Array adapter (VGA) o un Hercules Monochrome Graphics Adapter. Ognuno di questi può operare in un certo numero di modalità; la modalità specifica il fatto che sullo schermo vengono visualizzate 80 e 40 colonne (solo in modalità testo), la risoluzione (in modalità grafica) e il tipo dello schermo (a colori o in bianco e nero). La modalità operativa dello schermo viene definita quando il programma richiama una delle funzioni definite della modalità grafica: **textmode**, **initgraph** o **setgraphmode**.

In modalità testo, lo schermo del PC è suddiviso in celle (80 o 40 colonne di 25,43 o 53 righe) ognuna composta da un attributo e da un carattere. Il carattere è chiaramente un carattere ASCII visualizzato, mentre l'attributo specifica il modo in cui il carattere visualizzato (il suo colore, l'intensità, e così via). Il C è dotato di un'ampia di routine di gestione del testo dallo schermo, per scrivere il testo direttamente sullo schermo e per controllare gli attributi della cella.

In modalità grafica, lo schermo del PC è suddiviso in pixel; ogni pixel rappresenta un punto dello schermo. Il numero di pixel (o risoluzione) dipende dal tipo di adattatore video utilizzato e

dalla modalità in cui opera. La libreria grafica del Turbo C offre molte funzioni grafiche: è possibile tracciare linee e disegnare aree e controllare il colore di ogni pixel.

In modalità testo, l'angolo superiore sinistro dello schermo è rappresentato dalla coordinata (1,1); le coordinate x crescono da sinistra a destra, mentre la coordinate y crescono dall'alto verso il basso.

In modalità grafica, l'angolo superiore sinistro è rappresentato dalla coordinata (0,0) e le coordinate x ed y crescono esattamente come avviene in modalità testo.

Introduzione Informale Alle Finestre Ed Ai ViewPort

Il linguaggio C++ fornisce una serie di funzioni per creare e gestire le finestre in modalità testo (ed i **viewport** in modali grafica). Una finestra è un'area rettangolare dello schermo in modalità testo. Quando il programma scrive sullo schermo, il suo output rimane all'interno della finestra attiva. Il resto

dello schermo (all'esterno della finestra) non viene dunque modificato. La finestra su cui si opera normalmente è una finestra di testo che occupa l'intero schermo. Il programma può (con una chiamata alla funzione **Window**) decidere di operare su una finestra di dimensioni inferiori; questa funzione specifica la posizione della finestra in termini di coordinate sullo schermo.

Che Cos'è Una ViewPort

Si può definire un'area rettangolare dello schermo anche in modalità grafica; tale area prende il nome di **viewport** e si comporta come uno schermo virtuale. La parte rimanente dello schermo (all'esterno del **viewport**) non viene dunque modificata. Il **viewport** viene definito in termini di coordinate dello schermo con una chiamata alla funzione **setviewport**.

Coordinate

Se si escludono le funzioni di definizione delle finestre e dei viewport, tutte le coordinate delle funzioni in modalità testo e in modalità grafica, devono essere specificate in modo relativo all'origine dell'oggetto

(finestra o viewport) su cui operano. Così, la coordinata (1,1) viene posta nell'angolo superiore sinistro di una finestra, mentre la coordinata (0,0) viene posta nell'angolo superiore sinistro di un viewport.

Programmazione In Modalità Testo

In C le funzioni di input e output (nel seguito abbreviate con I/O) dirette alla console (**cprintf**, **cputs** e così via) permettono di effettuare in modo molto efficiente tutte le operazioni di output del testo, di gestione della finestra, del posizionamento del cursore e di controllo degli attributi. Tali funzioni sono contenute nelle librerie standard di Turbo C, i relativi prototipi si trovano nel file include **conio.h**.

Le Funzioni Di I/O Della Console

Le funzioni del C che operano in modalità testo possono essere utilizzate in tutte e sei le modalità testo. Le modalità disponibili sul sistema, dipendono dal tipo di adattatore video e dal monitor utilizzato; per specificare la modalità testo da utilizzare si deve richiamare la funzione **textmode**.

Le funzioni che operano in modalità testo sono diverse in cinque gruppi separati:

- output e gestione del testo;
- controllo delle finestre e della modalità...;
- controllo degli attributi;
- richiesta informazioni sullo stato utilizzato;
- forma del cursore.

Le funzioni di output e gestione del testo sono:

CPRINTF	Invia allo schermo l' output formattato.
CPUTS	Invia uno stringa sullo schermo.
GETCHE	Legge un carattere e lo stampa sullo schermo.
PUTCH	Invia un carattere allo schermo.
CLREOL	Cancella dal cursore alla fine della riga.
CLRSCR	Cancella il testo della finestra.
DELLINE	Cancella la riga del cursore.
GOTOXY	Posiziona il cursore.
INLINE	Inserisce una riga vuota sotto alla riga del cursore.
MOVETEXT	Copia il testo da un' area dello schermo ad un' altra.
GETTEXT	Copia il testo da un' area dello schermo alla memoria.
PUTPIXEL	Copia il testo della memoria ad un' area dello schermo.

I programmi che effettuano operazioni di output sullo schermo, operano normalmente su una finestra di testo grande quanto l' intero schermo e quindi è possibile scrivere, leggere e gestire il testo immediatamente senza bisogno di definire in anticipo le modalità di funzionamento.

Le funzioni di output diretto sullo schermo sono: **cprintf**, **cputws** e **cutch**, mentre per visualizzare il testo immesso si utilizza la funzione **getche**. Per cancellare il contenuto della finestra attiva si utilizza la funzione **clrscr**, per cancellare una parte di una riga si utilizza **clreol**, per cancellare l' intera riga si utilizza **delline**, mentre per inserire una riga vuota **inline**.

Le ultime tre funzioni si basano sulla posizione del cursore: per spostare in una posizione ben determinata si utilizza **gotoxy**. Per copiare un blocco di testo da una posizione all' altra di una finestra si usa **movetext**. Per copiare in memoria un' area di testo si utilizza la funzione **gettext**: per riportare sullo schermo (in qualunque posizione) tale area si utilizza **puttext**.

Controllo Delle Finestre E Della Modalità Operativa

Vi sono due funzioni di controllo delle finestre e delle modalità operative:

textmode	Attiva la modalità testo
Window	Definisce una finestra in modalità testo.

Per definire la modalità operativa dello schermo (solo per il testo) si utilizza la funzione **textmode**: tale funzione è limitata dall' adattatore e dal monitor utilizzati. Tale funzione analizza lo schermo come finestra di testo di dimensioni pari all' intero schermo, utilizzando la modalità specificata e cancella ogni immagine o testo che si trova sullo schermo. Quando lo schermo è in modalità testo, si può scegliere di visualizzare il testo sull' intero schermo o su una porzione, cioè in una finestra.

Per creare una finestra di testo, si utilizza la funzione **window** e si specifica l' area di schermo che dovrà occupare. Le funzioni di controllo degli attributi in modalità testo sono:

textattr	Definizione dei colori (attributi) per il primo piano e o sfondo.
textbackground	Definizione del colore (attributo) dello sfondo.
textcolor	Definizione del color del primo piano.
highvideo	Testo ad alta intensità di visualizzazione.
lowvideo	Testo a bassa intensità di visualizzazione.
normvideo	Testo riportato all' intensità originale.

Le funzioni di controllo degli attributi definiscono l' attributo indicato rappresentandolo con un valore di 8 bit: i quattro bit inferiori rappresentano il colore del primo piano, i tre bit successivi rappresentano il colore dello sfondo e il bit più alto è il bit di "lampeggio". Tutto il testo visualizzato in seguito, assume gli attributi scelti. Con le funzioni di controllo degli attributi, è possibile scegliere i colori dello sfondo e del primo piano (il carattere) separatamente con **textbackground** e **textcolor** o insieme con **textattr**. Si può definire anche l' attributo anche l' attributo di lampeggio del carattere in primo piano. I monitor a colori in modalità colore, visualizzano molto probabilmente i colori esatti: i monitor non a colori possono convertire alcuni o tutti gli attributi in varie sfumature monocromatiche o in altri effetti visivi, ad esempio il grassetto, la sottolineatura, l' inversione ed altri ancora. Per visualizzare ad alta intensità i caratteri che sono attualmente in alta intensità si utilizza la funzione **lowvideo** (che disattiva il bit di alta intensità dei caratteri).

Per ripristinare i valori di intensità originali, si utilizza la funzione **normvideo**. Le funzioni di informazione sul tipo di visualizzazione in uso, sono le seguenti:

gettextinfo	Memorizza in una struttura text_info tutte le informazioni relativa alla finestra di testo corrente.
wherex	Fornisce la coordinata x della cella contenete il cursore.
wherey	Fornisce la coordinata y della cella contenete il cursore.

Le funzioni di I/O di console disponibili in C, ne includono alcune che permettono di conoscere

informazioni sullo stato del sistema, ad esempio la modalità di testo utilizzata per la finestra e la posizione corrente del cursore della finestra. La funzione **gettextinfo** memorizza in una struttura **text_info** (definita in **conio.h**) alcune informazioni relative alla finestra di testo; tra esse vi sono:

- la modalità video corrente;
- la posizione della finestra in termini di coordinate assolute dello schermo;
- le dimensioni della finestra;
- i colori del primo piano e dello sfondo;
- la posizione corrente del cursore.

Talvolta si desidera conoscere solo alcune di tali informazioni, ad esempio per conoscere le coordinate del cursore relative alla finestra, è sufficiente richiamare le funzioni **wherex** e **wherey**.

Forma Del Cursore

Per modificare l' aspetto del cursore si utilizza la nuova variabile globale **_setcursortype**. I valori ammessi sono **_nocursor** che disattiva il cursore, **_solidcursor** che visualizza un cursore costituito da un rettangolo pieno e **_normalcursor** che visualizza il cursore standard (ovvero il carattere di sottolineatura).

Finestra Di Testo

La finestra di testo standard è costituita dall' intero schermo; per definire una finestra di testo di dimensioni inferiori si utilizza la funzione **window**. Le finestre di testo possono contenere al massimo 50

righe di 40 o 80 colonne. Il punto di origine delle coordinate di una finestra di testo in C è costituito dall' angolo superiore sinistro della finestra che costituisce il punto di coordinate (1,1): le coordinate dell' angolo inferiore destro di una finestra di testo grande quanto l' intero schermo do 80 colonne e 25 righe è (80,25).

Un Esempio

Supponiamo che il PC sia in modalità testo ad 80 colonne e che si voglia, creare una finestra. L' angolo superiore sinistro della finestra dovrà trovarsi alle coordinate (10,8), mentre l' angolo inferiore destro della finestra dovrà trovarsi alle coordinate (50,21). Si dovrà pertanto utilizzare una funzione **windows**, come la seguente:

```
window(10, 8, 50, 21);
```

Ora che è stata creata la finestra di testo, per spostare il cursore nella posizione della finestra (5,8), relativa all' origine della finestra e per scrivere una stringa si utilizzano le funzioni **gotoxy** e **cputs**.

Il Tipo Text_Mode

Per cambiare modalità di visualizzazioni in una delle sette modalità testo del PC si deve richiamare la funzione **textmode**. Il tipo enumerativo **text_mode**, definito in **conio.h**, permette di utilizzare, per l' argomento modalità della funzione **tetxmode**, una serie di costanti numeriche che potrebbero generare dubbi o confusione. Tuttavia, se si intendono usare le costanti simboliche, si deve includere nel codice la riga seguente:

```
include <conio.h>
```

I valori numerici e simbolici definiti da **text_mode** sono i seguenti:

COSTANTE SIMBOLICA	VALORE NUMERICO	MODALITA' VIDEO
LASTMODE	-1	Abilitazione modalità' testo
BW40	0	Bianco, nero 40 colonne
C40	1	16 colori, colonne
BW80	2	Bianco, nero 80 colonne
C80	3	16 colori , 80 colonne
MONO	7	Monocromatico, 80 colonne
C4350	64	EGA, 80x43 righe : VGA 80x50 righe

Ad esempio le seguenti chiamate a **textmode** attivano su un monitor a colori la modalità operative indicate:

textmode(0)	Bianco e nero, 40 colonne
textmode(BW80)	Bianco e nero, 80 colonne
textmode(C40)	16 colori, 40 colonne
textmode(3)	16 colori, 80 colonne
textmode(7)	Monocromatico,80 colonne
textmode(C4350)	EGA, 80x43 righe: VGA 80x50 righe

Per determinare il numero di righe del testo dello schermo dopo aver chiamato **textmode** nella modalità C4350, si deve utilizzare **SETTTEXTINFO**.

Testo A Colori

Quando una cella è occupata da un carattere, il colore del carattere costituisce il primo piano e il carattere della parte rimanente della cella è lo sfondo. I monitor a colori della parte rimanente video a colori possono visualizzare fino a 16 colori diversi, i monitor monicromatici visualizzano i colori utilizzando altri effetti visivi (alta intensità, sottolineata , video inverso e così via).

COSTANTE SIMBOLICA	VALORE NUMERICO	PRIMO PIANO O SFONDO ?
BLACK	0	Entrambi
BLUE	1	Entrambi

GREEN	2	Entrambi
CYAN	3	Entrambi
RED	4	Entrambi
MEGENTA	5	Entrambi
BROWN	6	Entrambi
LIGHTGRAY	7	Entrambi
DARKGRAY	8	Solo primo piano
LIGHTBLUE	9	Solo primo piano
LIGHTGREEN	10	Solo primo piano
LIGHTCYAN	11	Solo primo piano
LIGHTRED	12	Solo primo piano
LIGHTMAGENTA	13	Solo primo piano
YELLOW	14	Solo primo piano
WHITE	15	Solo primo piano
BLINK	128	Solo primo piano

Il file **include conio.h** definisce alcuni simbolici per i colori disponibili. Se si intendono usare tali costanti simboliche, si deve includere nel codice sorgente il file **conio.h**. La tabella elenca tali costanti simboliche ed i corrispondenti valori numerici. Si noti che per lo sfondo possono essere utilizzati solo i primi otto colori; i rimanenti (da 8 a 15) possono essere utilizzati solo per il primo piano (caratteri).

Se si vuole attivare l'attributo di lampeggio dei caratteri (primo piano) si deve utilizzare la costante simbolica **BLINK** (valore numerico 128).

Visualizzazione Ad Alta Velocità

Le funzioni di I/O di console del Borland C++ includono una variabile chiamata **directvideo**. Questa variabile controlla il fatto che l'output debba essere indirizzato alla RAM video direttamente (**directvideo** = 1) o tramite le routine standard del BIOS (**directvideo** = 0). Normalmente il valore della variabile **directvideo** è 1 (output diretto sulla RAM video). In generale, in questo modo si ottengono prestazioni superiori, ma è necessario operare su un PC compatibile IBM al 100%: l'hardware che governa l'output su schermo deve essere identico a quello degli adattatori video IBM.

L'output prodotto con **directvideo** = 0 viene visualizzato su qualunque PC compatibile IBM a livello di BIOS, ma la visualizzazione è notevolmente più lenta.

Programmazione In Modalità Grafica

Nel Borland C++ è presente una libreria di oltre 70 funzioni grafiche, che spazia dalle chiamate ad alto livello (come **setviewport**, **bar3d** e **drawpoly**) alle funzioni che operano sui bit (come **getimage**, **putimage**). La libreria permette di utilizzare numerosi modelli di riempimento e stili di linee, fornendo molti font di testo che possono essere dimensionati, orientati ed allineati a piacere sia in senso orizzontale che in senso verticale.

Tali funzioni sono contenute nel file di libreria **GRAPHICS.LIB**; i relativi prototipi si trovavano nel file **include ghphics.h**. Oltre a questi due file, il pacchetto grafico include alcuni driver grafici (i file *.BGI) ed i font dei caratteri (i file *.CHR). Per utilizzare la funzioni grafiche si vedano i passi seguenti. Se si sta utilizzando l' ambiente integrato (IDE - Integrated Development Environment), si deve attivare l' opzione Full Menus e quindi Options Linker Graphics Library. Quando si compila il programma, il linker utilizza automaticamente la libreria grafica del Borland C++.

Se invece si utilizza il compilatore BCC.EXE o BCCX.EXE, si deve indicare nel comando la libreria **GRAPHICS.LIB**. Ad esempio, se il programma MYPROG.C usa la routine grafiche, il comando da utilizzare è:

BCC MYPROG GRAPHICS.LIB

Attenzione !!!!

Poichè le funzioni grafiche utilizzano i puntatori, non sarà possibile utilizzare le funzioni grafiche nei modelli di memoria più piccoli. Vi è una sola libreria grafica e versioni distinte per ogni modello di memoria (a differenza delle librerie standard **CS.LIB**, **CC.LIB**, **CM.LIB** e così via , che dipendono dal modello di memoria utilizzando). Ogni funzione di **GRAPHICS.LIB** è una funzione e le funzioni grafiche che richiedono l' uso di puntatori, utilizzano puntatori **far**. Per fare in modo che tali funzioni operino correttamente, è importante includere in ogni modulo che utilizza le funzioni grafiche il file **graphics.h**.

Le Funzioni Della Libreria Grafica

Le funzioni grafiche del Borland C++ rientrano in sette categorie:

- controllo del sistema grafico;
- tracciamento e riempimento di aree;
- gestione dello schermo e dei viewport;
- output del testo;
- gestione degli errori;
- interrogazioni sullo stato del sistema.

Controllo Del Sistema Grafico

Ecco un breve elenco delle funzioni di controllo del sistema grafico.

closegraph	Disattiva il sistema grafico.
detectgraph	Verifica l' hardware per determinare il driver grafico da utilizzare; consiglia una modalità.
graphdefaults	Reinizializza tutte le variabili del sistema grafico ai vari colori.
_graphfreemem	Dealloca la memoria grafica: possibilità di definire routine personalizzate.

_graphgetmem	Alloca la memoria grafica: possibilità di definire routine personalizzate.
getgraphmode	Restituisce la modalità grafica corrispondente.
getmoderange	Restituisce le modalità valide (limite inferiore e superiore) per il driver grafico.
initgraph	Inizializza il sistema grafico e attiva l'hardware per la modalità grafica.
installuserdriver	Installa un driver fornito dall'utente alla tabella dei driver BGI.
installuserfont	Carica un file nella tabella di caratteri BGI.
registerbgidriver	Memorizza un driver utilizzato in fase di link o fornito dall'utente per includerlo al momento del link.
restorecrtmode	Ripristina la modalità di schermo originale (prima di initgraph).
setgraphbufsize	Specifica la modalità del driver grafico interno.
setgraphmode	Seleziona la modalità grafica da selezionare inizialmente, cancella lo schermo e ripristina la situazione standard.

Il sistema grafico del C++ fornisce i driver adatti ai seguenti adattatori (e compatibili):

- Color/Graphics Adapter (cga)
- Multi-Color Graphics Array (mcga)
- Enhanced Graphics Adapter (ega)
- Video Graphics Array (vga)
- Hercules Graphics Adapter
- AT&T 400 -line Graphics Adapter
- 3270 PC Graphics Adapter
- IBM 8514 Graphics Adapter.

Per inizializzare il sistema grafico, si deve richiamare la funzione **initgraph** che carica il driver grafico ed entra in modalità grafica. Mediante **initgraph** è possibile decidere di utilizzare un determinato driver grafico ed una determinata modalità o ricorrere al rilevamento automatico dell'adattatore video presente sul PC al momento dell'esecuzione del programma da caricare automaticamente il driver appropriato.

Se si ricorre al rilevamento automatico, **initgraph** richiamerà **detectgraph** per selezionare il driver grafico e la modalità da utilizzare. Se invece si indica ad **initgraph** il driver grafico e la modalità da utilizzare, è necessario essere certi che l'adattatore video prescelto sia effettivamente presente. Se si dovesse scegliere un adattatore video che non è presente sulla macchina, è possibile ottenere risultati imprevedibili.

Dopo il caricamento di un driver grafico, per conoscerne il nome si può utilizzare la funzione **getdrivername**, mentre per conoscerne il numero di modalità grafica ammessa dal driver, si utilizza la funzione **getmaxmode** che indica invece la modalità grafica attualmente in uso; dopo aver ottenuto il numero corrispondente alla modalità video, per conoscerne il nome si utilizza la funzione **getmodename**. Per cambiare modalità grafica si utilizza **setgraphmode**, mentre per tornare alla modalità video originale (prima che fosse inizializzato il sistema grafico) si utilizza **restorecrtmode** che riporta lo schermo alla modalità testo, ma non chiude il sistema grafico (i font e i driver rimangono in memoria). **getdefaults** reinizializza i valori standard del sistema

grafico (dimensioni dei **viewport**, colore utilizzato per la grafica, colore e modelli di riempimento della aree e così via. Le funzioni **installuserdriver** e **installuserfont** permettono di aggiungere nuovi driver e nuovi font al BGI (Borland Graphic Interface).

Infine, quando si è terminato di usare lo schermo in modalità grafica, per chiudere il sistema grafico si utilizza **closegraph** che libera la memoria occupata dal driver e ripristina la modalità video originale (con **restorecrtmode**).

Una Discussione Più Approfondita

Nelle precedenti parti è stata fornita una panoramica di **initgraph**. Ora verranno trattati in maggior dettaglio il comportamento di **initgraph**, **_graphgetmem** e **_graphfreemem**.

Normalmente, la routine **initgraph** carica un driver grafico allocando la memoria necessaria ed inserendovi l' appropriato file .BGI. Come alternativa a questo schema di caricamento dinamico, si può effettuare il linking del driver grafico (o di più driver grafici) direttamente all' interno del programma. A tale scopo si deve convertire il file .BGI in un file .OBJ (mediante il programma di servizio BGIOBJ; si veda il file UTIL.DOC) e quindi inserire nel codice sorgente una o più chiamate a **registerbgidriver** (prima di richiamare **initgraph**) in modo da registrare i driver grafici. Quando si deve costruire il programma, è necessario effettuare il linking del file .OBJ relativo ai driver registrati. Dopo aver determinato il driver grafico da utilizzare (mediante **detectgraph**), **initgraph** verifica che si registrato il driver desiderato. In caso affermativo, **initgraph** usa tale driver direttamente dalla memoria, altrimenti alloca la memoria necessaria per il driver e carica il file .BGI.

L' uso di **registerbgidriver** costituisce una tecnica di programmazione avanzata sconsigliata ai non esperti: questa è la stessa tecnica con cui sono lavorano i programmi G.I.P. e SPRITE. Al momento dell' esecuzione del programma, il sistema grafico può dover allocare altra memoria per i driver, i font ed i buffer interni. In questo caso, per allocare la memoria richiama **_graphgetmem**, mentre per renderla nuovamente disponibile richiama **_graphfreemem**. Nonostante queste routine richiama semplicemente **malloc** e **free** rispettivamente, si possono utilizzare routine personalizzate delle funzioni **_graphgetmem** e **graphfreemem**; per fare ciò si dovranno predisporre le appropriate routine utilizzando però gli stessi nomi delle routine originali. Per l' allocazione della memoria verranno utilizzate le routine fornite dall' utente, mentre verranno ignorate le routine di libreria standard del C.

Tracciamento E Riempimento Di Aree

Ecco un breve riassunto delle funzioni che si occupano del tracciamento e del riempimento di aree:

TRACCIAMENTO

arc	Traccia un arco di circonferenza.
circle	Traccia un cerchio.
drawpoly	Traccia il perimetro di un poligono.

	ellipse	Traccia un arco ellittico.
	getrccords	Restituisce le coordinate dell' ultima chiamata ad arc o ellipse .
grafica	getspectratio	Restituisce il rapporto bidimensionale utilizzando dalla modalità corrispondente.
	getlinesettings	Restituisce lo stile, il tipo e lo spessore della linea.
	line	Traccia una linea da (x0,y0) a (x1,y1).
posizione	linerel	Traccia una linea ad un punto posto ad una distanza relativa alla corrente (CP).
	lineto	Traccia una linea dalla posizione corrente (CP) (x,y).
	moveto	Sposta la posizione corrente (CP) a (x,y).
	moverel	Sposta la posizione corrente (CP) ad una distanza relativa.
	rectangle	Traccia un rettangolo.
	setaspectratio	Modifica il fattore di correzione del rapporto dimensionale.
	setlinestyle	Definisce la larghezza e lo stile della line.

RIEMPIMENTO DI AREE

	bar	Traccia e riempie una barra.
	bar3d	Traccia e riempie una barra 3d.
	fillellipse	Traccia a riempie un' ellisse.
	fillpoly	Traccia e riempie un poligono.
	floodfill	Riempie una ragione delimitata dallo schermo.
	getfillpattern	Restituisce il modello di riempimento definito dall' utente.
colore	getfillsettings	Restituisce alcune informazioni sul modello di riempimento e sul corrente.
	pieslice	Traccia e riempie un settore circolare.
	setcolor	Traccia e riempie un settore ellittico.
	setfillpattern	Seleziona un modello di riempimento definito dall' utente.
	setfillstyle	Definisce il modello ed il colore di riempimento.

Con le funzioni di riempimento e disegno fornite dalla Borland C++, è possibile tracciare e colorare linee, archi, cerchi, ellissi, rettangoli, settori circolari, barre bi-tridimensionali, poligoni e forme rettangolari o irregolari costituiti da più oggetti semplici. Si può inoltre riempire ogni area delimitata (o una qualunque regione che la circonda) con uno qualunque degli 11 modelli predefiniti o con un modello definito dell' utente. E' infine possibile controllare lo spessore e lo stile della linea ed il punto in cui si trova la posizione corrente (CP).

Per tracciare le linee e le forma vuote si utilizzano le funzioni: **arc**, **circle**, **drawpoly**, **ellipse**, **line**, **linerel**, **lineto** e **rectangle**. Si possono riempire queste forme con **floodfill** o utilizzare combinazioni di tracciamento e riempimento e di aree con **bar**, **bar3d**, **fillellipse**, **pieslice** e **sector**. Con **setlinestyle** si specifica lo stile di tracciamento di una linea (e del bordo nel caso di forma riempite): ad esempio è possibile utilizzare linee sottili o spesse, selezionare uno stile o trattamento o un altro modello dall' utente. Si può selezionare un modello di riempimento predefinito tramite **setfillstyle** e definire uno proprio con **setfillpattern**.

Per spostare CP in una posizione ben determinata si utilizzando **moveto** e **moverel**. Per

conoscere lo stile e lo spessore della linea, si utilizza **getlinesettings** e per informazioni relative al modello ed al colore di riempimento correnti, si utilizza **getfillsettings**; è possibile ottenere informazioni sul modello di riempimento definito dall' utente tramite la funzione **getfillpattern**.

Per conoscere il rapporto dimensionale utilizzato (il fattore di scala usato dal sistema grafico per assicurarsi che i cerchi siano effettivamente rotondi) si utilizza **getaspetratio**, mentre per conoscere le coordinate dell' ultimo arco o ellisse tracciata, si usa **getarccoords**. Se i cerchi non sono perfettamente rotondi, si usi **setaspectratio**.

Gestione Dello Schermo E Del ViewPort

Di seguito viene presentato una delle funzioni di gestione dello schermo, del **viewport**, dell' immagine e dei pixel.

cleardevice	Cancella lo schermo (pagina attiva).
setactivepage	Definisce la pagina attiva per l' output grafico.
setvisualpage	Definisce il numero della pagina grafica.
clearviewport	Cancella il viewport corrente.
getviewsettings	Restituisce alcune informazioni relative al viewport corrente.
setviewport	Definisce il viewport corrente per l'otuput grafico.
getimage	Salva in memoria un'immagine contenuta in una regione specificata.
imagesize	Restituisce il numero di byte richiesti per memorizzare un' area rettangolare dello schermo.
putimage	Visualizza un' immagine precedentemente salvata.
getpixel	Fornisce il colore del pixel posto in (x,y).
putpixel	Accende un pixel in (x,y).

Oltre alle funzioni di tracciamento e disegno, la libreria grafica offre molte altre funzioni per la gestione dello schermo, il **viewport**, le immagini e i pixel. Si può cancellare l' intero schermo con una chiamata a **cleardevice**; questa routine cancella lo schermo e porta la posizione corrente (CP) nell' origine del viewport, ma lascia inalterate tutte la altre impostazioni del sistema grafico (gli stili delle linee, il modello di riempimento, lo stile del testo, la palette, il viewport e così via).

A seconda dell' adattatore grafico utilizzando, il sistema può disporre di un numero variabile (da uno a quattro) di buffer di schermo; di buffer di schermo: i buffer sono punto a punto le immagini dello schermo. Inoltre, per specificare la pagine) e la pagina visibile (quella attualmente visualizza sullo schermo) si utilizzano rispettivamente le funzioni **setactivepage** e **setvisualpage**.

Una volta che lo schermo è in modalità grafica, è possibile definire un **viewport** (uno "schermo virtuale" rettangolare) mediante la funzione **setviewport**. Si deve definire la posizione del viewport in termini di coordinate assolute dello schermo e specificare se deve essere attivo o meno; per cancellare il **viewport** si utilizza **clearviewport**. Per conoscere le coordinate assolute del viewport e lo stato di visualizzazione si utilizza **getviewsettings**. Le coordinate per tutte le funzioni di output sullo schermo (tracciamento, riempimento di aree, testo e così via) sono relative all' origine del viewport. Per gestire il colore dei pixel che compongono un disegno, si utilizzano le funzioni **getpixel** (che restituisce il colore di un pixel) e **putpixel** (che accende un

pixel con un determinato colore).

Output Del Testo In Modalità Grafica

Ecco un breve riassunto delle funzioni di output del testo in modalità grafica:

gettextsettings correnti	Restituisce il font, la direzione, le dimensioni e la giustificazione nel testo.
outtext	Invia sullo schermo una stringa alla posizione corrente(CP).
outtextxy	Invia sullo schermo una stringa alla posizione specificata.
registerbfont	Registra un font collegato o caricato dall' utente.
settextjustify	Definisce i valori di giustificazione del testo utilizzati da outtext e outtextxy .
settextstyle	Definisce il font, lo stile ed il fattore di ingrandimento del carattere.
setusercharsize	Definisce i rapporti di larghezza ed altezza per i font.
textheight	Restituisce altezza di una stringa in pixel.
textwidth	Restituisce la larghezza di una stringa in pixel.

La libreria grafica include un font bit-mapped 8x8 e molti altri font per l' output del testo in modalità grafica.

- In un font bit-mapped, ogni carattere è definito mediante una matrice di pixel.
- Nel caso di altri font, ogni carattere è costituito da una serie di vettori che dicono al sistema grafico come deve essere costituito il carattere.

Il vantaggio derivante dall' uso di questi ultimi è evidente quando si tratta di utilizzare caratteri di grandi dimensioni. I font costruiti tramite vettori mantengono una buona risoluzione anche quando vengono notevolmente ingranditi. Se invece si ingrandisce un font bit-mapped viene semplicemente moltiplicata la matrice per il fattore di scala; mano a mano che il fattore di scala cresce, la risoluzione dei caratteri diviene sempre più scarsa. Per caratteri di piccole dimensioni, i font bit-mapped potrebbero essere sufficienti, ma per i caratteri di maggiori dimensioni si consiglia di utilizzare i font costruiti tramite vettori.

Per visualizzare del testo grafico possono utilizzare le funzioni **outtext** e **outtextxy**, mentre per controllarne la giustificazione rispetto alla posizione corrente (CP) si utilizza **settextjustify**. Per scegliere il font, la direzione (orizzontale e verticale) e le dimensioni (scala) del testo si utilizza **settextstyle**. Per conoscere le impostazioni correnti relative al testo si utilizza **gettextsettings**, che restituisce il font, la giustificazione, il fattore di ingrandimento e la direzione del testo corrente in una struttura **textsettings**; **setusercharsize** permette di modificare la larghezza e l'altezza dei font costruiti tramite vettori. Se la funzione di clipping (troncamento) è on, la stringa di testo scritta con le funzioni **outtext** e **outtextxy** verranno troncate al margine del viewport. Se la funzione di clipping è off, se una parte di stringa dovesse oltrepassare i margini del viewport, nel caso di font bit-mapped queste due funzioni tagliano i caratteri se qualsiasi parte della stringa dovesse uscire dallo schermo; nel caso di font vettoriali, i caratteri in eccesso vengono troncati. Per determinare la dimensioni di una stringa di testo, si utilizzano

textheight (che misura l'altezza in pixel della stringa) e **textwidth** (che misura la larghezza in

pixel della stringa). Il font bit-mapped standard 8x8 è contenuto nel pacchetto grafico, quindi è sempre disponibile al momento dell' esecuzione. I font vettoriali sono contenuti in un file .CHR distinti; questi file possono essere caricati al momento dell' esecuzione o convertiti in file .OBJ (mediante il programma di utilità BGIOBJ) e quindi collegati tramite linking al file .EXE. Normalmente, la routine **setttextstyle** carica un file di font allocando la memoria necessaria ed inserendovi l' appropriato file .CHR. Come alternativa a questo schema di caricamento dinamico, è possibile effettuare il linking del file di font (o di più file font) direttamente all'interno del programma. A tale scopo si deve convertire il file.CHR in un file .OBJ (mediante il programma di servizio BGIOBJ, si veda a questo proposito il file UTIL.DOC) e quindi inserire nel codice sorgente una o più chiamate a **registerbgifont** (prima di richiamare **setttextstyle**), in modo da registrare i font di caratteri. Quando si deve costruire il programma, è necessario effettuare il linking del file .OBJ relativo ai font registrati. L' uso di **registerbgifont** costituisce una tecnica di programmazione avanzata sconsigliata ai non esperti; per ulteriori informazioni su questa funzione, si veda il file UTIL.DOC.

Controllo Del Colore

Di seguito viene presentato un breve riassunto delle funzioni di controllo del colore:

getbkcolor	Restituisce il colore attuale del fondo.
getcolor	Restituisce il colore attuale utilizzato per il disegno.
getdefaultpalette	Restituisce la struttura di definizione della palette.
getmaxcolor	Restituisce il numero massimo dei colori disponibili nella modalità grafica corrente.
getpalette	Restituisce al palette corrente e le sue dimensioni.
getpalettesize	Restituisce la dimensione della tabella di consultazione della palette.
setallpalette	Modifica i colori della palette nel modo specificato.
setbkcolor	Definisce il colore attuale dello sfondo.
setcolor	Definisce il colore attuale per il disegno.
setpalette	Modifica un colore della palette come specificato dagli argomenti.

Prima di presentare l' introduzione alle funzioni di controllo del colore, presenteremo una descrizione del modo in cui questi colori sono effettivamente prodotti.

Pixel E Palette

Lo schermo grafico è composto da una serie di pixel; ognuno corrisponde ad un punto dello schermo. Il valore dei pixel non specifica direttamente il colore all'interno di una tabella chiamata palette (tavolozza); il colore utilizzato da tale pixel è indicato dal valore contenuto nella tabella. Questo schema indetto ha un certo numero di applicazioni. Anche se l' hardware può visualizzare molti colori, sullo schermo ne potrà essere visualizzata contemporaneamente solo una parte di essi. Il numero di colori visualizzati è uguale al numero di elementi che compongono la palette, ovvero della sua dimensione.

Ad esempio, con una scheda EGA, l'hardware può visualizzare fino a 64 colori, ma solo 16 per

volta; la dimensione della palette EGA è quindi 16. La dimensione della palette determina il numero di valori che un pixel può assumere (da 0 a dimensione -1). La funzione `getmaxcolor` restituisce il massimo colore ammesso per un pixel (dimensione -1) in base al driver ed alla modalità video utilizzata. Quando si parla delle funzioni grafiche disponibili dalla Borland, si utilizza spesso il termine colore, come il colore corrente del disegno, colore di riempimento e colore del pixel. In effetti, il colore è un valore di un pixel, ovvero un elemento della palette: la palette determina il colore che apparirà sullo schermo. Se si modifica la tabella della palette, si possono modificare i colori visualizzati sullo schermo senza modificare il valore del pixel.

Colori Dello Sfondo E Del Disegno

Il colore dello sfondo corrisponde, sempre ad un pixel contenente il valore 0. Quando si cancella un' area dello sfondo, ogni pixel assume il colore dello sfondo e dunque in tutti i pixel dell' area viene memorizzato il valore 0. Il colore del disegno è il valore assunto da un pixel quando si deve tracciare una linea; per scegliere un colore del disegno si utilizza **`setcolor(n)`**, dove `n` deve essere un valore valido per la palette corrente.

Controllo Del Colore Con Una Scheda

A causa di alcune differenze in modi hardware, il controllo del colore è leggermente diverso in CGA e EGA e dunque tali modalità grafiche sono trattate in modo diverse. Il controllo del colore dei driver AT&T ed NCGA a bassa risoluzione è simile a quello CGA. Su una scheda CGA è possibile scegliere la visualizzazione a bassa risoluzione (320x200) a quattro colori o ad alta risoluzione (640x200) a due colori.

CGA A Bassa Risoluzione

Nella modalità a bassa risoluzione, si può scegliere una palette tra le quattro standard. Con tali palette, è possibile solo definire il primo elemento della stessa; elementi 1, 2 e 3 sono fissi. Il primo elemento della palette (il colore 0) è quello dello sfondo, che può essere scelto tra uno dei 16 colori disponibili (si veda la tabella relativa ai colori per lo sfondo in CGA). Per scegliere la palette si deve optare per una delle modalità disponibili (CGACO, CGAC1, CGAC2, CGAC3) che utilizzano i colori illustrati in CGA e le relative costanti sono definite nel file **`include graphics.h`**.

Costanti Assegnate Al Numero Di Colore (Valore Del Pixel)

NUMERO	1	2	3
PALETTE			
0	CGA_LIGHTGREEN	CGA_LIGHTRED	
	CGA_YELLOW		

1	CGA_LIGNTCYAN	CGA_LINGHTMAGENTA	CGA_WHITE
2	CGA_GREEN	CGA_RED	CGA_BROWN
3	CGA_CYAN	CGA_MAGENTA	
	CGA_LIGHTGRAY		

Per utilizzare uno di questi colori come colore del disegno CGA, si richiami la funzione **setcolor** specificando il numero del colore o la costante corrispondente; ad esempio, se si utilizza la palette 3 e si vuole scegliere per il disegno il colore ciano, si dovrà utilizzare:

```
setcolor (1)
o
setcolor (CGA_CYAN)
```

I colori disponibili per lo sfondo in CGA, definiti in **graphics.h**, sono elencati nella tabella seguente.

VALORE NUMERICO	NOME SIMBOLICO	VALORE NUMERICO	NOME SIMBOLICO
0	BLACK	8	DARKGRAY
1	BLUE	9	LIGHTBLUE
2	GREEN	10	LIGHTGREEN
3	CYAN	11	LIGHTCYAN
4	RED	12	LIGHTRED
5	MAGENTA	13	LIGHTMAGENTA
6	BROWN	14	YELLOW
7	LIGHTGRAY	15	WHITE

Per assegnare al colore dello sfondo uno di tali colori, si deve utilizzare la funzione **setbkcolor(colore)**, dove colore è uno degli elementi della tabella precedente. Nel caso CGA, il colore non è il valore del pixel, ma specifica direttamente il vero colore che deve essere utilizzato come primo elemento della palette.

CGA Alta Risoluzione

In modalità ad alta (640x200), la scheda CGA visualizza due colori: uno sfondo nero ed un primo piano colorato. I pixel possono assumere i valori 0 o 1. A causa di una caratteristica particolare della CGA, il colore del primo piano è effettivamente quello che l'hardware pensa sia il colore dello schermo; per definirlo si utilizza la routine **setbkcolor**. I colori disponibili per il primo piano sono gli stessi elencati nella tabella precedente; tale colore viene usato per tutti i pixel contenenti il valore 1. La modalità che si comportano in questo modo sono CGAHI, MCGAMED, ATT400MED e ATT400HI.

Routine Operanti Sulla Palette CGA

Poiché la disponibilità dei colori nella CGA è predeterminata, non è necessario utilizzare la funzione **setallpalette**. Inoltre non si deve utilizzare la routine **setpalette(indice,colore_effettivo)**, se si esclude indice = 0 (questo è un modo alternativo per definire dello sfondo con il valore colore_effettivo).

Controllo Del Colore In EGA E VGA

In EGA, la palette contiene 16 elementi su un totale di 64 possibili e ogni elemento è definibile dall'utente. Per conoscere la palette utilizzata si usa la funzione **getpalette**, che compila in una struttura la dimensioni della palette (16) ed un array contenente gli effettivi elementi della palette (i valori "hardware" memorizzati nella palette). Si possono modificare gli elementi della palette uno ad uno con

setpalette o tutti insieme con **setallpalette**. La palette EGA standard è composta dai 16 colori CGA, presentati nella tabella precedente: il nero (black) è l' elemento 0, il blu è elemento 1, il bianco (white) è l' elemento 15. Nel file **graphics**, sono definite tutte le costanti relative a tali colori: **EGA_BLACK**, **EGA_WHITE** e così via. Per ottenere informazioni su tali valori è possibile utilizzare la funzione **getpalette**.

La routine **setbkcolor(colore)** è diversa in EGA rispetto al caso CGA. In EGA, **setbkcolor** copia il valore presente nell' elemento numero colore nell' elemento 0. Per quanto riguarda i colori, il driver VGA si comporta esattamente come il driver EGA; ciò che cambia è la definizione (quindi le dimensioni dei pixel).

Gestione Degli Errori In Modalità Grafica

Di seguito viene riportata un breve riassunto delle funzioni di gestione degli errori in modalità grafica.

grapherrormsg	Restituisce un messaggio di errore relativo al codice di errore specificato.
graphresult che ha	Restituisce un codice di errore relativo all' ultima operazione grafica rivelato un problema.

Se durante la chiamata di una funzione grafica si verifica un errore, ad esempio perché non viene trovato un file di font richiesto tramite **settextstyle**, viene attivato un codice di errore dell' ultima funzione grafica che ha riscontrato dei problemi, si utilizza **graphresult**. Una chiamata a **grapherrormsg(graphresult())** restituisce un messaggio di errore tra quelli elencati nella seguente tabella. I codice di errore cambiato solo quando un funzione grafica rivela un errore. Il codice di errore viene reinizializzato a 0 solo quando si esegue (con successo) **inigraph** o quando si utilizza **graphresult**. Quindi, se si vuole conoscere la funzione grafica che ha prodotto il codice di errore, si deve memorizzare il valore di **graphresult** in una variabile temporanea e controllarla.

CODICE	COSTANTE	CORRISPONDENTE
ERRORE	GRAPHICS_ERRORS	MESSAGGIO DI ERRORE

0	grOk	No error.
-1	grNotIntGraph	(BGI) graphics not installed (use inigraph).
-2	grNotDetected	Graphics hardware not detected.
-3	grFileNotFound	Device driver file not found.
-4	grInvalidDriver	Invalid device driver file.
-5	grNoLoadMem	No enough memory to load driver.
-6	grNoScanMem	Out of memory in scan fill.
-7	grNoFloodMem	Out of memory in flood fill.
-8	grFontNotFound	Font file not found.
-9	grNoFontMem	Not enough memory to load font.
-10	grInvalidMode	Invalid graphics mode for selected driver.
-11	grError	Graphic error.
-12	grIoError	Graphic I/O error.
-13	grInvalidFont	Invalid font file.
-14	grInvalidFontNum	Invalid font number.
-15	grInvalidDeviceNum	Invalid device number.
-18	grInvalidVersion	Invalid version of file.

Interrogazione Sullo Stato Del Sistema

La tabella seguente riassume le funzioni di interrogazione sullo stato del sistema in modalità grafica.

FUNZIONE	VALORE RESTITUITO
getarccoords	Informazioni relative all'ultima chiamata ad arc o ellipse .
getaspectratio	Rapporto dimensionale del rapporto grafico.
getbkcolor	Valore attuale dello sfondo.
getdrivername	Nome del driver corrente.
getcolor	Colore corrente del disegno.
getfillpattern	Nome del driver grafico corrente.
getfillsettings	Modello di riempimento definito dall'utente.
getgraphmode	Informazioni relative al modello di riempimento a al
modello	corrente.
getlinesettings	Modalità grafica corrente.
getmaxcolor	Stile, tipo e spessore della linea corrente.
getmaxmode	Numero di modalità ammesse dal driver corrente.
getmaxx	Risoluzione x corrente.
getmaxy	Risoluzione y corrente.
getmodename	Nome della modalità utilizzata
getmoderange	Modalità ammesse per un dato driver.
getpalette	Palette corrente e sue dimensioni.
getpixel	Colore del pixel x,y.
gettextsettings	Font, direzione, dimensione e giustificazione corrente del
testo.	

getviewsettings	Informazioni relative al viewport corrente.
getx	Coordinata x nella posizione corrente. (CP)
gety	Coordinata y nelle posizione corrente. (CP)

Per ogni categorie di funzione grafica di Borland C++ vi è almeno una funzione di interrogazione sullo stato del sistema; per ulteriori informazioni su queste funzioni, si consultino le categorie a cui esse appartengono. Ogni funzione di interrogazione sullo stato del sistema del Borland C++ (tranne le funzioni di gestione degli errori) si chiama **getqualcosa**. Alcune di esse non richiedono argomenti e restituiscono un valore che rappresenta l' informazione richiesta: altre richiedono un puntatore ad una struttura definita in **graphics.h**, compilano tale struttura con le appropriate informazioni e non restituiscono alcun valore. Le funzioni di interrogazione sullo stato del sistema della categoria di controllo del sistema grafico sono **getgraphmode**, **getamaxmode** e **getmoderange**: la prima restituisce un intero che rappresenta il numero di modalità per un determinato driver ed il terzo restituisce le modalità ammesse da un determinato driver grafico. Le **getmaxx** e **getmaxy** restituiscono le coordinate x e y massime ammesse per la modalità grafica corrente.

Le funzioni di interrogazione sullo stato del sistema relative al tracciamento di disegni e al riempimento delle aree sono **getarccoords**, **getaspectratio**, **getfillpattern**, **getfillpattern**, **getfillsetting** e **getlinesetting**, **getarccoords** compila una struttura con le coordinate utilizzate per l'ultima chiamata ad una funzione **arc** o **ellipse**; **getaspectratio** indica il rapporto di dimensionale corrente, necessario per tracciare cerchi effettivamente rotondi; **getfillpattern** restituisce il modello di riempimento corrente definito dall'utente, **getfillsettings** compila una struttura con il modello e il colore di riempimento correnti, **getlinesettings** riempie una struttura con lo stile della linea (normale o spessa) ed il tipo do linea.

Le funzioni di interrogazione sullo stato del sistema nel campo della gestione dello schermo e dei viewport sono **getviewsettings**, **getx**, **gety** e **getpixel**.

Dopo aver definito un viewport, si può conoscere la sua posizione in termini di coordinate assolute dello schermo e se la funzione di clipping (troncamento) è attivata richiamando **getviewsettings**, che compila una struttura con le relativa informazioni; **getx** e **gety** restituiscono la coordinate x ed y (relativa al viewport) della posizione corrente (CP); **getpixel** restituisce il colore di un pixel.

Le categoria di controllo del colore di Borland C++ include tre funzioni di interrogazione sullo sfondo del sistema: **getbkcolor** restituisce il colore corrente del disegno; **getpalette** compila una struttura contenente le dimensioni della palette ed il suo contenuto; **getmaxcolor** restituisce il massimo valore assegnabile ad un pixel per il driver grafico e la modalità e del driver grafico correnti.

Il Turbo Assembler

Compatibilità

Il Turbo Assembler (TASM) è quasi totalmente compatibile con il Macro Assembler (MASM). Il primo fornisce parecchie caratteristiche che aiutano il programmatore e consentono l'interfacciamento con il Turbo Pascal o con il Turbo C molto più rapido e semplice.

Introduzione

Uno dei luoghi comuni dell'informatica è che il linguaggio Assembler sia un'arte alla portata di pochi programmatori, ma come tutti sanno, i luoghi comuni spesso non sono altro che pregiudizi. Il linguaggio Assembler è una rappresentazione comprensibile all'uomo del linguaggio del computer: è molto potente, il solo in grado di sfruttare al meglio le potenzialità della famiglia dei processori Intel 80x86, che sono il cuore della famiglia dei PC IBM e dei suoi compatibili.

Si possono scrivere interi programmi in linguaggio Assembler o, se necessario, integrarlo con programmi scritti in Borland C++, Turbo Pascal ed altri linguaggi: in entrambi i modi, con il linguaggio Assembler si possono scrivere programmi brevi e veloci. Importante come la velocità è l'abilità del codice del linguaggio Assembler di controllare ogni aspetto delle operazioni del computer, fino agli impulsi dell'orologio di sistema.

Verrà introdotto il linguaggio Assembler esaminandone le caratteristiche peculiari della sua programmazione: seguendo gli esempi contenuti sarà possibile immettere ed eseguire numerosi programmi in linguaggio Assembler, sia per avere un'idea della potenzialità di tale linguaggio che per abituarsi a lavorare con esso.

Scrittura Del Primo Programma In Turbo Assembler

Nel mondo della programmazione, il primo programma tradizionalmente visualizza il messaggio, "Hello, world" ed è considerato un buon punto di partenza.

Si scelga un editor di testi che sia in grado di produrre file ASCII e si digitino le righe seguenti che costituiscono il programma HELLO.ASM.

```
.MO__~ZL. small
.STACK iOOh
.DATA
HelloMessage DB 'Hello, world',13,10,'$'
.CODE
mov AX,@DATA
mov ds,ax
mov ah,9
mov dx,OFFSET HelloMessage
int 21h
mov ah,4ch
int 21h
END
```

;imposta DS per puntare al segmento dati
;funzione di stampa di DOS
;punta a "Hello, world"
;visualizza "Hello, world"
;funzione DOS che termina il programma
;termina il programma

Dopo aver immesso HELLO.ASM, è necessario salvarlo su disco. Se si ha familiarità con C, C++

o Pascal, si potrebbe pensare che la versione in linguaggio Assembler di "Hello, world" sia un po' lunga; questo è vero, i programmi in linguaggio Assembler tendono ad essere lunghi in quanto ogni singola istruzione esegue meno operazioni di una istruzione in C, C++ o Pascal. D' altra parte, si è liberi di combinare le istruzioni in linguaggio Assembler nel modo desiderato; questo significa che, a differenza di C o di Pascal, il linguaggio Assembler consente di programmare il computer per far in modo che esegua qualsiasi operazione.

Assemblaggio

Dopo aver salvato HELLO.ASM, prima di eseguirlo è necessario convertirlo in una forma eseguibile; questo richiede due operazioni, l' assemblaggio e l' operazione di link, come mostrato in figura, che illustra l' intero ciclo di sviluppo comprendente: la scrittura, l' assemblaggio, l' operazione di link e l' esecuzione.

μ §

La fase di assemblaggio converte il programma sorgente in una forma intermedia chiamata modulo oggetto, mentre la fase di link combina uno o più moduli oggetto in un programma eseguibile; queste due operazioni si possono eseguire dalla riga di comando.

Per assemblare HELLO.ASM, si digiti:

TASM Hello

A meno che non venga specificato un altro nome di file, HELLO.ASM viene assemblato in HELLO.OBJ; si noti che non è necessario digitare l' estensione in quanto Turbo Assembler assume .ASM. Questo è quanto compare sullo schermo:

Turbo Assembler Version 3.0 Copyright (c) 1988, 1991 by Borland
International Inc.

Assembling file:	HELLO.ASM
Error messages:	None
Warning messages:	None
Passes:	1
Remaining memory:	266K

Se HELLO.ASM è stato correttamente digitato non compariranno messaggi od errori; se così non dovesse essere, questi compariranno sullo schermo associati al numero di riga in cui si sono verificati; in questo caso si controllino le istruzioni assicurandosi che siano precisamente identiche a quelle riportate, quindi si assembli nuovamente il programma.

Fase Di Link

Dopo aver assemblato HELLO.ASM, è possibile eseguirlo; infatti dopo aver eseguito la fase di link del modulo oggetto assemblato in una forma eseguibile, è possibile eseguire il programma.

Per eseguire la fase di link, è necessario utilizzare TLINK, il linker di Turbo Assembler. Sulla linea di comando si digiti:

TLINK Hello

Anche in questo caso non occorre digitare l'estensione, poichè TLINK assume .OBJ. Quando la fase di linking viene completata, il linker assegna automaticamente al file .EXE lo stesso nome del file oggetto, a meno che non venga specificato diversamente. Se l'operazione viene eseguita con successo, sullo schermo appare il seguente messaggio:

Turbo Link Version 3.0 Copyright (c) 1987, 1991 by Borland
International, Inc.

E' possibile che si verifichino degli errori durante la fase di link, sebbene sia improbabile con questo programma d'esempio. Se sono presenti errori sullo schermo, si modifichi il programma per fare in modo che sia identico a quello riportato e si eseguano nuovamente le operazioni di assemblaggio e di link.

Esecuzione

A questo punto è possibile eseguire il programma; si digiti HELLO sulla linea di comando di DOS, il messaggio:

Hello, world

appare sullo schermo; questo è il primo programma in Turbo Assembler creato ed eseguito.

Che Cosa E' Successo ?

Ora che il file HELLO.ASM è stato creato ed eseguito, è necessario rivedere attentamente cosa è successo durante tutte le operazioni, dall'immissione del testo all'esecuzione del programma.

Quando HELLO.ASM è stato assemblato, Turbo Assembler ha convertito le istruzioni di testo nei loro equivalenti binari nel file oggetto HELLO.OBJ. HELLO.OBJ è un file intermedio, a metà strada tra il programma sorgente ed un file eseguibile; inoltre contiene tutte le informazioni necessarie per creare un codice eseguibile dalle istruzioni contenute in HELLO.ASM ed è in una forma tale, che può, essere facilmente combinata con altri file oggetto in un singolo programma.

Dopo la fase di link di HELLO.OBJ, TLINK lo converte nel file eseguibile HELLO.EXE; finalmente è possibile eseguire HELLO.EXE dopo aver digitato HELLO in risposta al prompt.

Ora si digiti:

DIR HELLO.*

per visualizzare l'elenco dei diversi file HELLO contenuti nel disco, sono presenti:

HELLO.ASM, HELLO.OBJ, HELLO.EXE ed HELLO.MAP.

Modifica Del Primo Programma In Turbo Assembler

Ora è necessario tornare all' editor e modificare il programma per fare in modo che accetti un input dall' utente; si modifichino le istruzioni nel seguente modo:

```
.MODEL small
.STACK 100h
TimePrompt DB 'Is it after 12 noon (Y/N) ?$'
GoodMorningMessage LABEL BYTE
    DB 13,10,'Good morning, world! ',13,10,'$'
GoodAfternoonMessage LABEL BYTE
    DB 13,10,'Good afternoon, world! ',13,10,'$'
.CODE
mov ax,@data
mov ds,ax                ;imposta DS per puntare al segmento dati
dati
    mov dx,OFFSET TimePrompt        ;punta richiesta prompt dell' ora
    mov ah,9                        ;funzione di stringa di stampa di DOS
    int 21h                         ;visualizza richiesta prompt ora
    mov ah,1                        ;funzione accettazione caratteri DOS
    int 21h                         ;accetta singolo carattere
    cmp al,'y'                      ;digitato Y minuscolo per pomeriggio?
    jz IsAfternoon                  ;si, è pomeriggio
    cmp al,'Y'                      ;digitato Y maiuscolo per pomeriggio?
    jnz IsMorning                   ;no, e' mattino

IsAfternoon
    mov dx,OFFSET GoodAfternoonMessage ;punta al saluto del pomeriggio
    jmp DisplayGreeting

IsMorning
    mov dx,OFFSET GoodMorningMessage   ;punta al saluto del mattino

DisplayGreeting
    mov ah,9                            ;funzione di stampa di DOS
    int 21h                             ;visualizza il saluto appropriato
    mov ah,4ch                          ;funzione DOS termina il programma
    int 21h                             ;termina il programma
END
```

Sono state aggiunte due nuove funzioni al programma: l' input e la possibilità di prendere decisioni; questo programma chiede se è passata l' ora del mezzogiorno, accettando una risposta di un solo carattere dalla tastiera; se il carattere digitato è una Y maiuscola o minuscola, visualizza un saluto appropriato per il pomeriggio mentre nel caso contrario visualizza un saluto riferito al mattino.

In questo semplice esempio sono presenti tutti gli elementi essenziali che rendono utile un programma: l'input, l'elaborazione dei dati, la possibilità di prendere decisioni e l'output.

Si salvi il programma modificato su disco; si esegua nuovamente la fase di assemblaggio e di link come negli esempi precedenti e a questo punto si esegua il programma digitando HELLO sulla riga di comando di DOS.

Il messaggio:

Is it after 12 noon (Y/N)?

viene visualizzato con il cursore luminoso posizionato dopo il punto interrogativo, in attesa di una risposta. Si preme Y, ed il programma risponderà:

Good afternoon, world!

Ora HELLO.ASM è un programma interattivo che ha la possibilità di prendere decisioni.

Durante la scrittura di programmi in linguaggio Assembler, è normale commettere errori di ortografia e di sintassi; Turbo Assembler, durante la fase di assemblaggio, individua e visualizza gli errori presenti. Tali errori possono essere classificati in due categorie: avvertimenti ed errori. Turbo Assembler visualizza i messaggi di avvertimento se individua qualcosa di sospetto ma non necessariamente errato, nelle istruzioni; qualche volta gli avvertimenti possono essere ignorati ma è sempre meglio controllarli, assicurandosi di aver capito il problema. Turbo Assembler visualizza un messaggio di errore se incontra un vero e proprio errore nel codice che impedisce di completare la fase di assemblaggio e di generazione del file oggetto.

Gli avvertimenti sono messaggi ammonitori e non gravi, mentre gli errori devono essere corretti prima di poter eseguire un programma.

Come qualsiasi altro linguaggio di programmazione, Turbo Assembler non è in grado di individuare gli errori logici; infatti può evidenziare se il programma può essere assemblato, ma non è capace di comunicare se il codice assemblato eseguirà le operazioni per le quali è stato creato, compito prettamente del programmatore. Per trasmettere il programma alla stampante, si consulti il manuale specifico del text editor; i file sorgente di Turbo Assembler sono normalmente dei file testo ASCII, che possono essere stampati dalla riga di comando del DOS utilizzando il comando PRINT.

Invio Dell' Output Alla Stampante

La stampante è un' unità di output; talvolta non solo è possibile che il programmatore voglia stampare i file sorgenti, ma far sì che i programmi stessi invino output alla stampante. Quanto segue è una versione di "Hello, world", che trasmette il suo output alla stampante e non al video.

.MODEL small

```

.STACK
.DATA
HelloMessage DB 'Hello, world',13,10,12
HELLO_MESSAGE_LENGTH EQU $-HelloMessage
.CODE
mov ax,@data
mov ds,ax                ;imposta DS per puntare al segmento dati
mov ah,40h               ;funzione DOS scrittura sull' unità
mov bx,4                 ;handle della stampante
mov cx,HELLO_MESSAGE_LENGTH ;numero di caratteri da stampare
mov dx,OFFSET HelloMessage ;stringa da stampare
int 21h                  ;stampa 'Hello, world'
mov ah,4ch               ;funzione DOS terminare programma
int 21h                  ;termina il programma
END

```

In questa versione di "Hello, world", viene sostituita la funzione DOS che consente di visualizzare una stringa sullo schermo, con una funzione DOS che trasmette una stringa su un' unità selezionata o su un file, in questo caso la stampante. Dopo aver immesso ed eseguito il programma, viene stampato un foglio contenente il messaggio "Hello, world"; si ricordi di salvare il programma prima di eseguirlo poichè altrimenti sostituirà la versione precedente di HELLO.ASM che andrà persa.

E' possibile modificare questo programma per fare in modo che trasmetta l' output sullo schermo e non sulla stampante, visualizzando nuovamente "Hello, world", semplicemente modificando:

```

mov bx,4                ;handle della stampante

```

con:

```

mov bx,1                ;handle del dispositivo standard di output

```

Dopo aver apportato questa modifica, è necessario effettuare nuovamente l' assemblaggio e la fase di link del programma; si può osservare che, quando l' output viene visualizzato sullo schermo, il carattere finale che compare è il simbolo universale di "femmina" (codice ASCII 12): solitamente è un carattere di avanzamento pagina, che il programma trasmette alla stampante per forzare un salto pagina dopo aver scritto "Hello, world"; poichè lo schermo non ha fogli, non conosce il carattere di avanzamento pagina così visualizza semplicemente il corrispondente carattere del PC, impostato.

Scrittura Del Secondo Programma In Turbo Assembler

A questo punto è possibile immettere ed eseguire un programma che esegua realmente qualcosa, REVERSE.ASM. Si carichi l' editor di testi e si immettano le seguenti righe di codice:

```

.MODEL small
.STACK 100h
.DATA
MAXIMUM_STRING_LENGTH EQU 1000
StringToReverse DB MAXIMUM_STRING_LENGTH DUP ?
ReverseString DB MAXIMUM_STRING_LENGTH DUP ?
.CODE
mov ax,@data
mov ds,ax                ;imposta DS per puntare al segmento dati
mov ah,3fh                ;funzione DOS lettura dell' handle
mov bx,0                  ;handle di input standard
mov cx,MAXIMUM_STRING_LENGTH ;legge il numero massimo di
caratteri
mov dx,OFFSET StringToReverse ;memorizza qui la stringa
int 21h                  ;accetta la stringa
and ax,ax                 ;sono stati letti tutti i caratteri ?
jz Done                  ;no, allora termina
mov cx,ax                 ;pone la lunghezza della stringa in CX, dove è
possibile                ;utilizzarla come contatore
push cx                  ;salva la lunghezza della stringa
mov bx,OFFSET StringToReverse
mov si,OFFSET ReverseString
add si,cx
dec si                    ;punta al termine del buffer della stringa invertita

ReverseLoop:
mov al,[bx]               ;accetta il carattere successivo
mov [si],al               ;memorizza i caratteri in ordine inverso
inc bx                    ;punta il carattere successivo
dec si                    ;punta alla posizione precedente nel buffer della
stringa                  ;invertita
loop ReverseLoop          ;si sposta al carattere successivo, se presente
pop cx                    ;restituisce la lunghezza della stringa
mov ah,40h                ;funzione DOS di scrittura
mov bx,1                  ;handle dell' output standard
mov dx,OFFSET ReverseString ;stampa la stringa
int 21h                  ;stampa la stringa a rovescio

Done:
mov ah,4ch                ;funzione DOS che termina il programma
int 21h                  ;termina il programma
END

```

Il risultato dell' esecuzione del programma è subito visibile; prima di tutto, come sempre, è necessario salvare il lavoro.

Esecuzione Di REVERSE.ASM

Per eseguire REVERSE.ASM è necessario assemblarlo, digitando:

```
TASM Reverse
```

e dopo:

```
TLINK Reverse
```

per creare il file eseguibile.

Si digiti REVERSE sulla riga di comando di DOS per eseguire il programma; se Turbo Assembler individua errori od avvertimenti, è necessario controllare attentamente che il programma corrisponda a quello riportato precedentemente e dopo averlo corretto lo si esegua nuovamente. Dopo l' esecuzione del programma, il cursore lampeggiante si posiziona sullo schermo; apparentemente il programma attende che l' utente digiti qualcosa. Si provi a digitare:

```
ABCDEFGF
```

quindi si preme INVIO. Il programma termina dopo la visualizzazione di:

```
GFEDCBA
```

Ora è chiaro ciò che esegue REVERSE.ASM: inverte qualsiasi stringa di caratteri venga digitata; la manipolazione veloce dei caratteri e delle stringhe è una delle peculiarità del linguaggio Assembler.

A questo punto, sono stati immessi dei programmi, assemblati, linkati ed eseguiti, illustrando le caratteristiche fondamentali della programmazione in linguaggio Assembler: l' input, l' elaborazione e l' output. Se non si desidera un file oggetto ma si vuole un file listing, oppure se si desidera un file cross-reference ma non un file listing od un file oggetto, è possibile specificare il parametro nullo (NUL) come nome del file, ad esempio:

```
TLINK FILE1,,NUL,
```

assembla il file FILE1.ASM nel file oggetto FILE1.OBJ, non genera un file listing e crea un file cross-reference FILE1.XRF.

Il Basic

Per la conversione in BASIC sono state necessarie poche istruzioni:

LOCATE <x>, <y>
in per posizionare il cursore alle coordinate (x,y) dello schermo
modalità testo;

PRINT <stringa> [;]
con per stampare una stringa nella posizione attuale del cursore e
gli attributi correnti. Se non è presente il punto e
virgola finale (;), il cursore viene fatto andare a capo
alla riga seguente (CR+LF, ossia CarryReturn+LineFeed),
altrimenti il cursore rimarrà posizionato nella riga e
colonna successivi alla scrittura del testo;

COLOR <col1>, <col2>
<c2>, setta il colore di primo piano a <c1>, quello di sfondo a
secondo la seguente tabella dei colori:

NUMERO COLORE	NOME COLORE
1	Nero
2	Verde Medio
3	Verde Chiaro
4	Blu Scuro
5	Blu Chiaro
6	Rosso Scuro
7	Azzurro (Ciano)
8	Rosso Medio
9	Rosso Chiaro
10	Marrone
11	Giallo
12	Verde Scuro
13	Magenta
14	Grigio
15	Bianco;

CLS Cancella la pagina video corrente (modalità testo);

REM utilizzato per commentare il programma (REMark), simile al
REM dei files batch, alle parentesi graffe ('{','}') e tonde ('(',')')
del Turbo Pascal, alle barre trasversali ('/','/') del Turbo
C, ai punti e virgola (;) del Turbo Assembler, agli
asterischi ('*') o alle 'e' commerciali ('&') del Data
Base.

L' Ansi.sys del DOS

Per reperire informazioni sulle sequenze escape ANSI è stato consultato il "Manuale dell' utente"

della Microsoft per l' MS-DOS versione 5.0. Ogni riferimento o richiamo è relativo, se non diversamente specificato, a questo libro.

Cosa Sono Le Sequenze Escape ANSI

Una sequenza escape ANSI è un comando inviato alla console (al monitor o alla tastiera). Questo comando è chiamato sequenza escape perché inizia con il carattere escape (ESC) che non è considerato parte dell' output. I comandi ANSI sono stati sviluppati dall' American National Standards Institute.

NOTA: Diversamente dai comandi MS-DOS, non è possibile digitare una sequenza escape ANSI al prompt dei comandi. Per ulteriori informazioni relative all'esecuzione di una sequenza escape, consultare il paragrafo "Esecuzione di una sequenza escape ANSI". più avanti.

Le sequenze escape ANSI hanno un formato diverso da quello degli altri comandi MS-DOS. Una sequenza escape inizia con un carattere ESC ed una parentesi quadra ([]) seguiti dai parametri e termina con una lettera che rappresenta il nome del comando. Nei nomi dei comandi ANSI hanno rilevanza le maiuscole. Tra i caratteri non devono esserci spazi poiché uno spazio indica la fine di un comando. Più parametri di una sequenza escape ANSI devono essere separati da un punto e virgola (;) come in questo esempio:

ESC[3;12H

Cosa Sono I Codici ASCII

Spesso le sequenze escape ANSI richiedono dei codici ASCII come parametri. I codici ASCII rappresentano dei caratteri. Inizialmente, i codici ASCII erano 128 e rappresentavano l' alfabeto ed il sistema di punteggiatura inglese ed alcuni caratteri di controllo. Attualmente, la maggior parte dei sistemi riconosce 256 codici: i 128 ASCII originali più altri 128 che costituiscono l' insieme dei caratteri estesi. L' insieme di caratteri estesi include i caratteri europei, grafici e scientifici.

A ciascun tasto corrisponde un codice ASCII. Quando viene premuto un tasto, MS-DOS legge e lo associa ad un codice ASCII. A ciascun tasto sono assegnati dei codici diversi, a seconda che esso sia premuto da solo o in combinazione con MAIUSC. Inoltre, alla maggior parte dei tasti vengono assegnati ulteriori codici se premuti in combinazione con CTRL o ALT. I codici ASCII che visualizzano un solo carattere (A, S, D, F e così via) sono numeri. Ad esempio, il codice ASCII corrispondente alla A maiuscola è 65.

Alcuni tasti quali F1, CANCEL e freccia Su non visualizzano caratteri e di conseguenza non hanno codice ASCII corrispondenti. Tali tasti possono essere specificati utilizzando la loro sequenza di codici di scansione che consiste di un segnale e di un codice numerico. Il segnale di codice di scansione è rappresentato da 0 seguito da un punto e virgola (;). Il codice di scansione segue immediatamente il segnale. Ad esempio, il codice di scansione per F1 è 59; l' intera sequenza del codice di scansione per F1, incluso il segnale, corrisponde a 0;59.

Ad esempio, premendo MAIUSC+2, MS-DOS assegnerà a questa combinazione il codice ASCII 64, di solito corrispondente al simbolo della a commerciale (@). L'elenco dei codici ASCII è contenuto nell' Appendice "Tastiere e tabelle codici" del "Manuale dell' utente" del Microsoft MS-DOS 5.0.

NOTA: Il codice ASCII che MS-DOS assegna ad un tasto dipende dalla tabella codici corrente e dai driver della tastiera e del monitor in uso.

Esecuzione Di Una Sequenza Escape ANSI

Dopo aver installato ANSI.SYS, è possibile eseguire una sequenza escape ANSI in uno dei seguenti modi:

- Utilizzando il comando **prompt**
- Inserendo la sequenza escape ANSI in un file di testo non formattato, quindi utilizzando un comando type per eseguirla
- Inserendo una sequenza escape ANSI in un comando echo di un programma batch

Esecuzione di una sequenza escape utilizzando il comando prompt

Il modo migliore per eseguire una singola sequenza escape ANSI consiste nell' utilizzare il comando **prompt**. Infatti, il comando **prompt** consente di digitare le sequenze escape ANSI direttamente dalla tastiera e di modificarle con il programma **Doskey**. Per ulteriori informazioni relative a Doskey, consultare il Capitolo 7 "Tecniche di comando avanzate" oppure vedere il comando doskey nel Capitolo 14 "Comandi".

Se viene premuto ESC, MS-DOS annulla ciò che è stato digitato al **prompt** dei comandi. Poiché la sequenza escape ANSI inizia con ESC, è necessario trovare un modo per immettere ESC dalla tastiera senza annullare il comando. Al posto di ESC, con il comando prompt è possibile digitare la combinazione \$e. La lettera e può essere digitata in maiuscolo o minuscolo. Ad esempio, è possibile eseguire la sequenza escape **m** che modifica il colore di primo piano in rosso ed il colore di fondo in verde, utilizzando il seguente comando **prompt**:

```
prompt $e[31;42m
```

Ogni volta che viene utilizzato un comando **prompt** per eseguire una sequenza escape ANSI, il **prompt** dei comandi viene eliminato. Per ripristinarlo, è possibile ridigitare il comando prompt oppure aggiungere alla sequenza escape ANSI i caratteri che ripristinano il prompt dei comandi (**\$p\$g**). Ad esempio, per eseguire la sequenza escape **m** e ripristinare il prompt dei comandi, digitare il seguente comando **prompt**:

```
prompt $e[31;42m$p$g
```

Tale comando cambia il colore di primo piano in rosso ed il colore di fondo in verde, quindi visualizza l' unità e la directory corrente come prompt dei comandi. Da notare che nel comando **prompt** non vi sono spazi in quanto MS-DOS interpreterebbe ogni spazio tra **\$p** e **\$g** come parte del comando.

Esecuzione Di Una Sequenza Escape Da Un File Di Testo O Da Un Programma Batch

Per inserire una sequenza escape ANSI in un file di testo non formattato o in un file batch, è possibile utilizzare un editor di testo in grado di salvare i file come testo non formattato e di permettere l' immissione del carattere ESC. Ad esempio, in Microsoft Word, il carattere ESC viene immesso con la combinazione ALT+27, mentre in MS-DOS EDITOR occorre premere CTRL+P quindi ESC.

Se si devono eseguire più sequenze escape, può essere utile salvarle in un file di testo formattato o in un file batch. E' necessario che nel file non vi siano spazi o ritorni a capo tra le sequenze escape. Di conseguenza, un file contenente una serie di sequenze escape ANSI viene visualizzato come una riga continua di caratteri. Dato che in questo formato, il file risulta molto difficile da leggere e da modificare, è possibile inserire ciascuna sequenza escape su una propria riga. Prima di eseguire il file, occorre eliminare i ritorni a capo.

Se le sequenze escape vengono memorizzate in un file batch, è necessario che ciascuna riga inizi con un comando **echo**. Per eseguire un programma batch, è necessario digitare il nome del file batch. Per eseguire invece un file di testo, occorre utilizzare il comando **type** seguito dal nome del file.

Ridefinizione Dei Caratteri

IN BREVE: Per ridefinire il carattere visualizzato quando viene premuto un determinato tasto, utilizzare la sequenza escape (**p**) per l' impostazione di stringhe dalla tastiera. Ad esempio, il seguente comando converte la combinazione MAIUSC+6 (^) nel carattere ASCII 172 (¼):

```
ESC["^";172p
```

In tale formato la sequenza escape **p** presenta due parametri: il carattere o il codice ASCII assegnato al tasto che si desidera modificare ed il nuovo carattere o il codice ASCII che si desidera assegnare al tasto.

E' possibile modificare il codice ASCII che MS-DOS assegna ad un tasto utilizzando la sequenza escape (**p**) per l' impostazione di stringhe dalla tastiera. E' possibile specificare un codice ASCII digitando il numero corrispondente o il carattere racchiuso tra virgolette. Ad esempio, per modificare la combinazione MAIUSC+2 in modo che sia visualizzato il segno più (+) invece del simbolo della a commerciale (@), utilizzare la seguente sequenza escape **p**:

```
ESC [ " @ " ; "+"p
```

E' importante notare che non vi sono spazi sulla riga di comando e che **p** segue l' ultimo parametro senza alcun punto e virgola. Invece di digitare i caratteri, è possibile digitare i codici ASCII, come nel seguente esempio:

ESC[64;43p

Quando al posto dei caratteri vengono utilizzati i codici ASCII, le virgolette non sono necessarie. Una volta eseguito il comando è possibile premere MAIUSC+2 per visualizzare il segno più.

Per riassegnare al tasto il proprio codice originale, è necessario che questo sia digitato come primo e secondo parametro, come nel seguente esempio:

ESC[64;64p

Esistono 256 codici ASCII, ma non tutti sono assegnati ai tasti. Per assegnare uno di questi codici ad un tasto, è possibile utilizzare il codice ASCII del carattere in una sequenza escape **p**. Ad esempio, se si desidera che premendo MAIUSC+2 venga visualizzato un segno di spunta, corrispondente al codice ASCII 251, invece del simbolo della a commerciale (@), è possibile utilizzare la seguente sequenza escape:

ESC["@";251p

Poichè il simbolo della a commerciale è presente su quasi tutte le tastiere, per specificarle è possibile digitare o il simbolo della a commerciale (@) oppure il suo codice ASCII. Il segno di spunta, invece, non è sempre presente sulla tastiera, pertanto per specificarlo, è necessario utilizzare il codice ASCII corrispondente.

Quando ad un tasto viene assegnato un codice ASCII differente, non è più possibile digitare il relativo carattere per visualizzarlo. Per riassegnare alla combinazione MAIUSC+2 il suo codice originale, è necessario digitare tale codice come primo e secondo parametro, come nella seguente sequenza escape:

ESC[64;64p

Assegnazione Di Comandi Ad Un Tasto

IN BREVE: Per assegnare un comando o una sequenza di comandi ad un tasto, utilizzare la sequenza escape (**p**) per l' impostazione di stringhe dalla tastiera. Nel seguente esempio la sequenza escape assegna il comando C:\LAVORO\LUNEDI alla combinazione CTRL+W (ASCII 23):

ESC[23;"c:\lavoro\lunedì";13p Questa sequenza escape può avere diversi parametri. Il primo specifica sempre il carattere o il codice ASCII del tasto da modificare. Gli altri parametri specificano le operazioni che si desidera far eseguire al tasto. Nell' esempio precedente, alla combinazione CTRL+W sono state assegnate due operazioni differenti: l'esecuzione del comando C:\LAVORO\LUNEDI e la visualizzazione di un carattere ASCII 13, equivalente alla pressione

di INVIO.

La sequenza escape (**p**) per l' impostazione di stringhe dalla tastiera può essere utilizzata non solo per assegnare un nuovo singolo carattere ad un tasto, ma anche per assegnare più caratteri ad un unico tasto. Ad esempio, è possibile assegnare la parola 'percentuale' al tasto di percentuale (MAIUSC+5) utilizzando la seguente sequenza escape:

```
ESC["%";"percentuale"p
```

Una volta eseguito il comando, è possibile premere MAIUSC+5 per visualizzare la parola 'percentuale'.

La sequenza escape **p** offre anche l' opportunità di assegnare uno o più comandi MS-DOS ad un tasto. Quando si esegue un comando dal prompt dei comandi, è necessario digitare il comando e premere INVIO. Quando un comando viene assegnato ad un tasto, è necessario specificare queste operazioni: digitazione del comando e pressione di INVIO. Il comando da digitare deve essere racchiuso tra virgolette; per eseguire invece INVIO, occorre utilizzare il codice ASCII 13 (carattere di ritorno a capo). Ad esempi, se si desidera visualizzare la struttura di una directory una schermata alla volta, utilizza: sequenza escape **p** premendo MAIUSC+7 (il simbolo della e commerciale):

```
ESC["%";"dir /b /p";13p
```

Per ciascun comando da assegnare ad un tasto occorre inserire due parametri: il testo comando ed un carattere di ritorno a capo (INVIO). E' importante notare che all' interno delle virgolette è possibile digitare degli spazi. Per impostare F1 (sequenza del codice di scansione O;59) in modo che esso cambi la directory corrente ed avvii il programma WORD.EXE, utilizzare la seguente sequenza escape:

```
ESC[O;59;"cd \lettere";13;10;"word";13p
```

Il comando dispone di sei parametri: due comandi racchiusi tra virgolette, due codici di ritorno a capo (ASCTI 13) ed un codice nuova riga (ASCTI 10).

NOTA: Molti programmi non tengono conto delle ridefinizioni dei tasti.

Spostamento Del cursore

IN BREVE: Per spostare il cursore, utilizzare le sequenze escape **A**, **B**, **C**, **D**, **H** o **f**. Ad esempio, la sequenza escape (**A**) sposta il cursore dalla posizione attuale verso l' alto.

La seguente sequenza escape sposta il cursore di due righe verso l'alto:

```
ESC[2A
```

Una volta terminata l' esecuzione di un comando, il cursore ritorna sempre a destra del prompt dei comandi. Se si eseguono delle sequenze escape con il comando **prompt**, è possibile specificare il punto in cui il cursore verrà visualizzato al termine dell' esecuzione del comando. Se si esegue una sequenza escape ANSI con un file di testo o un programma batch, è possibile modificare la posizione del cursore in modo da visualizzare un testo in un determinato punto dello schermo. Tuttavia, al termine dell' esecuzione della sequenza escape ANSI, il cursore ritornerà al **prompt** dei comandi è possibile utilizzare le seguenti sequenze escape ANSI per controllare lo spostamento del cursore:

- A** Sposta il cursore verso l' alto rispetto alla posizione corrente.
- B** Sposta il cursore verso il basso rispetto alla posizione corrente.
- C** Sposta il cursore a destra rispetto alla posizione corrente.
- D** Sposta il cursore a sinistra rispetto alla posizione corrente.
- H** Sposta il cursore sulla riga e sul carattere specificati (analoga funzione di **f**).
- f** Sposta il cursore sulla riga e sul carattere specificati (analoga funzione di **H**).

Le sequenze escape da **A** a **D** spostano il cursore rispetto alla posizione iniziale. Le sequenze escape **H** ed **f** spostano il cursore sulla riga e sul carattere specificati, indipendentemente dalla posizione corrente. Se viene eseguita una sequenza escape per spostare il cursore al di fuori dello schermo quando esso si trova già ai margini, la sequenza escape verrà ignorata ed il cursore rimarrà nella posizione corrente.

Posizionamento Del Prompt Dei Comandi

Se si esegue una sequenza escape **H**, relativa allo spostamento del cursore, insieme al comando **prompt**, la posizione del cursore specificata diverrà la posizione permanente del prompt dei comandi. Ad esempio, il seguente comando sposta il cursore nell'angolo superiore sinistro dello schermo (la posizione originale) che sarà assunto come posizione permanente del prompt dei comandi:

```
prompt $e[H$p$g
```

Questa sequenza escape sposta il cursore nella posizione originale e visualizza l'unità, directory correnti ed un segno di maggiore (>) come prompt dei comandi. Fino a che non viene digitato di nuovo il comando **prompt**, MS-DOS continua a visualizzare il prompt dei comandi nell' angolo superiore sinistro dello schermo. Quando si utilizza la sequenza escape **H**, la nuova posizione del prompt è specificata in coordinate dello schermo. Se si utilizzano altre sequenze escape per spostare il prompt dei comandi, è necessario specificare la posizione in cui il prompt dei comandi sarà visualizzato in relazione alla sua posizione originale.

Ad esempio, il seguente comando sposta il prompt dei comandi sulla quarta colonna di una riga:

```
prompt $e[3C$p$g
```

Come Creare Una Schermata

Se ad una serie di comandi per lo spostamento del cursore si unisce del testo, è possibile creare una schermata personalizzata. Ad esempio, le seguenti sequenze escape ANSI di un programma batch visualizzano la schermata introduttiva di un altro programma:

```
echo off
cls
echo ESC[2;H*BenvenutoESC[2BESC[7D*nel programma*ESC[2BESC[6D*Utilità
    database*ESC[12;1H(Per visualizzare la Guida, premere F1)
prompt $e[10;1H Immettere un comando:
```

Queste sequenze escape posizionano il cursore e visualizzano il testo. La schermata generata da questo programma batch è la seguente:

```
*Benvenuto*

    *nel programma*

        *Utilità database*
```

Immettere un comando: _

(Per visualizzare la Guida, premere F1)

Il prompt dei comandi rimarrà sulla decima riga dello schermo fino a quando non è riposizionato.

Modifica Degli Attributi Dello Schermo

IN BREVE: Per modificare gli attributi dello schermo, utilizzare la sequenza escape (**m**) per l'impostazione della modalità grafica. Ad esempio, il seguente comando visualizzerà il testo in grassetto:

```
ESC[1m
```

Il comando successivo visualizzerà il testo in magenta e grassetto:

```
ESC[1;35m
```

La sequenza escape **m** può avere un numero illimitato di parametri.

Allo schermo possono essere assegnati tre diversi tipi di attributi: formato del testo, colore del testo e colore di fondo dello schermo. Gli attributi relativi al formato specificano se il testo visualizzato sarà in grassetto, sottolineato, intermittente o nascosto. Gli attributi relativi al colore del testo specificano il colore del testo. Gli attributi relativi al colore di fondo specificano il colore dello schermo. Se viene specificato un attributo non supportato dal sistema, MS-DOS ne ignora l'impostazione e non apporta alcuna modifica.

La sequenza escape ANSI (**m**) per l'impostazione della modalità grafica può essere utilizzata per modificare la visualizzazione della schermata. Una volta impostato un attributo, questo sarà valido per qualunque nuovo testo visualizzato. Ad esempio, il seguente comando visualizza il testo in grassetto.

ESC[lm

Per definizione, il testo è bianco mentre il fondo è nero. Per ritornare all' impostazione predefinita, è necessario utilizzare il parametro 0, come nel seguente esempio:

ESC[0m

Con una sequenza escape è possibile impostare quanti attributi si desidera. Ad esempio: il seguente comando imposta una visualizzazione del testo intermittente (5) ed in rosso (31) su di un fondo blu (44):

ESC[5;31;44m

Non è importante l' ordine in cui vengono digitati i parametri. E' necessario, invece, che ciascun parametro sia separato dal successivo da un punto e virgola (;).

Modifica Del Formato Del Testo

I seguenti attributi definiscono l' aspetto del testo:

- | | |
|---|--|
| 1 | Grassetto |
| 4 | Sottolineato (solo su schermi monocromatici) |
| 5 | Intermittente |
| 8 | Nascosto |

L' attributo testo nascosto impedisce la visualizzazione del testo, a meno che non venga successivamente modificato il colore di fondo. Nell' esempio seguente, la prima sequenza escape ANSI visualizza la parola 'ATTENZIONE' in grassetto e ad intermittenza alla posizione attuale del cursore:

ESC[l;5mATTENZIONE
ESC[0m

La seconda sequenza escape reimposta tutti gli attributi in modo che il testo successivo verrà visualizzato normalmente. Il seguente comando prompt visualizza il segno di maggiore (>) del prompt dei comandi in grassetto:

prompt \$p\$e[lm\$g\$e[0m

Il testo successivo al prompt non sarà visualizzato in grassetto dal momento che dopo **\$g** è stato

inserito il parametro 0.

Modifica Del Colore Del Testo

I seguenti attributi specificano il colore del testo:

30	Nero
31	Rosso
32	Verde
33	Giallo
34	Blu
35	Magenta
36	Azzurro
37	Bianco
7	Testo nero su fondo bianco.

Nell' esempio seguente, la prima sequenza escape ANSI visualizza la parola 'ATTENZIONE' in rosso, in grassetto e ad intermittenza:

```
ESC[31;l;5mATTENZIONE  
ESC[0m
```

La seconda sequenza escape reimposta gli attributi così che il nuovo testo avrà le impostazioni predefinite. Il seguente comando prompt visualizza l' unità e la directory del prompt dei comandi in grassetto ed in blu, mentre il segno di maggiore in grassetto ed in rosso:

```
prompt $e[l;34m$p$e[31m$g$e[0m
```

Modifica Del Colore Di Fondo

I seguenti attributi specificano il colore di fondo:

40	Nero
41	Rosso
42	Verde
43	Giallo
44	Blu
45	Magenta
46	Azzurro
47	Bianco
7	Testo nero su fondo bianco.

Nell' esempio seguente, la prima sequenza escape visualizza la parola ATTENZIOE in rosso, in grassetto e ad intermittenza su fondo giallo:

ESC[43;31;l;5mATTENZIONE
ESC[0m

La seconda sequenza escape reimposta gli attributi ai valori predefiniti.

Se si utilizza l'attributo ESC[7m, MS-DOS visualizza un testo nero su fondo bianco. Anche se si modifica il colore del testo, il fondo rimane bianco a meno che non venga modificato esplicitamente il colore di fondo o non vengano reimpostati gli attributi così che il nuovo testo abbia impostazioni predefinite.

NOTA: Gli attributi dello schermo sono conformi allo standard ISO 6429.

Il Data Base

Saranno date di seguito nozioni sul Data Base IV di carattere generale e qualche approfondimento sulle funzioni/procedure utilizzate per inviare l' output allo schermo.

L' Aspetto Dell' Applicazione

In questa sezione si descriveranno come fare per migliorare l' aspetto e la funzionalità delle applicazioni utilizzando linee, riquadri, finestre e simboli grafici.

Come Tracciare Linee E Riquadri

In uno schermo di immissione dei dati, linee e riquadri racchiudono, o separano dal resto, degli elementi fra loro collegati. Per tracciare una linea o un riquadro, si utilizza il comando

@ <coordinate> TO <coordinate>

scegliendo opportunamente le coordinate in modo da ottenere l' effetto desiderato. Per tracciare una linea o un riquadro doppi, occorre aggiungere il comando DOUBLE. Per decidere il colore della parte interna dei riquadri si usa il comando

@ <coordinate> FILL TO <coordinate> COLOR <colore>.

Per cancellare una riga o un riquadro si utilizza il comando

@ <coordinate> CLEAR TO <coordinate> ,

indicando le coordinate utilizzate per creare l' oggetto.

Per la visualizzazione di un file di testo si digita uno di questi comandi:

- ???
posizione

Visualizza il testo della riga successiva a partire dalla
corrente del cursore.

- @<coordinate> SAY Visualizza il testo nella posizione specificata.
- TEXT..ENDTEXT Visualizza il testo come viene immesso nel programma.

Visualizzazione Dei Simboli Grafici

Si possono personalizzare le applicazioni, rendendole inoltre esteticamente più gradevoli, utilizzando opportunamente i simboli grafici. Un simbolo grafico è in carattere del set di caratteri estesi IBM. Per visualizzare un simbolo grafico si invia sullo schermo il relativo codice ASCII con i comandi ??? o @<coordinate> SAY <codice ASCII>.

Impiego Delle Finestre

Una finestra è una zona rettangolare dello schermo in cui si eseguono azioni relative al programma. In genere le finestre si utilizzano per fare elenchi dell' output (comando LIST) o per passare in rassegna (comando BROWSE) i dati. Ci possono essere più finestre aperte contemporaneamente con visualizzata in ciascuna, un' operazione diversa.

Quando si vuole usare una finestra, bisogna innanzitutto definirla con il comando

DEFINE WINDOWS <nome> FROM <coordinate> TO <coordinate> ,

e poi attivarla con

ACTIVATE WINDOWS <nome> /ALL.

Si possono definire e visualizzare fino a 27 finestre. La finestra che si attiva per ultima è quella attiva, cioè quella che accetta e visualizza i comandi successivi.

Input E Output

Quasi tutti i programmi richiedono un input di qualche tipo, che può essere rappresentato semplicemente da una battuta in risposta a una domanda o può essere molto più complesso e composto da una serie di record di un file di database immessi e sottoposti a editing in una scheda di più videate.

Come Richiedere I Dati Mediante Le Schede

I files formato del video dispongono sullo schermo messaggi in aree riservate all' immissione dei dati. Per predisporre rapidamente il prototipo di questi schermi è consigliato di utilizzare lo schermo di impostazione delle schede. A questo schermo si accede dal pannello schede del Centro di Controllo, oppure immettendo, al Punto, il comando CREATE SCREEN <NomeFile>.

Il generatore di schede crea due file: un file di formato schermo (.SCR), in cui vengono registrate le informazioni per l' impostazione dello schermo, e un file di formato (.FMT), che contiene i

comandi in linguaggio dBASE che costruiscono la videata. Le eventuali modifiche apportate direttamente ai comandi del file di formato (.FMT) non si riflettono nel file di formato video (.SCR). Di conseguenza, conviene effettuare il progetto e l'editing sullo schermo di impostazione. Per utilizzare un file di formato video, dovete includere il comando SET FORMAT TO <NomeFile>: dBASE utilizzerà la scheda ogni volta che si impartisce un comando a tutto schermo (APPEND, EDIT, CHANGE, BROWSE o READ). Per chiudere un file di formato, occorre digitare il comando SET FORMAT TO.

Come Richiedere I Dati Da Tastiera

Se i dati da richiedere all'utente sono pochi, a volte non conviene creare una scheda a tutto schermo. Si può invece costruire un piccolo file di programma contenente istruzioni, messaggi ed aree per l'immissione dei dati.

??? e @..SAY sono utili per visualizzare singole righe di testo, come ad esempio domande e messaggi. Per visualizzare blocchi di testo, come ad esempio le informazioni contenute in un file di help, usate il comando TEXT..ENDTEXT.

Per ottenere delle informazioni dall'utente, si usano i comandi @..GET e READ. Il comando @..GET dice al dBASE dove inserire sullo schermo un'area riservata all'immissione dei dati, indica dove devono essere registrati i dati di ingrasso e, a input completo, li scrive o li assegna alle variabili di memoria designate.

I comandi @..SAY e @..GET possono essere combinazioni in un'unica riga da istruzioni. Ad esempio:

```
@ 5,3 SAY "Nome: " GET NOME
```

Questo comando immette sul video, alle coordinate (5,3) la scritta 'Nome:', seguita da un'area designata all'immissione del dato. Quando si esegue un comando READ la scritta viene visualizzata e l'informazione che l'utente scrive in quest'area viene memorizzata nella variabile di memoria NOME.

Si possono accumulare fino a 128 GET per volta (se ne occorre un numero maggiore bisogna modificare il file CONFIG.DB) prima di dare un comando READ. Il comando READ cancella automaticamente i GET dopo averli letti nel file di database o nelle variabili di memoria.

Come Impostare Il Formato Degli Schermi Video Per L' Immissione Dei Dati

L'aspetto degli schermi video può essere migliorato scegliendo il colore e altri attributi di visualizzazione per le diverse voci e per le zone di immissione dei dati.

SET INTENSITY ON/OFF serve per stabilire se le zone di immissione dei comandi GET devono apparire in video inverso (SET INTENSITY ON), che è l'impostazione standard), o normale

(OFF).

Per semplificare i delimitatori della stringa GET sul video si usa il comando

SET DELIMITERS TO <caratteri>.

SET DELIMITERS TO {}, ad esempio, racchiude tra parentesi i dati immesi in una zona GET. Per attivare e disattivare i delimitatori si usa il comando SET DELIMITERS ON/OFF.

Per stabile il colore dei SAY e dei GET si usa il comando

SET COLOR TO <colori SAY>. <colori GET>.

oppure con l' opzione COLOR del comando @...

Per visualizzare i campi memo in una finstra, innanzitutto bisogna definire la finstra, e poi includere l' opzione WINDOWS nel comando @ ..SAY ..GET relativo al campo Memo.

Per visualizzare i valori standard relativi a un campo si usa l' opzione DEFAULT del comando @..SAY..GET. Per esempio:

@ 10,12 SAY "Data: " GET A->DATA DEFAULT DATE()

inserisce come standard la data corrente nel campo DATA.

L' opzione PICTURE del comando @..SAY..GET consente di impostare il formato e di stabilire dei limiti per i dati da immettere in un campo. L' istruzione:

@ 12,12 SAY "Costo: " GET COSTO PICTURE "@ \$ 999,99"

imposta il formato del campo COSTO con il segno del dollaro e due decimali, e ne limita il valore a cifre inferiori a \$1,000.00.

Il comando SET CARRY TO <Campi> cansente di passare a un nuovo record conservando i valori che l' utente ha immesso in un dato campo. Questa caratteristica è molto utile nel caso di dati che non cambiano spesso da un record all' altro, come ad esempio il nome della provincia in un file di indirizzi dei dipendenti.

Le opzioni appena elencate possono essere usate da sole, oppure combinate fra loro. Molti dei comandi di cui abbiamo parlato (in particolare PICTURE) offrono parecchie opzioni.

Il Controllo Dei Dati Per Individuare Gli Errori

L' opzione RANGE <inferiore superiore> fissa i limiti superiore e inferiore per numeri e date.

@..GET IMPORTO RANGE 25,100

accetta solo numeri compresi fra 25 e 100. Volendo si può definire solo il limite superiore o quello inferiore. Per esempio:

```
@..GET DATA RANGE, {30/4/91}
```

accetta qualsiasi data purchè antecedente al primo maggio 1991 (si noti la virgola che precede il limite superiore).

L'opzione VALID <condizione> verifica la validità dei dati in funzione della condizione che viene specificata. Per esempio:

```
@5,15 SAY "Inizio stampa? (S/N)" GET RISPOSTA VALID RISPOSTA $ "S/N"
```

accetta in risposta alla richiesta di istruzioni solo caratteri S o N. Si può inoltre specificare un messaggio

di errore, aggiungendo dopo VALID, l'opzione ERROR. Aggiungendo nell'esempio riportato sopra

ERROR "Premere solo S o N", se l'utente batte un carattere diverso da S o N viene visualizzato il messaggio di errore. Perchè siano accettati sia i caratteri maiuscoli che minuscoli, occorre aggiungere prima di VALID la clausola PICT "!". VALID accetta come argomento qualsiasi espressione di dBASE, comprese le funzioni definite dall'utente.

L'opzione WHEN stabilisce quando un utente può modificare un campo. Per esempio:

```
GET ARTICOLO WHEN "" < > TRIM(COD_ARTIC) ERROR "Immettere prima il  
codice dell'  
articolo"
```

impedisce che vengano immessi dei dati nel campo ARTICOLO prima che nel campo COD_ARTIC.

Elaborazione Delle Battute

I comandi ON KEY e ON ESCAPE e le funzioni INKEY(), LASTKEY() e READKEY() fanno sì che

l'applicazione risponda in modo selettivo a seconda dei tasti che vengono premuti dall'utente.

ON KEY esegue un comando quando l'utente preme un certo tasto: il tasto da premere deve essere

specificato con la parola chiave LABEL, altrimenti potrà venire utilizzato qualsiasi tasto.

ON ESCAPE risponde solo al tasto ESC. Nei comandi EDIT, BROWSE, READ e nei menu definiti dall'utente, ON KEY e ON ESCAPE hanno effetto; negli altri casi, la battuta viene registrata dopo l'esecuzione del comando corrente.

Per far sì che il programma esegua determinate operazioni a seconda del tasto che viene premuto dall'utente, si possono anche usare alcune funzioni. INKEY(0) procura una pausa del programma fino a che l'utente preme un tasto. Con INKEY(n), la pausa dura il numero di secondi specificato.

READKEY() può restituire due codici: un numero compreso fra 0 e 36 se prima di uscire non è stata fatta alcuna modifica ai dati, e lo stesso numero aumentato di 256 unità se i dati sono stati modificati. Richiamiamo l'attenzione sul rapporto fra READKEY() e LASTKEY(). Con READKEY() è possibile sapere se, nell'uscire, impartisce un comando a tutto schermo, l'utente ha salvato i dati o ha scaricato le modifiche.

LASTKEY(), poi, fornisce il tasto che ha premuto l'utente per uscire.

Dati Sui Programmi

Diamo Qualche Numero

Verranno ora elencate le units utilizzate nei programmi e una breve descrizione di ciascuna di esse, oltre al numero di righe totali e all'occupazione di memoria di massa (disco) dell'eseguibile.

T.I.P. v1.0

Text Image Processor v1.0 ha in tutto 32.200 righe circa.

Utilizza 26 units, che sono:

CRT	unit standard per la gestione del video in modalità testo, dei codici estesi della tastiera, dei colori, delle finestre e del suono;
DOS	unit standard per la gestione di chiamate al sistema di gestione dei files, ed altro;
KEYBOARD	definisce tutti i codici ASCII dei tasti speciali in costanti che prendono il nome del tasto, più la lettera 'k' iniziale. Se, ad esempio, si vuole controllare il tasto F1, allora si dovrà controllare se il tasto premuto è uguale alla costante kF1 (più eventualmente il controllo del primo codice che vale #0, ossia kNull);
MOUSE	gestisce tutte le procedure e funzioni di supporto per gestire il mouse, compresa l'installazione. Il driver del mouse deve essere caricato prima di eseguire qualsiasi operazione in questa unit. Ci sono comunque due variabili, MOUSEOK e MOUSEERROR, che controllano ogni errore di installazione del mouse (la prima) e il risultato dell'esecuzione di una procedura/funzione di questa unit (la seconda);
SYSTEM	unit standard che implementa routines di supporto a basso livello di programmazione per le funzioni e procedure del Turbo Pascal,

	come I/O su file, manipolazione di stringhe, operazioni a virgola mobile e allocazioni dinamiche della memoria. Tutti i programmi fanno uso di questa speciale unit;
TIP	è il programma principale;
TIPBASE	gestisce la lettura e la scrittura del file di configurazione; la lettura e la scrittura di un file maschera strutturato, di un file di testo (ASCII), ed altro;
TIPBLOCK	contiene tutte le procedure che servono per eseguire le operazioni con i blocchi specificate dal menu BLOCCHI, come la definizione dei contorni, la copia o lo spostamento di una parte dello schermo, il salvataggio o il ripristino di essa, ecc.;
TIPCOLOR	contiene procedure/funzioni per il controllo dei colori e per i messaggi di uscita dal programma;
TIPCONFIG	contiene le procedure che modificano le varie configurazioni: dai bordi delle finestre alle velocità del mouse, a quelle dei vari ritardi, ai colori, al salvataggio/lettura del file di configurazione TIP.CFG, ecc.;
TIPCONV	serve per convertire l' immagine (in formato testo naturalmente) nel programma in Turbo Pascal, Turbo C, Turbo Assembler, Data Base, Sequenze Escape ANSI, GW-Basic. Per ogni linguaggio sono disponibili dalle due alle sette opzioni/varianti di conversione;
TIPFAST	unit che gestisce la memoria video (scrivendovi direttamente una stringa o salvando una certa area di memoria in un' altra pagina), il cursore del video (nascondendolo o visualizzandolo solo in parte) e cose simili;
TIPFILES	gestisce le chiamate alle opzioni del menu FILES, all' opzione 'SCEGLI ASCII' e a quella 'SCEGLI CORNICE' del menu principale;
TIPHELP	gestisce le chiamate alle schermate di aiuto e tutti gli spostamenti all' interno di queste. I tasti sono F1, CTRL-F1, SHIFT-F1, ALT-F1, i tasti cursore, TAB e SHIFT-TAB, ESCAPE, e molti altri. Per ulteriori informazioni premere, una volta entrati nel programma, il tasto F1 che darà una descrizione sommaria dell' utilizzo e della consultazione di un argomento;
TIPIMAGE	contiene la procedura che seleziona un' immagine fra quelle disponibili;
TIPINDIR	unit che gestisce l' input da tastiera di un nome di directory. L' utente può editarlo con l' utilizzo dei tasti INSERT, DELETE, LEFT, RIGHT, HOME, END, ecc.. Per facilitare la ricerca si può sfruttare l' elenco delle sub-directories del disco corrente. Per passare dall' editing alla lista della sub-directory basta premere i tasti TAB o SHIFT-TAB. E' possibile utilizzare la ricerca veloce per un file digitandone solo una parte (ad esempio le prime 3 lettere);
TIPINFIL	unit che gestisce l' input da tastiera di un nome di file. L' utente può editarlo con l' utilizzo dei tasti INSERT, DELETE, LEFT, RIGHT,

	HOME, END, ecc.. Per facilitare la ricerca si può sfruttare l' elenco dei files della directory appena sotto. Per passare dall' editing alla lista della directory basta premere i tasti TAB o SHIFT-TAB. E' possibile utilizzare la ricerca veloce per un file digitandone solo una parte (ad esempio le prime 3 lettere);
TIPINIT	inizializza le variabili di memoria, legge e scrive il file di configurazione nella directory corrente, controlla i colori, i ritardi, i pathes, ecc.;
TIPINSTR	unit che gestisce l' input da tastiera di una stringa. L' utente può editarla con l' utilizzo dei tasti INSERT, DELETE, LEFT, RIGHT, HOME, END, ecc.. La stringa compare nell' ultima riga dello schermo;
TIPMAIN	contiene le procedure principale del programma;
TIPMENU	unit che gestisce le chiamate ai menu a comparsa e le scelte che si possono effettuare in esso;
TIPTIME	gestisce la scrittura dell' ora nella linea di stato e dei tasti speciali (LEFTSHIFT, RIGHTSHIFT, ALT, CTRL, NUMLOCK, SCROLLLOCK, CAPSLOCK, INSERT). Questo è fatto con l' aggancio di due routines particolari agli interrupt 1Ch (timer software) e 09h (tastiera);
TIPTRACE	contiene tutte le procedure per il disegno di cornici e il raccordo dei caratteri scritti con quelli già esistenti nell' immagine;
TIPVAR	unit di definizione delle costanti, tipi e variabili globali che utilizza il programma TIP (Text Image Processor);
TIPVIDEO	contiene le procedure e funzioni per gestire le chiamate al menu VIDEO, cioè la copia di immagini, lo spostamento, lo riempimento, la cancellazione, la memorizzazione, il ripristino, ecc.;
TIPWIN	gestisce le finestre in Turbo Pascal: è possibile aprire e salvare una finestra, chiuderla e ripristinarla, riempirla di un certo carattere, disegnarne il bordo in un certo modo, scrivendo direttamente in memoria video.

L' occupazione in memoria del file TIP.EXE è di 213 KBytes circa.

Per compilare il programma sono necessari anche i seguenti files: TIP.HLP, VIDEO.ASM, VIDEO.H, VIDEO.OBJ, VIDEO.PAS, WIN.ASM, WIN.OBJ, TIP.CFG.

G.I.P. v1.0

Graphic Image Processor v1.0 ha in tutto 25.000 righe circa.

Utilizza 29 units, che sono:

CRT	vedi T.I.P.;
DOS	vedi T.I.P.;
GIP	è il programma principale;

GIPAPPPRC	contiene procedure di appoggio per la unit GIPPROC;
GIPBASE	gestisce il controllo della pressione del mouse e l' attivazione dello strumento, colore o pulsante selezionato. Consente il disegno di una singola figura sul video oppure di tutto il disegno. Contiene anche la scrittura della schermata iniziale del programma;
GIPCONV	serve per convertire l' immagine (in formato grafico naturalmente) nel programma in Turbo Pascal, Turbo C o Turbo Assembler;
GIPDRIV	contiene i drivers di visualizzazione (ATT, CGA, EGAVGA, HERC, PC3270, IBM8514) che verranno poi linkati nel file eseguibile per evitare di utilizzare i files *.BGI;
GIPFAST	unit che gestisce la pressione dei tasti speciali (SHIFT, CTRL, ALT, NUMLOCK) e la scrittura in modalità grafica ottimizzata (evitando l' effetto sfibrillamento);
GIPFILE	gestisce le chiamate alle opzioni del menu FILE, le scrittura di finestre video e di finestre di dialogo, nonché la gestione dell' input di un nome di file per letture, salvataggi, conversioni, ecc.;
GIPFONT1	contiene procedure e funzioni necessarie per linkare i driver dei vari tipi di font (parte 1); essi sono: GOTH, SANS, LITT;
GIPFONT2	contiene procedure e funzioni necessarie per linkare i driver dei vari tipi di font (parte 2); essi sono: TRIP, SCRI, SIMP, TSCR;
GIPFONT3	contiene procedure e funzioni necessarie per linkare i driver dei vari tipi di font (parte 3); essi sono: LCOM, EURO, BOLD;
GIPGRAPH	procedure e funzioni per premere un pulsante sul video, per controllare gli eventuali tasti premuti da tastiera, per attendere la pressione del mouse scrivendone intanto le coordinate sul video, ecc.;
GIPHELP	gestisce le chiamate alle schermate di aiuto e tutti gli spostamenti all' interno di queste. I tasti cono F1, CTRL-F1, SHIFT-F1, ALT-F1, i tasti cursore, TAB e SHIFT-TAB, ESCAPE, e molti altri. Per ulteriori informazioni premere, una volta entrati nel programma, il tasto F1 che darà una dscrizione sommaria dell' utilizzo e della consultazione di un argomento;
GIPICWIN	contiene tutti i dati per richiamare le finestre con le icone di scelta (nella parte degli strumenti a destra), finestre di piccole dimensioni temporanee;
GIPIMAGE	procedure per l' inserimento di un oggetto o di una stringa di testo all' interno della lista delle figure in memoria;
GIPINIT	inizializza le variabili di memoria, controlla i colori, la presenza di un mouse, la scheda video VGA adeguata, la memoria disponibile, e rilascia la memoria occupata, ripristana il video, gestisce i messaggi di errori, ecc.;
GIPLINK	contiene procedure e funzioni necessarie per linkare i drivers per le schede grafiche e i vari tipi di font;
GIPMAIN	contiene le procedure principale del programma;
GIPMENU	unit che gestisce le chiamate ai menu a comparsa e le scelte che si possono effettuare in esso;
GIPPROC	contiene tutte le procedure attivate dai vari comandi impartiti

GIPSHAPE	attraverso il mouse o la selezione di un' opzione da un menu;
GIPSTATO	contiene quelle procedure per disegnare una forma, come ad esempio un cerchio, un rettangolo, una stringa di testo, ecc.;
GIPVARS	assegna il comando impartito dall' utente alla procedura o funzione relativa;
GRAPH	Unit di definizione delle costanti, tipi e variabili globali che utilizza il programma GIP (Graphic Image Processor);
KEYBOARD	unit standard che comprende oltre 50 routines grafiche che vanno dalle procedure che gestiscono un singolo bit alle chiamate ad alto livello;
MOUSE	vedi T.I.P.;
SYSTEM	vedi T.I.P.;

L' occupazione in memoria del file GIP.EXE è di 381 KBytes circa.

Per compilare il programma sono necessari anche i seguenti files: GIP.HLP, ATT.OBJ, ATT.BGI, CGA.OBJ, CGA.BGI, EGAVGA.OBJ, EGAVGA.BGI, HERC.OBJ, HERC.BGI, PC3270.OBJ, PC3270.BGI, IBM8514.OBJ, IBM8514.BGI, BOLD.OBJ, BOLD.CHR, EURO.OBJ, EURO.CHR, GOTH.OBJ, GOTH.CHR, LCOM.OBJ, LCOM.CHR, LITT.OBJ, LITT.CHR, SANS.OBJ, SANS.CHR, SCRI.OBJ, SCRI.CHR, SIMP.OBJ, SIMP.CHR, TRIP.OBJ, TRIP.CHR, TSCR.OBJ, TSCR.CHR.

SPRITE v5.0

Sprite Manager v5.0 ha in tutto 14.200 righe circa.

Utilizza 14 units, che sono:

CRT	vedi T.I.P.;
DOS	vedi T.I.P.;
GRAPH	vedi G.I.P.;
MOUSE	vedi T.I.P.;
PULSANTI	contiene tutti i dati delle icone, delle costanti, dei tipi e delle variabili che utilizza il programma. Le icone sono in tutto 32 (MaxIcone-1), e ad ogni icona è associato un record che contiene, oltre l'immagine, le coordinate sullo schermo. La struttura principale è un ARRAY di record (Variabile puntatore, Coordinata X, Coordinata Y) : i primi 16 elementi (da 0 a 15) sono le icone dei 16 colori disponibili, mentre le altre sono quelle dei comandi, delle frecce, ecc. Per identificare l' icona che esegue il comando 'ESCI' basta prendere l' icona specificata dalla costante 'CMDESCI', e così per tutte le altre. Con questo metodo viene molto semplificata la gestione dei tasti da disegnare: se si vuole disegnare il tasto 'ESCI' basta fare un PutImage dell' elemento CMDESCI dell' ARRAY;
SPRDRIV	vedi G.I.P., unit GIPDRIV;
SPRFONT1	vedi G.I.P., unit GIPFONT1;
SPRFONT2	vedi G.I.P., unit GIPFONT2;

SPRFONT3	vedi G.I.P., unit GIPFONT3;
SPRITE	è il programma principale;
SYSTEM	vedi T.I.P.;
UTILSPR1	contiene procedure e funzioni varie per il programma (parte 1);
UTILSPR2	contiene procedure e funzioni varie per il programma (parte 2).

L'occupazione in memoria del file SPRITE.EXE è di 272 KBytes circa.

Per compilare il programma sono necessari anche i seguenti files: ATT.OBJ, ATT.BGI, CGA.OBJ, CGA.BGI, EGAVGA.OBJ, EGAVGA.BGI, HERC.OBJ, HERC.BGI, PC3270.OBJ, PC3270.BGI, IBM8514.OBJ, IBM8514.BGI, BOLD.OBJ, BOLD.CHR, EURO.OBJ, EURO.CHR, GOTH.OBJ, GOTH.CHR, LCOM.OBJ, LCOM.CHR, LITT.OBJ, LITT.CHR, SANS.OBJ, SANS.CHR, SCRI.OBJ, SCRI.CHR, SIMP.OBJ, SIMP.CHR, TRIP.OBJ, TRIP.CHR, TSCR.OBJ, TSCR.CHR.

Elenco Delle Procedure Per Ogni Unit

L'elenco è in ordine di programma; sarà data anche una breve descrizione della procedura (o funzione). Per ovvi motivi verranno tralasciate le unit standard del Turbo Pascal, quali CRT, DOS, GRAPH, ecc.

T.I.P. v1.0

CRT	unit standard;
DOS	unit standard;
KEYBOARD	sono presenti solo costanti;
MOUSE	- Procedure InstallMouse; Se il mouse è presente (HardWare) e il driver per tale dispositivo è stato caricato (SoftWare), questa procedura disegna sullo schermo il mouse (una freccia se in modalità grafica o un blocco se in modalità testo. La variabile globale MOUSEOK riporta il successo dell'operazione. NOTA: Per controllare se la procedura chiamata (qualsiasi procedura o funzione di questa unit) ha avuto esito positivo o negativo, basta testare il valore della variabile MOUSEERROR. - Procedure ShowMouse; Se il video è in modalità grafica apparirà una freccia, se è in modalità testo apparirà un blocco pieno. - Procedure HideMouse; Permette di nascondere il puntatore del mouse, senza visualizzarlo sul video. Questo è molto utile quando occorre scrivere proprio dove si trova tale puntatore: infatti, se si tenta l'operazione senza prima nascondere, non si otterrà nessun risultato. UNA COSA MOLTO IMPORTANTE: Ad ogni HideMouse corrisponde una ShowMouse, per cui se lo nascondiamo per due volte di seguito

(senza prima rivisualizzarlo) occorre chiamare due volte anche la procedura ShowMouse.

- Procedure GetMPos;

Alla chiamata di questa procedura vengono settate le seguenti variabili al valore letto in quell' stante:

- LeftButton
- MiddleButton
- RightButton
- MouseX
- MouseY
- MouseTextX
- MouseTextY
- MousePressed

Quindi, prima si scrivere una riga del tipo

If MousePressed Then

occorre eseguire 'GetMPos' per aggiornare le variabili appena elencate.

- Procedure SetTMPos (X: Integer; Y: Integer);

Posizione il puntatore del mouse alla posizione X,Y dello schermo (solo in modalità testo).

- Procedure SetGMPos (X: Integer; Y: Integer);

Posizione il puntatore del mouse alla posizione X,Y dello schermo (solo in modalità grafica).

- Procedure SetTHorRange (XMin: Integer; XMax: Integer);

Definisce l' ampiezza di spostamento del mouse in orizzontale (solo in modalità testo). Definendo questa, occorre anche definire quella verticale, altrimenti verrà considerata 1.

- Procedure SetTVertRange (YMin: Integer; YMax: Integer);

Definisce l' ampiezza di spostamento del mouse in verticale (solo in modalità testo). Definendo questa, occorre anche definire quella orizzontale, altrimenti verrà considerata 1.

- Procedure SetGHorRange (XMin: Integer; XMax: Integer);

Definisce l' ampiezza di spostamento del mouse in orizzontale (solo in modalità grafica). Definendo questa, occorre anche definire quella verticale, altrimenti verrà considerata 1.

- Procedure SetGVertRange (YMin: Integer; YMax: Integer);

Definisce l' ampiezza di spostamento del mouse in verticale (solo in modalità grafica). Definendo questa, occorre anche definire quella orizzontale, altrimenti verrà considerata 1.

- Procedure PenOn;

Abilita l' emulazione della penna ottica.

- Procedure PenOff;
Disabilita l' emulazione della penna ottica.
- Procedure SetRatio (X: Integer; Y: Integer);
Sceglie la velocità di movimento del mouse, indipendentemente per le direzioni su-giù (Y) e destra-sinistra (X), definita in N Mickey ogni 8 pixels, dove N vale X o Y. Per valori piccoli il puntatore del mouse è molto veloce.
- Procedure SetMSens (HorSpeed: Integer; VerSpeed: Integer);
Definisce la sensibilità del mouse.
- Procedure GetMSens (Var HorSpeed: Integer; Var VerSpeed: Integer);
Ritorna la sensibilità del mouse.
- Procedure SetPage (N: Integer);
Attiva il mouse nella pagina video numero N.
- Procedure GetPage (Var N: Integer);
Ritorna il numero di pagina N dove il mouse è stato attivato.
- Procedure DisMDrv;
Disabilita il driver software del mouse, ripristinando il vettori degli interrupts per il 10H e il 71H (nel caso la CPU sia un 8086) o 74H (nel caso in cui la CPU sia un 80286 o 80386).
- Procedure EnMDrv;
Abilita il driver software del mouse, ripristinando i vettori rimossi con la chiamata alla procedura DISMDRV.
- Procedure ResetM;
Inizializza il mouse ai valori di default: è identica alla procedura INSTALLMOUSE ma non lo resetta.
- Procedure SetLang (Language: Byte);
Sceglie il linguaggio del mouse. LANGUAGE può assumere i seguenti valori:
 - 0 per ENGLISH
 - 1 per FRENCH
 - 2 per DUTCH
 - 3 per GERMAN
 - 4 per SWEDISH
 - 5 per FINNISH
 - 6 per SPANISH
 - 7 per PORTUGUESE
 - 8 per ITALIAN
 Tutti gli altri valori verranno ignorati.
- Procedure GetLang (Var Language: Byte);
Ritorna il linguaggio del mouse. LANGUAGE può assumere i seguenti valori:
 - 0 per ENGLISH
 - 1 per FRENCH

- 2 per DUTCH
- 3 per GERMAN
- 4 per SWEDISH
- 5 per FINNISH
- 6 per SPANISH
- 7 per PORTUGUESE
- 8 per ITALIAN
- 9 per <Sconosciuta>

- Procedure GetMInfo (Var LoVer: Byte; Var HiVer: Byte;
 Var Interfaccia: Byte; Var IRQNum: Byte);
 Restituisce le informazioni del mouse e del driver caricato.
 LOVER e HIVER sono la versione del driver del mouse
 (es. se LOVER vale 3 e HIVER vale 0, la versione è 3.0);
 Interfaccia assume i seguenti valori:

- 1 per BUS
- 2 per Seriale
- 3 per Microsoft InPort
- 4 per IBM PS/2 Pointing Device Port
- 5 per Hawlett-Packard Mouse

IRQNUM vale:

- 0 per IRQ PS/2 Pointing Device
- 1 per IRQ <Sconosciuto>
- 2 per IRQ numero 2
- . . .
- . . .
- . . .
- 7 per IRQ numero 7

- Function MouseInT (XMin: Integer; YMin: Integer;
 XMax: Integer; YMax: Integer): Boolean;
 Restituisce TRUE se il mouse si trova nell' area delimitata
 da (X1,Y1) e da (X2,Y2) ed è stato premuto un qualsiasi
 pulsante. Questo solo in modalità testo.

- Function MouseInG (XMin: Integer; YMin: Integer;
 XMax: Integer; YMax: Integer): Boolean;
 Restituisce TRUE se il mouse si trova nell' area delimitata
 da (X1,Y1) e da (X2,Y2) ed è stato premuto un qualsiasi
 pulsante. Questo solo in modalità grafica.

- Procedure NewMouseCursor (Cursor: GraphicCursor);
 Cambia la forma del cursore in modalità grafica.

è il programma principale.

unit standard;

- Procedure WriteConfigFile;

Riscrive il file di configurazione che contiene le definizioni
 dei colori, il path per l' help, per le pagine video in
 memoria e sui dischi, per i settaggi del ritardi e delle
 opzioni speciali, per le cornici della finestra, del bordo,
 ecc.

TIP
SYSTEM
TIPBASE

- Procedure ReadConfigFile;
Legge il file di configurazione che contiene le definizioni dei colori, dei ritardi, il path per l' help, l' abilitazione o no del suono emesso dal PC, i pathes per le immagini, ed altre cose: se questo file non esiste sul disco, ne viene creato uno nuovo.
- Procedure InizializzaVariabili;
Setta le variabili globali ai valori di default; cancella le pagine video e i relativi dati; controlla se esiste il file di aiuto; controlla se il video è monocromatico o no; legge il file di configurazione, se esiste, oppure ne crea uno nuovo, se non esiste; controlla se il mouse è installato o no; ecc.
- Procedure LeggiFileMaschera;
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da leggere e lo visualizza sul video. Se l' immagine corrente non è stata salvata chiede se si è disposti a perdere le modifiche apportate.
- Procedure SalvaFileMaschera (NomeFile: String; Finestra: Boolean);
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da salvare e lo registra sul disco. Se esiste un altro file con lo stesso nome viene chiesto se lo si vuole sovrascrivere oppure no.
- Procedure PosizCursore;
Sposta il cursore ad una certa posizione. Se la variabile globale NORMALCURSOR vale TRUE allora il cursore è un blocco lampeggiante, altrimenti (cioè se vale FALSE) è dato da due linee (una orizzontale e l' altra verticale) che si intersecano nella posizione specificata.
- Function CheckCoordBlock: Boolean;
Controlla le coordinate del blocco e visualizza un eventuale messaggio di errore (mancata definizione delle coordinate di inizio o di fine del blocco stesso). La funzione restituisce TRUE quando le coordinate del blocco sono diverse dal valore 0; restituisce FALSE quando una di esse vale 0 (viene anche avvertito l' utente).
- Procedure MarkBeginBlock;
Definisce le coordinate iniziali del blocco.
- Procedure MarkEndBlock;
Definisce le coordinate finali del blocco.
- Function DefineCoordBlock: Boolean;
Definisce il blocco con i tasti cursore (SU, GIU', DESTRA, SINISTRA, HOME, END, PAGE-UP, PAGE-DOWN, ecc.). Il blocco definito viene evidenziato per una più facile lettura. I bordi, visualizzati in negativo, sono compresi nella definizione delle coordinate del blocco. I bordi del blocco sono compresi nel blocco

TIPBLOCK

considerato. La funzione restituisce TRUE se il blocco è stato definito; FALSE se non è stato definito (pressione del tasto ESCAPE).

- Procedure DefineBlock;

Effettua la chiamata alla funzione DEFINECOORDBLOCK per definire le coordinate del blocco.

- Procedure CopyAppBlockToBlock (Var Rec1: RecBlock;
Var Rec2: RecBlock; Var PosX: Byte; Var PosY:
Byte);

Trasferisce la copia del blocco in memoria nella pagina attiva, in modo da esserci sempre la possibilità di premere ESCAPE e di ripristinare la pagina video originale. Questa procedura effettua inoltre tutti i controlli sulle opzioni BLOCCACAR, BLOCCAFORE e BLOCCABACK per impostare gli attributi video opportuni.

- Procedure CopyAppBlockToImage (Var Rec1: RecBlock;
Var Rec2: RecImage; Var PosX: Byte; Var PosY:
Byte);

Trasferisce la copia del blocco in memoria nella pagina attiva, in modo da esserci sempre la possibilità di premere ESCAPE e di ripristinare la pagina video originale. Questa procedura effettua inoltre tutti i controlli sulle opzioni BLOCCACAR, BLOCCAFORE e BLOCCABACK per impostare gli attributi video opportuni. La differenza tra questa procedura e la precedente (COPYAPPBLOCKTOBLOCK) è che qui viene specificata una struttura dati differente (RECIMAGE invece di RECBLOCK).

- Procedure CopyBlock (MemoryBlock: Boolean);

Effettua la copia di un blocco. Se è stato definito in memoria verrà letto da essa, altrimenti verranno chiesti gli estremi degli angoli superiore-sinistro e inferiore-destro del rettangolo. Per definire il blocco basta premere i tasti direzionali e RETURN per confermarlo. Si possono effettuare copie multiple dello stesso blocco, premendo RETURN ogni volta che si vuole copiare il contenuto del blocco sulla pagina attiva. Premendo il tasto ESCAPE si esce dalla procedura.

- Procedure MoveAppBlockToImage (Var Rec1: RecBlock;
Var Rec2: RecImage; Car: Char; Var Colore: Byte);

Trasferisce la copia del blocco in memoria nella pagina attiva, in modo da esserci sempre la possibilità di premere ESCAPE e di ripristinare la pagina video originale. Questa procedura effettua inoltre tutti i controlli sulle opzioni BLOCCACAR, BLOCCAFORE e

BLOCCABACK per impostare gli attributi video opportuni. La struttura dei dati è quella relativa ad una pagina video e non ad un blocco, e l'immagine viene spostata e non copiata. Questa procedura serve solo per cancellare la parte di schermo relativa alle coordinate iniziali del blocco.

- Procedure MoveBlock;

Effettua lo spostamento di un blocco. Se è stato definito in memoria verrà letto da essa, altrimenti verranno chiesti gli estremi degli angoli superiore-sinistro e inferiore-destro del rettangolo. Per definire il blocco basta premere i tasti direzionali e RETURN per confermarlo. Premendo RETURN una volta spostato il blocco, verrà definita la sua nuova posizione nella pagina attiva. Premendo il tasto ESCAPE si esce dalla procedura.

- Procedure EraseVideoBlock;

Cancella il blocco definito dall'utente riempiendolo di caratteri nulli di attributo di colore scelto dall'utente.

- Procedure FillVideoBlock;

Riempie il blocco definito dall'utente del carattere e del colore scelti. Per non riempire il blocco, basta premere il tasto ESCAPE prima di aver scelto il colore.

- Procedure DisegnaCornice (Var Rec: RecImage; InizioX: Byte; InizioY: Byte; FineX: Byte; FineY: Byte; Attr: Byte; Corn: String013);

Disegna una cornice attorno al blocco evidenziato, scegliendo tra i diversi tipi di cornice disponibili. I parametri sono il puntatore della pagina video che si desidera modificare, le coordinate del blocco (o della pagina), il colore della cornice e i caratteri che la compongono.

- Procedure BordVideoBlock;

Contorna il blocco definito dall'utente del colore e della cornice scelti. Per interrompere il processo, basta premere il tasto ESCAPE prima di aver scelto la cornice.

- Procedure InvertBlock (Var Rec: RecImage; InizioX: Byte; InizioY: Byte; FineX: Byte; FineY: Byte; Img: Boolean);

Inverte i colori di un blocco oppure i caratteri di questo in senso verticale od orizzontale. I parametri passati sono il puntatore alla pagina video in memoria, le coordinate di inizio e di fine del blocco (quindi si possono specificare anche per invertire uno schermo intero), e un valore IMG (True o False) per indicare se l'inversione riguarda un blocco (False) oppure l'intera pagina video (True). Se le opzioni InvertXCar o InvertYCar sono attive, verranno invertiti i caratteri che hanno un corrispondente al

contrario; ad esempio i caratteri "^v" "v" "<>" "()" "___"
"L_" ecc.

- Procedure InvertVideoBlock;
Effettua la chiamata alla procedura INVERTBLOCK per invertire il blocco nel modo scelto dall' utente.
- Procedure ReadBlockFile;
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da leggere e lo visualizza sul video. Dopo aver definito il blocco, lo si può spostare a piacere ed effettuare copie multiple.
- Procedure SaveBlockFile;
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da salvare e lo registra sul disco. Se esiste un altro file con lo stesso nome viene chiesto se lo si vuole sovrascrivere oppure no.
- Procedure ReadTextBlock;
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da leggere e lo visualizza sul video. Dopo aver definito il blocco, lo si può spostare a piacere ed effettuare copie multiple. Il file viene letto come testo, per cui il colore è quello di default, mentre i caratteri sul video corrispondono a quelli che si avrebbero con il comando TYPE del DOS.
- Procedure SaveTextBlock;
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da salvare e lo registra sul disco. Se esiste un altro file con lo stesso nome viene chiesto se lo si vuole sovrascrivere oppure no. Il file viene salvato come testo, per cui il colore è quello di default, mentre i caratteri sul video corrispondono a quelli che si avrebbero con il comando TYPE del DOS.
- Procedure StoreClipboardBlock;
Memorizza il blocco definito dall' utente. Se il blocco non è stato definito viene chiesto in input.
- Procedure RestoreClipboardBlock;
Ripristina il blocco memorizzato, se c'è, per copiarlo in qualche zona dell' immagine attiva sullo schermo.
- Procedure MenuBlocchi;
Disegna un menu da cui è possibile marcare l' inizio e la fine di un blocco, copiarlo, spostarlo, cancellarlo, riempirlo, contornarlo, invertirlo (su-giù, destra-sinistra, colori, ecc.), leggerlo e salvarlo sia da/su disco sia da/su memoria, considerandolo come un file di tipo blocco o un file di tipo testo.

TIPCOLOR

- Function ScegliColori: Integer;
Visualizza una finestra da cui è possibile scegliere un qualsiasi colore per il foreground e/o per il background (da quelli che appaiono sul video). Con i tasti

cursore (SINISTRA, DESTRA, SU, GIU', HOME, END, PAGE-UP, PAGE-DOWN) è possibile muoversi all' interno della finestra. Premendo il tasto RETURN o SPAZIO si seleziona il colore puntato se ci si trova su un colore, si attiva o disattiva il lampeggio se ci si trova su di esso oppure si esce accettando le modifiche se ci si trova sull' ultima posizione ('ACCETTA ED ESCI'). Un simbolo ('<F>' per il foreground e '' per i background) indica quale sia il colore selezionato, oltre allo stesso scritto (in parole) in fondo alla finestra. La funzione restituisce il codice del colore scelto in caso di scelta effettuata oppure il valore -1, corrispondente alla pressione del tasto ESCAPE.

- Procedure SelezionaColori;

Effettua la chiamata alla funzione ScegliColori e memorizza il colore scelto (in caso di nessuna scelta, non esegue nulla).

- Procedure UscitaSiNo;

Chiede se si vuole veramente uscire dal programma oppure no. La selezione avviene tramite un piccolo menu a due voci.

- Procedure EndTextImageProcessor;

Scriva qualche riga di commento al programma e termina la sua esecuzione.

TIPCONFIG

- Procedure ModifyColors;

Consente di modificare tutti i colori del programma: i colori devono essere salvati sul file di configurazione (sono in tutto una cinquantina) altrimenti il lavoro effettuato sarà inutile e, alla prossima esecuzione del programma TIP, si ritornerà a quelli memorizzati nel file TIP.CFG oppure a quelli memorizzati all' interno del programma stesso.

- Procedure ModifySpeeds;

Modifica le varie velocità per i ritardi di apparizione dei menu, per quelli di visualizzazione delle informazioni, di lampeggio dei blocchi, ecc. Il discorso fatto per la procedura ModifyColors vale anche per questa e per tutte le altre di questa unit.

- Procedure ModifyWindows;

Modifica i caratteri che compongono la cornice delle finestre di dialogo e dei menu (i tipi di cornice sono gli stessi del menu TRACCIA). Il discorso fatto per la procedura ModifyColors vale anche per questa e per tutte le altre di questa unit.

- Procedure ModifyBordBlock;

Modifica i bordi del blocco selezionato, in modo analogo alla procedura ModifyWindows. Il discorso fatto per la procedura ModifyColors vale anche per questa e per tutte le altre di questa unit.

- Procedure ModifyArrowBlock;

Modifica le frecce dei bordi del blocco selezionato (frecce singole, frecce doppie, caratteri ASCII). Il discorso fatto per la procedura ModifyColors vale anche per questa e per tutte le altre di questa unit.

- Procedure ModifyMouseSpeed;

Modifica i settaggi del mouse (velocità X e Y di movimento). Il discorso fatto per la procedura ModifyColors vale anche per questa e per tutte le altre di questa unit.

- Procedure ModifySpecialOptions;

Modifica le opzioni speciali:

- | | |
|------------------|---|
| 1) BloccaFore: | Blocca il colore di ForeGround |
| 2) BloccaBack: | Blocca il colore di BackGround |
| 3) BloccaCar: | Blocca il carattere |
| 4) ReturnDown: | RETURN per avanzare di una riga o per scrivere l' ultimo carattere digitato |
| 5) InvertiXCar: | Inverti i caratteri X (opzione Inverti Blocco/Schermo Su-Giù) |
| 6) InvertiYCar: | Inverti i caratteri Y (opzione Inverti Blocco/Schermo Destra-Sinistra) |
| 7) EnableSound: | Abilita o disabilita il suono |
| 8) NormalCursor: | Abilita o disabilita il normale lampeggiante cursore |

Il discorso fatto per la procedura ModifyColors vale anche per questa e per tutte le altre di questa unit.

- Procedure LeggiConfigurazione;

Legge il file di configurazione TIP.CFG.

- Procedure SalvaConfigurazione;

Salva la configurazione attuale nel file TIP.CFG. Il discorso fatto per la procedura ModifyColors vale anche per questa e per tutte le altre di questa unit.

- Procedure MenuConfigurazione;

Disegna il menu Configurazione ed esegue l' opzione scelta dall' utente, chiamando la procedura opportuna.

- Procedure TurboPascal;

Converte l' immagine in un file in Turbo Pascal. I tipi di conversione disponibili sono i seguenti:

- 1) GoToXY + Write
- 2) Solo Write
- 3) Colori ottimizzati
- 4) Con WriteStr
- 5) Con Diretta

TIPCONV

- 6) Memoria video
- 7) Con file maschera
- Procedure TurboAssembler;
 Converte l' immagine in un file in Turbo Assembler. I tipi di conversione disponibili sono i seguenti:
 - 1) con le funzioni DOS
 - 2) con le funzioni BIOS
- Procedure TurboC;
 Converte l' immagine in un file in Turbo C. I tipi di conversione disponibili sono i seguenti:
 - 1) GoToXY + CPrintf
 - 2) Senza GoToXY
 - 3) Colori ottimizzati
 - 4) Con Diretta
- Procedure GWBasic;
 Converte l' immagine in un file in GW-Basic. I tipi di conversione disponibili sono i seguenti:
 - 1) Con Locate
 - 2) Senza Locate
- Procedure DataBase;
 Converte l' immagine in un file in DataBase. I tipi di conversione disponibili sono i seguenti:
 - 1) A colori
 - 2) Colori ottimizzati
 - 3) In bianco e nero
- Procedure SequenzeEscape;
 Converte l' immagine in un file in Sequenze Escape, visualizzabile da DOS con il comando TYPE. I tipi di conversione disponibili sono i seguenti:
 - 1) Con posizionamento
 - 2) Senza posizionamento
- Procedure MenuConversione;
 Visualizza il menu di conversione, da cui sono possibili scegliere i seguenti linguaggi:
 - 1) Turbo Pascal
 - 2) Turbo Assembler
 - 3) Turbo C
 - 4) GW-Basic
 - 5) DataBase
 - 6) Sequenze Escape
- Procedure CursorOFF; {Assembler;}
 Nasconde il cursore facendolo scomparire dal video.
- Procedure BlockCursor; {Assembler;}
 Visualizza il cursore come un blocco pieno.
- Procedure LineCursor; {Assembler;}
 Visualizza il cursore come una linea, del tipo '_ '.

TIPFAST

- Function ControlPressed: Boolean;
Restituisce TRUE se il tasto CTRL è premuto; FALSE in caso contrario.
- Function AltPressed: Boolean;
Restituisce TRUE se il tasto ALT è premuto; FALSE in caso contrario.
- Function ShiftPressed: Boolean;
Restituisce TRUE se il tasto SHIFT (di destra o di sinistra) è premuto; FALSE in caso contrario.
- Procedure ApriQuadro (X1: Byte; Y1: Byte; X2: Byte; Y2: Byte; Title: String080; AtBord: Byte; AtFore: Byte; AtTitle: Byte; NumPassi: Byte; Ritardo: Word);
Apre con effetto a scoppio (o zoom) un rettangolo di coordinate massime pari a (X1,Y1) e (X2,Y2) che ha come titolo TITLE. ATBORD, ATFORE ed ATTITLE sono rispettivamente il colore del bordo, il colore del testo nella finestra e il colore del titolo. NUMPASSI è la velocità di apertura del quadro: più il valore è piccolo e più sarà veloce; RITARDO aspetta un certo numero di millisecondi prima di disegnare ogni quadro. I colori sono definiti come:

$\text{<BackGround> * 16 + <ForeGround>}$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

- Function Dialog (Titolo: String080; StA: String; StB: String; StC: String; Tasti: OptionType; AtBord: Byte; AtTitle: Byte; AtText: Byte; AtSel: Byte; AtUnSel: Byte; AtKeySel: Byte; AtKeyUnSel: Byte);
Integer;

Visualizza una finestra di dialogo, al centro dello schermo, che contiene il messaggio passato come parametro in STA, STB ed STC, come nella funzione MENU. Una volta scritte queste righe, divise sempre dal carattere '|', vengono disegnate le scelte che l'utente può effettuare. La variabile TITOLO è il titolo della finestra che verrà aperta. La funzione restituisce il numero del tasto premuto (da 1 in poi) o il valore -1 se è stato premuto il tasto ESCAPE e quindi non è stata scelta nessuna opzione. ATBORD, ATTITLE, ATTEXT, ATSEL, ATUNSEL sono rispettivamente l'attributo del bordo della finestra, quello del titolo e del testo in essa, il colore del tasto selezionato e di quello non selezionato. Le variabili dei tasti sono:

__OK__ per il solo tasto OK

__SI_NO__ per i tasti SI e NO

E' implementato l' utilizzo del mouse. La variabile HELPARG è l' argomento dell' help che viene richiamato con i tasti CTRL-F1.

- Procedure Info (Stringa: String080; Colore: Byte);
Scrive la stringa STRINGA passata come parametro in fondo allo schermo nell' ultima riga, utilizzando il colore COLORE.
- Procedure Scrivi (PosX: Byte; PosY: Byte; St: String080; Attr: Byte; Evid1: Byte; Evid2: Byte; AttrEvid: Byte);
Scrive la stringa ST alle coordinate dello schermo POSX, POSY utilizzando il colore ATTR ed evidenziando (cioè scrivendole con il colore ATTREVID) le lettere di offset EVID1 ed EVID2 rispetto alla stringa stessa.
- Procedure ControlloSpecialKeys (SKey: SpecialMenuType);
Controlla lo stato dei tasti speciali (SHIFT, ALT e CTRL) e fa apparire la finestra opportuna, in relazione allo stato in cui ci si trova. Lo stato è definito dall' enumerativo SKEY, che assume valori diversi (ad esempio) se si è nell' help o nel programma principale.
- Procedure Attendi (Var Ch1: Char; Var Ch2: Char; SKey: SpecialMenuType);
Attende la pressione di un tasto e restituisce il codice ASCII di questo in CH1. Se il tasto è esteso, CH1 vale 0, mentre CH2 contiene il codice del carattere esteso (per la lista di molti dei caratteri estesi vedere la unit KEYBOARD.PAS. Se viene premuto un tasto del mouse, le variabili CH1 e CH2 non vengono modificate.
- Procedure AttendiMouse (Var Ch1: Char; Var Ch2: Char; SKey: SpecialMenuType; Var KeybPressed: Boolean);

Attende la pressione di un tasto e restituisce il codice ASCII di questo in CH1. Se il tasto è esteso, CH1 vale 0, mentre CH2 contiene il codice del carattere esteso (per la lista di molti dei caratteri estesi vedere la unit KEYBOARD.PAS). Oltre a questo controlla lo stato del mouse e ne segnala la pressione o il movimento. Se invece è stata premuta la tastiera si esce ugualmente e il valore logico della variabile KEYBPRESSED è TRUE. SKEY specifica quale menu far apparire alla pressione dei tasti speciali (SHIFT, ALT e CTRL).

TIPFILES

- Function MenuASCII: Char;
Disegna un menu da cui si può scegliere un carattere ASCII (in tutto sono 256). Per uscire senza modifiche basta premere il tasto ESCAPE. La funzione restituisce il carattere scelto nel caso si preme il tasto RETURN o il carattere nullo (codice ASCII 0) nel caso in cui si preme il

- tasto ESCAPE.
- Procedure ScegliASCII;
Effettua la chiamata alla funzione MENUASCII e stampa sul video il carattere scelto (in caso di nessuna scelta, non stampa nulla).
 - Function MenuCornice: Char;
Disegna un menu da cui si può scegliere il carattere ASCII corrispondente ad un angolo di un rettangolo o quadrato, per facilitare la creazione o la messa a punto di tabelle o righe. Per uscire senza modifiche basta premere il tasto ESCAPE. La funzione restituisce il carattere scelto nel caso si preme il tasto RETURN o il carattere nullo (codice ASCII 0) nel caso in cui si preme il tasto ESCAPE.
 - Procedure ScegliCornice;
Effettua la chiamata alla funzione MENCORNICE e stampa sul video il carattere scelto (in caso di nessuna scelta, non stampa nulla).
 - Function SelezionaCornice: String013;
Disegna un menu dal quale è possibile scegliere il tipo di cornice da utilizzare. Se non si effettua nessuna selezione, la funzione restituisce il carattere ASCII nullo.
 - Function DiskIOStatus (DriveSpec: Integer): Byte;
Verifica lo stato dei floppy disks e degli hard disks, e restituisce un numero intero che corrisponde al codice di errore compiuto:

0	Drive pronto e non protetto da scrittura
1	Drive pronto e protetto da scrittura
2	Drive non pronto
3	Disco non formattato
4	Il drive floppy richiesto non esiste
99	Errore fatale

- La funzione restituisce 0 se si è specificato un hard disk, senza controllare eventuali errori critici.
- Procedure LeggiFileMaschera;
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da leggere e lo visualizza sul video. Se l'immagine corrente non è stata salvata chiede se si è disposti a perdere le modifiche apportate.
 - Procedure SalvaFileMaschera (NomeFile: String; Finestra: Boolean);
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da salvare e lo registra sul disco. Se esiste un altro file con lo stesso nome viene chiesto se lo si vuole sovrascrivere oppure no.
 - Procedure LeggiFileTesto;

- Esegue la funzione INPUTFILE per ricevere in input il nome di un file da leggere e lo visualizza sul video. Se l'immagine corrente non è stata salvata chiede se si è disposti a perdere le modifiche apportate. Il file viene letto come testo, per cui il colore è quello di default, mentre i caratteri sul video corrispondono a quelli che si avrebbero con il comando TYPE del DOS.
- Procedure SalvaFileTesto;
Esegue la funzione INPUTFILE per ricevere in input il nome di un file da salvare e lo registra sul disco. Se esiste un altro file con lo stesso nome viene chiesto se lo si vuole sovrascrivere oppure no. Il file viene salvato come testo, per cui il colore è quello di default, mentre i caratteri sul video corrispondono a quelli che si avrebbero con il comando TYPE del DOS.
 - Procedure CambiaDirectoryCorrente;
Apre una finestra uguale a quella per l'immissione di un nome di file ma solo che questa volta occorre digitare una directory. Se la directory immessa non esiste viene visualizzato un messaggio di errore a video.
 - Procedure ListaDirectory;
E' molto simile alla procedura CAMBIADIRECTORYCORRENTE, solo che questa volta, invece di listare il path immesso si esegue un 'CHDIR', come si farebbe in DOS.
 - Procedure AccessoAlDOS (Comando: String080);
E' la procedura che esegue realmente il comando specificato da COMANDO, cambiando il prompt nel nuovo environment, comprime la memoria dello heap, ecc.
 - Procedure EseguiComandoDOS;
Esegue un qualsiasi comando che si darebbe in DOS, restando però con il programma caricato in memoria. Una volta terminata l'esecuzione di ciò che è stato digitato viene aspettato un RETURN per tornare a TIP. Se non c'è abbastanza memoria per eseguire il comando o se non viene trovato il file COMMAND.COM, viene visualizzata una finestra con il messaggio di errore opportuno.
 - Procedure ShellDOS;
Questa procedura non si differenzia molto da quella che esegue un comando DOS, tranne per il fatto che il comando da eseguire è ora COMMAND.COM. Una volta che si desidera ritornare a TIP basta digitare EXIT. Se non c'è abbastanza memoria per eseguire il comando o se non viene trovato il file COMMAND.COM, viene visualizzata una finestra con il messaggio di errore opportuno.
 - Procedure MenuFiles;
Disegna il menu files ed attende una scelta da parte dell'

utente: a seconda dell' opzione selezionata, esegue il compito specifico.

TIPHELP

- Procedure Stato (NumPgVideo: Byte; Row: Byte; Col: Byte; Attr: Byte; SelCar: Char; Modify: Boolean);
Aggiorna il numero di riga, di colonna, di pagina, il colore corrente, l' ultimo carattere selezionato, lo stato dell' immagine (modificata o no), ecc., scrivendoli nell' ultima riga del video (riga di stato). Tutti questi campi vengono passati come parametro alla procedura.
- Procedure Help (Argomento: String; Tipo: TipoEsciHelp);
Apre una finestra di aiuto e visualizza l' argomento ARGOMENTO passato come parametro. Se non viene trovato nel file di aiuto, l' errore è segnalato all' utente. Se il file dell' help non viene trovato all' inizio del programma (quando si digita da DOS il comando TIP), la variabile HELPONLINE viene settata a FALSE: in questo caso la pressione dei tasti F1, ALT-F1, CTRL-F1, SHIFT-F1 e ALT-H non avranno effetto sul programma. Per richiamare l' help si possono utilizzare 5 combinazioni di tasti:

F1	Help generale
ALT-F1	Help precedente
CTRL-F1	Help specifico
SHIFT-F1	Indice degli argomenti
ALT-H	Help generale (solo nel menu principale).

TIPIMAGE

- Procedure SelezionaImmagine;
Effettua la chiamata alla funzione SCEGLIIMMAGINE per scegliere la pagina in memoria da rendere attiva (fra le 11 disponibili).

TIPINDIR

- Function InputDirectory (Title: String; ExtFiles: String004; Pezza: String; Allowed: SetOfChar): String;
Apre una finestra di dialogo in cui è possibile specificare il nome della nuova directory di default. In alto si può immettere il nome desiderato: se il nome che l' utente vuole immettere è più lungo dello spazio riservato, la stringa scorrerà verso sinistra, e sarà possibile ritornare indietro con i tasti cursore (freccia DESTRA e freccia SINISTRA). Con i tasti TAB e SHIFT-TAB si può passare dall' editing del drive/percorso alla scelta della sub-directory nella lista e viceversa. Una volta entrati in questa la si può scorrere con i tasti freccia GIU' e freccia SU, selezionabile con il tasto RETURN. Premendo ESCAPE si annulla la selezione, e il risultato della funzione sarà una stringa vuota; viceversa conterrà il nome del file digitato o scelto. La colonna più a destra serve per visualizzare la posizione dell' evidenziatore rispetto al resto della lista. In

basso sono visualizzati il numero di directories totali, il totale della dimensione occupata in bytes, lo spazio libero (sempre in bytes) del disco corrente e il numero di sub-directory evidenziata. Il nome della directory da ricercare è il nome che si digita per una ricerca più veloce: una volta che si è nella lista della directory per sceglierne una, basta digitarla, per esteso o in parte, e l' evidenziatore verrà posizionato sul primo nome che presenta la stringa digitata o solo le sue iniziali (la cosa è la stessa per le directories o per i files; in Turbo Pascal v7.0 (o 6.0) invece si ha un comportamento leggermente diverso). In fondo alla finestra, prima dei bytes totali, dello spazio libero e delle altre informazioni, è scritto il path corrente. E' inoltre possibile inserire del testo premendo il tasto INSERT. Tutto può essere effettuato anche con l' ausilio del mouse.

TIPINFIL

- Function InputFile (Title: String; ExtFiles: String004; Pezza: String; Allowed: SetOfChar): String;

Aprire una finestra di dialogo in cui è possibile specificare il nome del file da leggere, salvare o altro. In alto si può immettere il nome del file desiderato: se il nome che l' utente vuole immettere è più lungo dello spazio riservato, la stringa scorrerà verso sinistra, e sarà possibile ritornare indietro con i tasti cursore (freccia DESTRA e freccia SINISTRA). Con i tasti TAB e SHIFT-TAB si può passare dall' editing del percorso/nomefile alla scelta del file nella lista della directory e viceversa. Una volta entrati in questa la si può scorrere con i tasti freccia GIU' e freccia SU, selezionabile con il tasto RETURN. Premendo ESCAPE si annulla la selezione, e il risultato della funzione sarà una stringa vuota; viceversa conterrà il nome del file digitato o scelto. La colonna più a destra serve per visualizzare la posizione dell' evidenziatore rispetto al resto della lista. In basso sono visualizzati il numero di files totali, il totale della dimensione occupata in bytes, lo spazio libero (sempre in bytes) del disco corrente e il numero di file evidenziato. Il nome del file da ricercare è il nome del file che si digita per una più veloce ricerca: una volta che si è nella lista della directory per scegliere un file, basta digitarlo, per esteso o in parte, e l' evidenziatore verrà posizionato sul primo file che presenta il nome digitato o solo le sue iniziali (la cosa è la stessa per le directories o per i files; in Turbo Pascal v7.0 (o 6.0) invece si ha un comportamento leggermente diverso). In fondo alla finestra, prima dei bytes totali, dello spazio libero e delle altre informazioni, è scritto il path corrente. E' inoltre possibile inserire del testo premendo il tasto INSERT. Tutto può essere effettuato anche con l' ausilio del mouse.

TIPINIT

- Procedure WriteCar (Carattere: Char);
Scrive il carattere CARATTERE passato come parametro sul video ed aggiorna la riga e la colonna.
- Procedure Beep;
Produce un BEEP, un suono di 1000 Hz per circa 300 millisecondi
- Procedure SetStandardColors;
Riporta i colori ai valori di default, quelli cioè memorizzati nel codice del programma.
- Procedure CheckHelpFile;
Controlla se il file che contiene le schermate di aiuto esiste o no sul disco: se non esiste si può digitare il nuovo percorso oppure si può decidere di non utilizzarlo durante l'esecuzione del programma.
- Function ModifyColor (OldAttr: Byte; NewAttr: Byte; Row: Byte; Col: Byte): Byte;
Permette di modificare i colori delle finestre, del testo in esse, del testo dei menu, delle pagine attive, e tutti i colori presenti (sono circa una cinquantina).
- Procedure ApriFileMaschera (NomeFile: String; PgVideo: Byte);
Ricerca sul disco il file specificato da NOMEFILE nella pagine video PGVIDEO. Se il file non esiste fisicamente, o se è stato specificato un percorso inesistente, viene visualizzato un messaggio di errore. Se la pagina PGVIDEO è stata modificata viene chiesto se si desidera procedere con la lettura del file o no. Prima di caricare il file viene controllato che sia il tipo supportato dal programma (file maschera).
- Procedure ApriFileTesto (NomeFile: String; PgVideo: Byte);
Ricerca sul disco il file specificato da NOMEFILE nella pagine video PGVIDEO. Se il file non esiste fisicamente, o se è stato specificato un percorso inesistente, viene visualizzato un messaggio di errore. Se la pagina PGVIDEO è stata modificata viene chiesto se si desidera procedere con la lettura del file o no. Non viene fatto nessun controllo sul tipo di file specificato.

TIPINSTR

- Function InputString (StrInput: String080; InAttr: Byte; LungVirt: Byte; LungReale: Byte; Pezza: String; OutAttr: Byte; ArrowAttr: Byte; Allowed: SetOfChar): String;
Attende l' input da tastiera di una stringa. Se la stringa supera la lunghezza consentita, si potrà continuare a digitarla continuando a scrivere: il testo scorrerà verso destra e sarà possibile rivederlo premendo il tasto FRECCIA SINISTRA. Se si ha familiarità con l' ambiente integrato (IDE) del Turbo Pascal versione 7.0 (o anche 6.0) si noterà la forte somiglianza. I parametri in input sono la

stringa di richiesta di immissione (STRINPUT), il suo attributo di colore (INATTR), il numero di caratteri da visualizzare (LUNGVIRT) e quello da memorizzare (LUNGREALE, non oltre i 255 caratteri), la stringa che verrà stampata subito (PEZZA, per immissioni parziali), l'attributo video della stringa che verrà immessa (OUTATTR), l'attributo video dei caratteri freccia (ARROWATTR) e, per finire, i caratteri ammessi nell'immissione (ALLOWED). Le coordinate della stringa sono quelle della riga di stato, ossia quelle dell'ultima riga dello schermo (1,25). Vengono inoltre disegnate le due frecce ai lati, nel caso in cui la stringa fosse più lunga del testo visualizzato sul video. La funzione restituisce la stringa immessa e la stringa nulla nel caso in cui venga premuto il tasto ESCAPE. Il mouse non è operativo nell'editing.

TIPMAIN

- Procedure MenuPrincipale;
Visualizza il menu principale, richiamabile premendo il tasto ESCAPE, e attende una scelta da parte dell'utente. Una volta effettuata tale scelta manda in esecuzione la procedura opportuna. Premendo il tasto ESCAPE si esce dal menu senza effettuare alcuna scelta.
- Procedure EditScreen;
E' la procedura principale, che gestisce le chiamate ai menu, all'help, tutti gli spostamenti del cursore e tutte le altre cose che si possono fare nel programma.

TIPMENU

- Function Menu (Titolo: String080; StA: String; StB: String; StC: String; StD: String; StE: String; StF: String; StG: String; StH: String; AtTitle: Byte; AtSel: Byte; AtUnSel: Byte; AtBord: Byte; AtText: Byte; AtKeySel: Byte; AtKeyUnSel: Byte; Type: SpecialMenuType): Integer;
Disegna un menu al centro dello schermo ed attende una selezione da parte dell'utente: l'utente può utilizzare i tasti cursore e RETURN oppure si può spostare con il mouse e premere due volte il pulsante sinistro dello stesso. Premendo il tasto F1 sono sempre disponibili ulteriori informazioni. TITOLO è il titolo del menu, mentre StA, StB, StC, ecc. definiscono le opzioni del menu. Il programma separa tutte le opzioni elencate nelle stringhe, da considerarsi però una unica, facendo in modo che termini con il carattere '|'; il numero di opzioni disponibili è noto, sapendo il numero di questi caratteri. Il numero che restituisce la funzione è quello della riga selezionata. Restituisce -1 nel caso in cui non venga scelto niente o premuto il tasto ESCAPE o il pulsante DESTRO del mouse. Il numero minimo di righe è 1, in quanto il titolo è considerato una cosa a parte. ATTITLE, ATSEL,

ATUNSEL, ATBORD ed ATTEXT sono rispettivamente gli attributi del titolo, dell' opzione selezionata, di quella non selezionata, del bordo della finestra e della finestra stessa. Il testo nella finestra sarà scritto con attributo ATUNSEL. I colori sono definiti come:

$$<\text{BackGround}> * 16 + <\text{ForeGround}>$$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

TIPTIME

- Procedure Time1; Interrupt;
Questa procedura è collegata all' interrupt 09h (tastiera) e visualizza, nella riga di stato, i settaggi di CapsLock, ScrollLock, NumLock ed Insert. Gli stati dei tasti speciali sono prelevate dall' indirizzo di memoria 0000h:0417h (Byte).
- Procedure Time2; Interrupt;
Questa procedura è collegata all' interrupt 1Ch (timer) e visualizza, nella riga di stato, l' ora e i minuti (con i secondi che fanno lampeggiare i due punti - ":") e settaggi di LeftShift, RightShift, Ctrl, Alt. L' ora è prelevata dalla locazione di memoria 0000h:046Ch (LongInt), mentre gli stati dei tasti speciali sono prelevate dall' indirizzo di memoria 0000h:0417h (Byte).
- Procedure NoTime;
Ripristina gli indirizzi memorizzati nella RAM degli interrupt 1Ch e 09h, modificati all' inizio del programma.

TIPTRACE

- Procedure MenuTraccia;
Attende che venga scelto il tipo di carattere da utilizzare come traccia oppure il colore da modificare.
- Procedure TraceControl (Car: Char);
Aggiorna in modo opportuno il colore e l' attributo della posizione corrente del cursore nella pagina video attiva.

TIPVAR

TIPVIDEO

- non ci sono procedure;
- Procedure MoveVideo;
Effettua lo spostamento di un blocco. Se è stato definito in memoria verrà letto da essa, altrimenti verranno chiesti gli estremi degli angoli superiore-sinistro e inferiore-destro del rettangolo. Per definire il blocco basta premere i tasti direzionali e RETURN per confermarlo. Premendo RETURN una volta spostato il blocco, verrà definita la sua nuova posizione nella pagina attiva. Premendo il tasto ESCAPE si esce dalla procedura.
 - Function ScegliImmagine: Byte;
Visualizza una finestra dalla quale è possibile scegliere una immagine tra quelle disponibili (sono in tutto 11). Queste

sono numerate (da 1 a 11) ed è specificato se sono state modificate (il simbolo appare subito dopo il numero), il nome del file associato (se esiste) e il tipo di file (maschera o testo).

- Procedure CopyVideo;
Effettua la copia di un blocco. Se è stato definito in memoria verrà letto da essa, altrimenti verranno chiesti gli estremi degli angoli superiore-sinistro e inferiore-destro del rettangolo. Per definire il blocco basta premere i tasti direzionali e RETURN per confermarlo. Si possono effettuare copie multiple dello stesso blocco, premendo RETURN ogni volta che si vuole copiare il contenuto del blocco sulla pagina attiva. Premendo il tasto ESCAPE si esce dalla procedura.
- Procedure MoveAppVideoToImage (Var Rec1: RecPage;
Var Rec2: RecImage; Car: Char; Var Colore: Byte);
Trasferisce la copia del video in memoria nella pagina attiva, in modo da esserci sempre la possibilità di premere ESCAPE e di ripristinare la pagina video originale. Questa procedura effettua inoltre tutti i controlli sulle opzioni BLOCCACAR, BLOCCAFOR e BLOCCABACK per impostare gli attributi video opportuni. La struttura dei dati è quella relativa ad una pagina video, e l'immagine viene spostata e non copiata. Questa procedura serve solo per cancellare lo schermo e riempirlo di un certo attributo/carattere.
- Procedure EraseVideo;
Cancella il blocco definito dall'utente riempiendolo di caratteri nulli di attributo di colore scelto dall'utente.
- Procedure FillVideo;
Riempie il blocco definito dall'utente del carattere e del colore scelti. Per non riempire il blocco, basta premere il tasto ESCAPE prima di aver scelto il colore.
- Procedure BordVideo;
Contorna il blocco definito dall'utente del colore e della cornice scelti. Per interrompere il processo, basta premere il tasto ESCAPE prima di aver scelto la cornice.
- Procedure InvertVideo;
Effettua la chiamata alla procedura INVERTBLOCK per invertire la pagina nel modo scelto dall'utente (Su-Giù, Destra-Sinistra, Colori).
- Procedure StoreClipboardVideo;
Memorizza il blocco definito dall'utente. Se il blocco non è stato definito viene chiesto in input.
- Procedure RestoreClipboardVideo;
Ripristina il blocco memorizzato, se c'è, per copiarlo in qualche zona dell'immagine attiva sullo schermo.

TIPWIN

- Procedure MenuSchermo;
Disegna un menu da cui è possibile marcare l' inizio e la fine di un blocco, copiarlo, spostarlo, cancellarlo, riempirlo, contornarlo, invertirlo (su-giù, destra-sinistra, colori, ecc.), leggerlo e salvarlo sia da/su disco sia da/su memoria, considerandolo come un file di tipo blocco o un file di tipo testo.

- Procedure WriteStr (X: Byte; Y: Byte; S: String; Attr: Byte);
Scrive una stringa direttamente in memoria video. I parametri sono la posizione x (X), quella y (Y), la stringa da visualizzare (S) e il colore della stessa (ATTR). Il colore è definito come:

$$<\text{BackGround}> * 16 + <\text{ForeGround}>$$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

- Procedure WriteChar (X: Byte; Y: Byte; Count: Byte; Ch: Char; Attr: Byte);
Scrive un carattere direttamente in memoria video. I parametri sono la posizione x (X), quella y (Y), il carattere da visualizzare (CH), il colore dello stesso (ATTR) e il numero di volte che si desidera stamparlo (COUNT) (questo significa che se COUNT vale 3 e CH vale 'A', viene stampata la stringa 'AAA'. Il colore è definito come:

$$<\text{BackGround}> * 16 + <\text{ForeGround}>$$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

- Procedure FillWin (Ch: Char; Attr: Byte);
Riempie la finestra corrente con un certo carattere (CH) e colore (ATTR). Il colore è definito come:

$$<\text{BackGround}> * 16 + <\text{ForeGround}>$$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

- Procedure ReadWin (Var Buf);
Legge il contenuto di una finestra dalla memoria video. Il parametro BUF è un puntatore all' indirizzo di partenza dal quale verrà salvata la zona di schermo da coprire.
- Procedure WriteWin (Var Buf);
Scrive il contenuto di una finestra in memoria sul video. E'

ovvio che per poterla ripristinare occorre averla salvata precedentemente. Il parametro BUF è un puntatore all'indirizzo di partenza dal quale verrà letta la zona di schermo da ripristinare.

- Function WinSize: Word;
Ritorna la dimensione della finestra in occupazione di memoria.
- Procedure SaveWin (Var W: WinState);
Salva i parametri della finestra corrente, come il colore, la posizione, ecc. Il parametro W è un record formato dai seguenti campi:

```
WinState=   Record
            WindMin:   Word;
            WindMax:   Word;
            WhereX:    Byte;
            WhereY:    Byte;
            TextAttr:  Byte;
            End;
```

'WindMin' e 'WindMax' sono rispettivamente gli angoli in alto a sinistra e in basso a destra. Per avere le coordinate del primo occorre prendere la parte meno significativa del valore, tramite la funzione LO (WindMin); la stessa cosa funziona con quella più significativa (HI (WindMin)). I campi 'WhereX' e 'WhereY' memorizzano la posizione della finestra rispetto allo schermo. 'TextAttr' serve per memorizzare il colore, definito come:

$\text{<BackGround> * 16 + <ForeGround>}$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

- Procedure RestoreWin (Var W: WinState);
Ripristina i parametri nella finestra corrente, come il colore, la posizione, ecc. Il parametro W è un record formato dai seguenti campi:

```
WinState=   Record
            WindMin:   Word;
            WindMax:   Word;
            WhereX:    Byte;
            WhereY:    Byte;
```

TextAttr: Byte;
End;

'WindMin' e 'WindMax' sono rispettivamente gli angoli in alto a sinistra e in basso a destra. Per avere le coordinate del primo occorre prendere la parte meno significativa del valore, tramite la funzione LO (WindMin); la stessa cosa funziona con quella più significativa (HI (WindMin)). I campi 'WhereX' e 'WhereY' memorizzano la posizione della finestra rispetto allo schermo. 'TextAttr' serve per memorizzare il colore, definito come:

$\text{<BackGround> * 16 + <ForeGround>}$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

- Procedure FrameWin (Title: TitleStr; Var Frame: FrameChars;
TitleAttr: Byte; FrameAttr: Byte);

Disegna il bordo della finestra. I parametri sono il titolo (TITLE), il tipo di bordo (FRAMECHARS), il colore del titolo (TITLEATTR) e il colore del bordo (FRAMEATTR). I colori sono definiti come:

$\text{<BackGround> * 16 + <ForeGround>}$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128). Il tipo FRAMECHARS è una stringa che contiene i caratteri del bordo. Ci sono quattro tipi di bordi SingleFrame, DoubleFrame SingleHorFrame (o DoubleVerFrame) e SingleVerFrame (DoubleHorFrame).

- Procedure UnFrameWin;

Cancella il bordo della finestra.

- Procedure ActiveWindow (Active: Boolean; Var Frame: FrameChars);

Attiva una finestra disegnandone il bordo in un certo modo.

Il tipo FRAMECHARS è una stringa che contiene i caratteri del bordo. Ci sono quattro tipi di bordi SingleFrame, DoubleFrame SingleHorFrame (o DoubleVerFrame) e SingleVerFrame (DoubleHorFrame).

- Procedure OpenWindow (X1: Byte; Y1: Byte; X2: Byte; Y2: Byte;
T: TitleStr; TAttr: Byte; FAttr: Byte;
Var Frame: FrameChars);

Apri una finestra di coordinate (X1,Y1) per l'angolo in

alto a sinistra e (X2,Y2) per quello in basso a destra. Il titolo è definito dalla variabile T; TATTR è il colore del titolo, mentre FATTR è il colore all' interno della finestra. I colori sono definiti come:

$\text{<BackGround> * 16 + <ForeGround>}$

Se si desidera un colore lampeggiante, aggiungere 128 al risultato (se si usa la unit CRT della Borland, si può usare la costante BLINK, che vale appunto 128).

Il tipo FRAMECHARS è una stringa che contiene i caratteri del bordo. Ci sono quattro tipi di bordi SingleFrame, DoubleFrame SingleHorFrame (o DoubleVerFrame) e SingleVerFrame (DoubleHorFrame).

- Procedure CloseWindow;

Chiude la finestra corrente ripristinando il contenuto della zona di schermo precedente. Se è presente un' altra finestra questa viene resa attiva disegnando il bordo scelto alla sua apertura.

- Procedure InitializeWindow;

Questa procedura deve essere eseguita prima di eseguire qualsiasi procedura di questa unit, in quanto inizializza i contatori e libera la memoria di partenza per i record delle finestre, ecc.

G.I.P. v1.0

CRT
DOS
GIP
GIPAPPPRC

vedi T.I.P.;

vedi T.I.P.;

è il programma principale;

- Function ConsensoNuovo: Boolean;

Attende che l' utente risponda SI o NO alla domanda 'Vuoi iniziare un nuovo disegno ?'.

- Function AttendiInizio: Boolean;

Attende che l' utente preme il mouse: se lo preme nella finestra di disegno allora viene utilizzato lo strumento attivo (disegnato un cerchio, rettangolo, testo, ecc.) altrimenti il controllo ritorna alla procedura chiamante.

GIPBASE

- Procedure GraphicScreen;

Disegna la schermata iniziale del programma, completa delle icone laterali, quelle dei colori, le barre di scorrimento, ecc.

- Procedure EseguiProcedura (NomeMenu: Byte; NomeOpz: Byte);

Esegue l' opzione NOMEOPZ del menu NOMEMENU.

- Procedure DisegnaImmagine (X1: Integer; Y1: Integer; X2: Integer; Y2: Integer; Clr: Boolean);

Visualizza l' intero disegno nella finestra video specificata da X1, X2, Y1 e Y2, scorrendo la lista dall' inizio alla fine.
Se CLR è True viene prima cancellata la finestra.

- Procedure DisegnaFigura (X1: Integer; Y1: Integer; X2: Integer; Y2: Integer; Op: RecOp; Var P:

PTROperation);

Visualizza la figura, contenuta nell' elemento OP della lista in memoria, nella finestra specificata dalle coordinate X1, X2, Y1 e Y2.

- Procedure AggiornaLineBar (Direzione: TipoDirezione);
Aggiorna la barra di scorrimento dell' immagine: a seconda del valore di DIREZIONE verrà aggiornata quella orizzontale o quella verticale.
- Procedure TestArrow;
Verifica se è stato premuto il mouse su una delle quattro frecce.
- Procedure TestStrumenti;
Verifica se è stato premuto il mouse su un' icona strumento (laterale).
- Procedure TestColori;
Verifica se è stato premuto il mouse su un' icona colore (in basso).
- Procedure AggiornaColori;
Aggiorna il colore attuale nello spazio riservato al colore selezionato.

GIPCONV

- Procedure ConvertiFile (Ling: TipoLing);
A seconda del valore di LING converte il file in Turbo Pascal, in Turbo C oppure in Turbo Assembler.

GIPDRIV

- Procedure ATTDriverProc;
Procedura relativa al file 'ATT.OBJ'.
- Procedure CgaDriverProc;
Procedura relativa al file 'CGA.OBJ'.
- Procedure EgaVgaDriverProc;
Procedura relativa al file 'EGAVGA.OBJ'.
- Procedure HercDriverProc;
Procedura relativa al file 'HERC.OBJ'.
- Procedure PC3270DriverProc;
Procedura relativa al file 'PC3270.OBJ'.

GIPFAST

- Procedure WaitToWrite;
Serve per evitare l' effetto grafico di sfibrillamento, attendendo che il ritracciamento del video arrivi in fondo all' ultima posizione.
- Procedure ReleaseMouse;
Rilascia i pulsanti del mouse.
- Function ShiftPressed: Boolean;
Ritorna il valore logico TRUE se un tasto SHIFT è premuto; ritorna FALSE in caso contrario.

GIPFILE

- Function ControlPressed: Boolean;
Ritorna il valore logico TRUE se il tasto CTRL è premuto;
ritorna FALSE in caso contrario.
- Function AltPressed: Boolean;
Ritorna il valore logico TRUE se il tasto ALT è premuto;
ritorna FALSE in caso contrario.
- Function NumLockActivated: Boolean;
Ritorna il valore logico TRUE se NUMLOCK è attivato;
ritorna FALSE in caso contrario.
- Procedure WriteWin (Titolo: String; X1: Integer; Y1: Integer;
X2: Integer; Y2: Integer; Sfondo: Byte; Riemp: Byte;
Color: Byte);
Disegna una finestra in modalità grafica di coordinate
iniziali (X1,Y1) e coordinate finali di (X2,Y2), con sfondo
di colore RIEMP, titolo (che vale TITOLO) di colore
COLOR su background SFONDO.
- Procedure WriteLine (Titolo: String; X1: Integer; Y1: Integer;
X2: Integer; Y2: Integer; Sfondo: Byte; Riemp: Byte;
Color: Byte);
Disegna una finestra in modalità grafica di coordinate
iniziali (X1,Y1) e coordinate finali di (X2,Y2), con sfondo
di colore RIEMP, titolo di colore COLOR su background
SFONDO; la finestra è formata da una linea sola, per cui il
testo verrà posizionato a destra di TITOLO.
- Function Warning (St1: String; St2: String; NumIcon: Byte);
Boolean;
Aprire una finestra di dialogo in cui è possibile effettuare
una scelta fra SI o NO (NUMICONE=2) oppure la
semplice selezione OK (NUMICONE=1). Nel primo caso
restituisce TRUE se è stato scelto SI e FALSE se è stato
scelto NO; nel secondo invece restituisce sempre TRUE.
- Procedure WriteCursor (X1: Integer; X2: Integer; Y: Integer;
InsState: Boolean);
Cancella il cursore alla posizione X1 e lo riscrive a quella
X2, tenendo sempre Y fisso.
- Procedure ReadFile (NomeFile1: String080);
Legge un file di disegno dal disco (*.GIP), compreso
quello di stringhe (*.STR); se NomeFile1 è una stringa
vuota viene aperta una finestra completa di files,
directories, drives, ecc.; se contiene il nome di un file viene
letto senza aprire la finestra suddetta.
- Procedure WriteFile (NomeFile1: String080);
Scrivere un file di disegno sul disco (*.GIP), compreso
quello di stringhe (*.STR); se NomeFile1 è una stringa
vuota viene aperta una finestra completa di files,
directories, drives, ecc.; se contiene il nome di un file viene
letto senza aprire la finestra suddetta.

- Procedure ReadPalette (NomeFile: String080);
Legge un file di colori dal discc (*.PAL); se NomeFile è una stringa vuota viene aperta una finestra completa di files, directories, drives, ecc.; se contiene il nome di un file viene letto senza aprire la finestra suddetta.
- Procedure WritePalette (NomeFile: String080);
Salva un file di colori sul discc (*.PAL); se NomeFile è una stringa vuota viene aperta una finestra completa di files, directories, drives, ecc.; se contiene il nome di un file viene letto senza aprire la finestra suddetta.
- Procedure InputFile (TitWin: String; Var NomeFile: String080; Ext: String003; Operaz: TipoOpFile; FileView: TipoView);
E' la procedura che apre la finestra di selezione di un file, con directories, drives, percorsi, visualizzazioni, informazioni sui files, ecc.

GIPFONT1

- Procedure GothicFontProc;
Procedura relativa al file 'GOTH.CHR'.
- Procedure SansSerifFontProc;
Procedura relativa al file 'SANS.CHR'.
- Procedure SmallFontProc;
Procedura relativa al file 'LITT.CHR'.

GIPFONT2

- Procedure TriplexFontProc;
Procedura relativa al file 'TRIP.CHR'.
- Procedure ScryptFont;
Procedura relativa al file 'SCRI.CHR'.
- Procedure SimpleFont;
Procedura relativa al file 'SIMP.CHR'.
- Procedure TSCRFont;
Procedura relativa al file 'TSCR.CHR'.

GIPFONT3

- Procedure LComFont;
Procedura relativa al file 'LCOM.CHR'.
- Procedure EuroFont;
Procedura relativa al file 'EURO.CHR'.
- Procedure BoldFont;
Procedura relativa al file 'BOLD.CHR'.

GIPGRAPH

- Procedure ControllaTastiera;
Controlla lo stato della tastiera: se è premuta memorizza nella variabili globali CHAR1 e CHAR2 i codici ASCII corrispondente al contenuto del buffer della tastiera, altrimenti li inizializza a #0 (kNull).
- Function PremiPulsante (X: Integer; Y: Integer; Icn: Pointer; Zona: TipoZona; Icona: Byte; Attesa: TipoAttesa; Sfondo: Byte): Boolean;
Preme un pulsante con l' effetto tridimensionale a ombra: se il mouse è premuto e si trova nell' area delimitata dal pulsante, è tenuto schiacciato; altrimenti è riportato allo

stato originale. X e Y sono le coordinate sul video, mentre ICN è il puntatore ai dati numerici (memoria immagine). ZONA indica la zona in cui viene premuto, ICONA il numero di icona progressivo (sono tutte numerate); ATTESA indica se bisogna attendere il rilascio dei pulsanti del mouse per continuare; L' ombra sarà riempita del colore SFONDO.

- Procedure GetMouse (Coordinate: Boolean);
Attende la pressione del mouse, scrivendone intanto le coordinate in basso a sinistra.
- Function MyMouseInG (X1: Integer; Y1: Integer; X2: Integer; Y2: Integer): Boolean;
Verifica che il mouse sia nell' area specificata dal rettangolo X1, Y1, X2, Y2, senza però controllare la pressione dei pulsanti: restituisce TRUE se è interno all' area, FALSE se esterno.

GIPHELP

- Procedure Beep;
Emette un suono di 100 Hz per circa 30 ms (millisecondi).
- Procedure Help (Argomento: String; Tipo: TipoEsciHelp);
Questa è la procedura che gestisce l' intero help, compreso i collegamenti nello stesso, la pressione dei tasti e dei pulsanti del mouse. Si ricordano i tasti che lo richiamano: F1, ALT-F1, CTRL-F1 e SHIFT-F1 }

GIPIWIN

- Function AllocaFinestra (X: Integer; Y: Integer; NumCol: Byte; NumRow: Byte; Var P: Pointer): Boolean;
Alloca in memoria una finestra di dimensioni iniziali X,Y, di larghezza dipendente da NUMCOL e di altezza dipendente da NUMROW, memorizzando l' inizio dei dati nel puntatore P.
- Procedure RilasciaMemoria (X: Integer; Y: Integer; Var P: Pointer);
Rilascia la memoria precedentemente salvata.
- Procedure IconWinOpDisco (X: Integer; Y: Integer);
Apre la finestra corrispondente all' icona con il dischetto (operazioni sul disco) o al menu FILES, alle coordinate X,Y.
- Procedure IconWinOpBlocchi (X: Integer; Y: Integer);
Apre la finestra corrispondente all' icona delle operazioni con i blocchi o al menu BLOCCHI, alle coordinate X,Y.
- Procedure IconWinConversione (X: Integer; Y: Integer);
Apre la finestra corrispondente all' icona di conversione del disegno o al menu CONVERSIONE, alle coordinate X,Y.
- Procedure IconWinSceltaFont (X: Integer; Y: Integer);
Apre la finestra corrispondente all' icona di visualizzazione del testo o all' opzione TOOLS/TESTO, alle coordinate X,Y.

- Procedure IconWinSceltaForma (X: Integer; Y: Integer);
Apre la finestra corrispondente all' icona di forme di disegno o all' opzione TOOLS/FORME, alle coordinate X,Y.
- Procedure IconWinSceltaRetino (X: Integer; Y: Integer);
Apre la finestra corrispondente alla scelta del retino o all' opzione TOOLS/RETINO, alle coordinate X,Y.
- Procedure IconWinSceltaLinea (X: Integer; Y: Integer);
Apre la finestra corrispondente alla scelta della linea o all' opzione TOOLS/LINEA, alle coordinate X,Y.
- Procedure IconWinMovim (X: Integer; Y: Integer);
Apre la finestra corrispondente alla scelta dello spostamento della selezione di un oggetto, alle coordinate X,Y.

GIPIIMAGE

- Procedure InserisciFigura (nFigura: Word; nLineSt: Word;
nLinePt: Word; nLineTh: Word; nColLin: Word;
nRetino: Word; nColRet: Word; nWord1: Integer;
nWord2: Integer; nWord3: Integer; nWord4: Integer;
nWord5: Integer; nWord6: Integer);
Inserisce la figura caratterizzata dal codice nFIGURA e tutti i codici e i valori dei vari campi del record nell' elemento della lista in memoria.
- Procedure InserisciStringa (nSt: String);
Inserisce la stringa nST nella lista in memoria, aggiornando tutti i campi relativi.
- Procedure RefreshScreen;
Ridisegna lo schermo per eliminare qualche possibile interferenza o disturbo nel programma.

GIPINIT

- Procedure AddBackSlash (Var St: String);
Aggiunge il carattere backslash '\' alla stringa se è necessario per indicare una directory o un file.
- Procedure RemoveBackSlash (Var St: String);
Rimuove il carattere backslash '\' finale dalla stringa se questa deve essere passata alla procedura ChDir, MkDir, Rmdir, ecc., per evitare errori in esecuzione (run-time errors).
- Procedure InitVar;
Inizializza tutte le variabili, i puntatori, ecc.
- Procedure InitVGACard;
Inizializza la scheda grafica VGA alla modalità 640x480: se viene riscontrato un errore, il programma ne determina la natura e cerca di rimediare (es. mancano i files *.BGI); se l' errore non è rimediabile informa l' utente e visualizza il messaggio corrispondente: il programma termina.
- Procedure InitMouse;
Inizializza il mouse: se viene riscontrato un errore, il programma termina l' esecuzione e informa l' utente che il

mouse non è disponibile (necessario al programma) oppure che manca il driver in memoria.

- Procedure CreaImmagine;
Inizializza i puntatori al disegno (NIL).
- Procedure DistruggiImmagine;
Elimina il disegno rilasciando la memoria occupata.
- Procedure DefineOriginalSize;
Inizializza le coordinate del disegno relativamente alla finestra attiva.
- Procedure ReleaseMemory;
Rilascia la memoria occupata dall' immagine, dai menu, dalle icone, ecc.
- Procedure Defaults;
Porta ai default il tipo di linea, il limite orizzontale e verticale (range) del movimento del mouse, il colore, il tipo di retino, ecc.
- Procedure ClearKeyBuf;
Vuota il buffer della tastiera se pieno o occupato; non esegue nulla in caso contrario.

GIPLINK GIPMAIN

non contiene procedure.

- Procedure ImageProcessor;
E' la procedura principale del programma, che gestisce le chiamate a tutte le altre secondarie.

GIPMENU

- Procedure DrawMenu (NumMenu: Byte);
Disegna il menu numero NUMMENU.
- Procedure SelectMenu;
Attendo una selezione o un cambio di menu/opzione, uscita, ecc. nel menu attivo.
- Function MouseInMenuBar: Boolean;
Restituisce TRUE se il mouse è stato premuto in una zona in cui è presente un titolo di menu; FALSE in caso contrario.

GIPPROC

- Procedure pBlockMemorizza;
Memorizza il blocco evidenziato.
- Procedure pBlockRichiama;
Richiama il blocco memorizzato (memoria o disco).
- Procedure pBlockSposta;
Sposta il blocco evidenziato nella nuova posizione.
- Procedure pBlockCopia;
Copia il blocco evidenziato nella nuova posizione.
- Procedure pDiscoLeggi;
Legge un file disegno dal disco.
- Procedure pDiscoSalva;
Salva un file disegno sul disco.
- Procedure pDiscoPaletteLeggi;
Legge un file colore dal disco.
- Procedure pDiscoPaletteSalva;

- Salva un file colore sul disco.
- Procedure pDiscoNuovo;
Comincia un nuovo disegno.
- Procedure pDiscoEsci;
Esce dal programma.
- Procedure pConvTurboC;
Converte il disegno in Turbo C.
- Procedure pConvTurboPascal;
Converte il disegno in Turbo Pascal.
- Procedure pConvTurboAsm;
Converte il disegno in Turbo Assembler.
- Procedure pUserDefinedFill;
Personalizza il tipo di retino.
- Procedure pUserDefinedLn;
Presonalizza il tipo di linea.
- Procedure pSceltaOggetti;
Seleziona un oggetto dal video.
- Procedure pHelpOnLine;
Apre la finestra di aiuto generale.
- Procedure pDiscoStampa;
Stampa il disegno in memoria.
- Procedure pNewPalette;
Modifica i colori, scomponendoli in Red, Green e Blue.
- Procedure pHelpPrecedente;
Apre l' ultima schermata di aiuto.
- Procedure pHelpIndice;
Apre la schermata di aiuto e visualizza l' indice degli argomenti.
- Procedure pHelpGIP;
Apre la schermata di aiuto e visualizza informazioni su G.I.P.
- Procedure pClock;
Visualizza l' orologio sul video (sia analogico sia digitale).
- Procedure pSfondoImmagine;
Aggiorna/modifica lo sfondo dell' immagine.
- Procedure pMovPrimo;
Sposta l' evidenziatore al primo oggetto della lista in memoria (il primo disegnato).
- Procedure pMovPrec;
Sposta l' evidenziatore all' oggetto precedente quello corrente.
- Procedure pMovSucc;
Sposta l' evidenziatore all' oggetto successivo quello corrente.
- Procedure pMovUltimo;
Sposta l' evidenziatore all' ultimo oggetto della lista in memoria (non è necessariamente l' ultimo disegnato).

GIPSHAPE

- Procedure pMovDelete;
Cancella dalla memoria l' oggetto evidenziato.
- Procedure MyGetArcCoords (Var ArcC: ArcCoordsType);
Restituisce le coordinate dell' ultimo ellisse disegnato dalla procedura MYELLIPSE.
- Procedure MyEllipse (X: Integer; AngInizio: Integer; AngFine: Integer;
Rx: Integer; Ry: Integer; Passo: Byte);
Disegna un ellisse: è simile alla procedura standard ELLIPSE a cui è stato però aggiunta la possibilità di lavorare in XOrPut.
- Procedure pCerchio (Vuoto: Boolean);
Disegna un cerchio vuoto (VUOTO=True) o pieno (VUOTO=False).
- Procedure pRettangolo (Vuoto: Boolean);
Disegna un rettangolo vuoto (VUOTO=True) o pieno (VUOTO=False).
- Procedure pPoligono (Vuoto: Boolean);
Disegna un poligono vuoto (VUOTO=True) o pieno (VUOTO=False).
- Procedure pSettore (Vuoto: Boolean);
Disegna un settore vuoto (VUOTO=True) o pieno (VUOTO=False).
- Procedure pArco (Corda: Boolean);
Disegna un arco con corda (CORDA=True) o senza (CORDA=False).
- Procedure pRiempimento;
Riempie un' area grafica delimitata.
- Procedure pLinea;
Disegna una linea.
- Procedure pRettangolo3D (Vuoto: Boolean);
Disegna un cubo vuoto (VUOTO=True) o pieno (VUOTO=False).
- Procedure pDisegnoLibero;
Disegna un punto.
- Procedure pTesto;
Disegna una riga di testo sul video.
- Procedure pUserCharSize;
Determina una dimensione personalizzata del font.
- Procedure EseguiStato (Stato: TipoStato);
Lega lo stato del programma alla procedura relativa (esempio: stato 'Disegno di un cerchio'; procedura relativa: pCerchio).

GIPSTATO

GIPVARS

GRAPH

KEYBOARD

MOUSE

SYSTEM

non ci sono procedure;
unit standard;
vedi T.I.P.;
vedi T.I.P.;
vedi T.I.P..

SPRITE v5.0

CRT	vedi T.I.P.;
DOS	vedi T.I.P.;
GRAPH	vedi G.I.P.;
MOUSE	vedi T.I.P.;
PULSANTI	non ci sono procedure;
SPRDRIV	vedi G.I.P., unit GIPDRIV;
SPRFONT1	vedi G.I.P., unit GIPFONT1;
SPRFONT2	vedi G.I.P., unit GIPFONT2;
SPRFONT3	vedi G.I.P., unit GIPFONT3;
SPRITE	è il programma principale;
SYSTEM	vedi T.I.P.;
UTILSPR1	- Procedure PutPixel16 (X: Word; Y: Word; C: Word; UpMode: Byte); Disegna un pixel alle coordinate X e W dello schermo (deve essere una VGA a 16 colori). Il colore è definito da C, mentre UpMode è il modo con cui deve essere visualizzato (pRep, pXor, pAnd, pOr). - Function GetPixel16 (X: Word; Y: Word): Byte; Restituisce il colore del pixel che ha come coordinate X e Y. Il video deve essere in modalità VGA a 16 colori. - Procedure WaitToWrite; Questa procedura viene utilizzata per evitare l' effetto di sfarfallio quando si scrive e cancella ripetutamente una stessa zona. - Procedure DefineEndCoords; Definisce il massimo numero di quadri per l' immagine, settando opportunamente le variabili di limiti. - Procedure PremiPulsante (Quale: Byte; Punt: RecP; Luogo: TipoLuogo); Serve per premere un qualsiasi pulsante sullo schermo; i parametri sono: Quale: numero dell'immagine Punt: puntatore all'immagine Luogo: luogo in cui si trova il tasto. - Procedure VuotaSchermo; Questa procedura serve per vuotare lo schermo in memoria, ossia per azzerare al colore NERO tutti i pixel dell'immagine. - Procedure DisegnaPixel (Modo: TipoModo; Reale: TipoReal); Serve per disegnare l'immagine che si trova in memoria, stabilendo il Modo (Clear = cancella prima di disegnare, NotClear = non cancellare) e se si vuole disegnare anche l'immagine in dimensioni reali con Reale (SiReale e

NoReale).

- Procedure DisegnaGriglia;

Disegna la griglia sullo schermo secondo il valore della variabile Griglia:

PerLinee: linee che formano un reticolo,

PerPunti: punti,

NoGriglia: nessuna griglia.

- Procedure AssegnaPulsanteGriglia (Metti: TipoMetti;

gGriglia: TipoGriglia; Var Punt: RecP);

Per limitare l' utilizzo della memoria, le icone per il tipo di griglia vengono definite solo in ambienti locali. I parametri sono:

Metti: vale Disegna se deve essere visualizzata la finestra e se si deve attendere che l' utente scelga un' icona (altrimenti vale

NonDisegnare),

gGriglia: individua il tipo di griglia (gLinee, gPunti, gNessuna),

Punt: puntatore all' icona.

- Procedure AssegnaPulsanteShape (Metti: TipoMetti;

sShape: TipoShape; Var Punt: RecP);

Per limitare l' utilizzo della memoria, le icone per il tipo di forma vengono definite solo in ambienti locali. I parametri sono:

Metti: vale Disegna se deve essere visualizzata la finestra e se si deve attendere che l' utente scelga un' icona (altrimenti vale

NonDisegnare)

sShape: individua il tipo di forma corrente, scelte fra sCerchio,

sCerchioPieno, sLinee,

sLineePiene, sRettangolo, sRettangoloPieno,

sDisegnoLiberoestra,

Punt: puntatore all' icona.

- Procedure AssegnaPulsanteLine (Metti: TipoMetti; lLines:

TipoLine; Var Punt: RecP);

Per limitare l' utilizzo della memoria, le icone per il tipo di linea vengono definite solo in ambienti locali. I parametri sono:

Metti: vale Disegna se deve essere visualizzata la finestra se si deve attendere che l' utente scelga un' icona

- (altrimenti vale NonDisegnare)
- ILines: individua il tipo di linea corrente, scelte fra
 lSolid1, IDotted1, lCenter1, lDashed1,
 lSolid3, lDotted3, lCenter3, lDashed3
- Punt: puntatore all' icona.
- Procedure AssegnaPulsantePattern (Metti: TipoMetti;
 pPattern: TipoPattern; Var Punt: RecP);
 Per limitare l' utilizzo della memoria, le icone per il tipo di
 retino vengono definite solo in ambienti locali. I parametri
 sono:
 Metti: vale Disegna se deve essere
 visualizzata la finestra e se si
 deve attendere che l' utente scelga
 un' icona (altrimenti vale
 NonDisegnare)
 pPattern: individua il tipo di retino corrente,
 scelte fra pEmpty, pSolid, pLine,
 pLtSlash, pSlash, pBkSlash,
 pLtBkSlash, pHatch, pXHatch,
 pInterleave, pWideDot, pCloseDot
 Punt: puntatore all' icona.
 - Procedure DefinisciPulsanti;
 Definisce l' ARRAY principale di tutti i tasti presenti nel
 programma e le loro posizioni.
 - Procedure RettangoloReal (ValX: Integer; ValY: Integer);
 Serve per disegnare l'area che rappresenta lo schermo
 grande in quello a dimensioni reali, in modo da sapere
 sempre in che posizione ci si trova con lo schermo, anche
 se questo viene aiutato dal fatto che ai bordi del lo schermo
 si trovano le coordinate dei vertici. I parametri indicano l'
 angolo di inizio (in alto a sinistra) del rettangolo appena
 descritto.
 - Procedure WriteBarCoord (Direzione: TipoDirezione; ValMin:
 Integer; ValMax: Integer);
 Questa procedura serve per disegnare le coordinate dello
 schermo grande rispetto allo schermo in dimensioni reali,
 specificando quali coordinate scrivere (Orizz per quelle
 orizzontali e Vert per quelle verticali) e i valori di minimo
 e massimo da scrivere.
 - Procedure WritePosCoord (ValX: Integer; ValY: Integer);
 Serve per scrivere le coordinate della posizione del mouse
 su entrambi gli schermi di disegno.
 - Procedure AggiornaScala;
 Scrive la scala corrente fra il disegno nella finestra più
 grande e quello nella finestra più piccola. Quella di defalut
 è 10:1.
 - Procedure AggiornaNomefile;

- Scrive in alto a destra il nome del file corrente.
- Procedure MettiPuntini (Var Numero: String);
Serve per trasformare una stringa '12345' in '12.345', ossia mette i puntini ogni 3 cifre della stringa.
 - Procedure MergeSort (Var Dir: PTRTipoDir; Min: Integer; Max: Integer);
Serve per ordinare la directory del disco.
 - Procedure Informazione (Var NumRighe: Byte; Var Stringhe: TipoStringhe);
Visualizza alcune informazioni riguardanti il disco, il computer, il file, ecc., premendo l' icona INFORMAZIONI.
 - Procedure NormalWindow (Var NumRighe: Byte; Var Stringhe: TipoStringhe; Var Stringa: String);
Aggiusta il contenuto della finestra attiva, separando i colori dal testo di ogni riga.
 - Procedure CheckTime (Var Stringa: String; Var PulsSfondo: Byte; Var Hour: Word; Var Minute: Word; Var Second: Word; Var Sec100: Word; Var OldHour: Word; Var OldMinute: Word; Var OldSecond: Word; Var OldSec100: Word);
Se occorre, visualizza l' ora, i minuti, i secondi e i centesimi di secondi nelle coordinate grafiche opportune.
 - Procedure Typer (Stringa: String; Riga: Byte; Colonna: Byte);
Visualizza sul video (in modalità testo) la stringa specificata come parametro in una maniera diversa dal normale (come una macchina da scrivere).
 - Procedure InsertPath (Var PathBGI: String; Errore: Integer);
Attende che l' utente inserisca il path dove si trovano i files *.BGI e *.CHR, continuando fino a che l' utente non vuole uscire o fino a che il path non sia corretto.
 - Procedure Abort (Errore: Integer);
Esce dal programma perchè non è disponibile la scheda grafica VGA.
 - Procedure ErroreMouse;
Esce dal programma perchè non è stato trovato il mouse oppure non è stato caricato il suo driver in memoria. Il programma utilizza solo questo dispositivo.
 - Procedure EndSpriteManager;
Esce dal programma liberando la memoria e chiudendo la modalità grafica, tornando quindi alla modalità testo 25x80. Cambia la directory corrente (ritorna quindi a quella in cui si era al momento dell' esecuzione del programma) e visualizza sul video alcune informazioni per l' utente.
 - Procedure SpegniCursore;

UTILSPR2

- Nasconde la visualizzazione del cursore alla vista dell'utente.
- Procedure CursorePiccolo;
Visualizza il cursore come una piccola linea in fondo ('_').
 - Procedure CursoreGrande;
Visualizza il cursore come un blocco pieno.
 - Procedure StampaDir (Var Inizio: Integer; Var InizioDialX: Integer;
Var InizioDialY: Integer; Var FineDialX: Integer;
Var FineDialY: Integer; Var NumFiles: Integer; Var Dir: PTRTipoDir; Var Select: Integer;
Var OldSelect: Integer; Var Risult: Boolean);
Visualizza sul video il contenuto della directory corrente. I parametri hanno gli stessi nomi delle variabili globali.
 - Procedure Aggiorna (Var Select: Integer; Var OldSelect: Integer;
Var Inizio: Integer; Var NumFiles: Integer;
Var InizioDialX: Integer; Var InizioDialY: Integer;
Var FineDialX: Integer; Var FineDialY: Integer;
Var Dir: PTRTipoDir; Var Risult: Boolean);
Aggiorna l' evidenziatore all' interno della lista della directory. I parametri hanno gli stessi nomi delle variabili globali.
 - Procedure LeggiPath (Var Path: StringPath; Var Risult: Boolean;
Var Tasti: TipoTasti; Var InizioDialX: Integer;
Var InizioDialY: Integer; Var FineDialX: Integer;
Var FineDialY: Integer; Var Dir: PTRTipoDir;
Var NumFiles: Integer; Var Inizio: Integer;
Var Select: Integer; Var OldSelect: Integer);
Legge il contenuto del disco e setta opportunamente le variabili interessate (vettore nello heap, numero di files, ecc.). I parametri hanno gli stessi nomi delle variabili globali.
 - Procedure Windows (X1: Integer; Y1: Integer; X2: Integer;
Y2: Integer; Stringa: String; PulsAlto: Byte;
PulsBasso: Byte; PulsSfondo: Byte; Tasti: TipoTasti;
Ext: String003);
E' la procedura più importante del programma, in quanto tutte le finestre di dialogo sono gestite e disegnate da questa. I parametri sono:

X1:	inizio X
Y1:	inizio Y
X2:	fine X
Y2:	fine Y
Stringa:	testo della finestra (usare ^ per separare le righe)
PulsAlto:	colore delle linne in alto (come i tasti)

- PulsBasso: colore delle linee in basso
- PulsSfondo: colore di sfondo
- Tasti: tipo di tasti da integrare nella finestra.
- Procedure SalvaFile (Var FileName: StringFile);
Salva il file (con conferma) di nome FileName.
- Procedure LeggiFile (Var FileName: StringFile);
Legge il file (con conferma) di nome FileName.
- Procedure CreaFileTurbo (Var NomeFile: StringFile);
Crea un file in Turbo Pascal che visualizzi lo sprite che si trova in memoria.
- Function Expand3 (Valore: Integer): String;
Espande la cifra a 3 cifre, aggiungendogli se necessario degli zero in testa. Restituisce una stringa (generalmente di 3 cifre).
- Procedure SpriteManager;
E' il cuore del programma, quello che lo fa partire.
- Procedure ReleaseButtons;
Attende che l' utente rilasci tutti i pulsanti del mouse (nel frattempo non esegue nessuna operazione).
- Procedure IconWindows (X1: Integer; Y1: Integer; X2: Integer; Y2: Integer; Stringa: String; PulsAlto: Byte; PulsBasso: Byte; PulsSfondo: Byte; Icone: TipoIcone);

E' molto simile alla procedura WINDOWS, ma questa visualizza delle icone (per la scelta del tipo di griglia, retino, linea o forma da disegnare). I parametri sono quasi gli stessi, tranne l' ultimo:

- X1: inizio X
- Y1: inizio Y
- X2: fine X
- Y2: fine Y
- Stringa: testo della finestra (usare ^ per separare le righe)
- PulsAlto: colore delle linee in alto (come i tasti)
- PulsBasso: colore delle linee in basso
- PulsSfondo: colore di sfondo
- Icone: tipo di icona da integrare nella finestra, scelte fra IconeShape, IconeLines, IconePattern, IconeGriglia.
- Procedure GetSprite;
Legge il contenuto dell' immagine piccola in basso a destra e lo copia in quella grande (settando opportunamente tutti i valori dei pixels in memoria).
- Procedure GetAllSprite;
Legge il contenuto dell' intera immagine piccola in basso a

- destra e lo copia in quella grande (settando opportunamente tutti i valori dei pixels in { memoria}).
- Procedure DisegnaFigura (Luogo: TipoDisegna);
Disegna la figura sullo schermo (cerchio, punto, linea, ellisse, figure piene, ecc.). L' unico parametro è Luogo, che indica il luogo in cui deve essere disegnata (nella finestra grande o in quella piccola) e può assumere i valori: FinGrande, FinPiccola.

Mancanza Di Tempo

Per motivi dovuti alla mancanza di tempo, non sono state implementate nei programmi alcune caratteristiche previste al momento della progettazione degli obiettivi da raggiungere. In particolare:

1) In T.I.P. rimane ancora da completare:

- tutti i controlli sulla lettura dei file strutturati, sul tipo di file valido, sulla protezione del disco, sul salvataggio riuscito o meno, e qualsiasi errore di I/O su disco;
- la conversione in Turbo Assembler;
- finire l' help, file 'TIP.HLP'.

2) In G.I.P. rimane ancora da completare:

- Scelta oggetti;
- Stampa immagine;
- Operazioni con i blocchi;
- Conversione Turbo Assembler;
- Ottimizzazione dell' Help;
- Miglioramenti di InputFile;
- Finire l' help, file 'GIP.HLP'.

3) Il programma SPRITE rappresenta un caso a parte, in quanto non dovrebbe neanche figurare nella tesina, per cui ogni miglioramento o modifica è senz' altro apprezzata ma non precedentemente progettata. A titolo informativo elenco le parti che potrebbero essere migliorate in futuro:

- miglioramento della finestra di lettura/scrittura di un file;
- controlli minuziosi (come in T.I.P.) sul file da processare e sul disco su cui si lavora;
- miglioramento della procedure di pressione di un pulsante;
- velocizzazione delle procedure per la scrittura in grafica, direttamente in assembler;
- help on-line;
- conversione in altri linguaggi;
- operazioni con i blocchi;
- ...

Esempi Di Conversione

T.I.P.

Verrà preso come modello il seguente disegno:

Nell' importazione dei file di testo (ASCII) in Microsoft WinWord v2.0, non è stato possibile visualizzare i caratteri ASCII esatti: le cornici, per esempio, non ci sono. Al loro posto però vi è un carattere corrispondente allo stile del testo normale dell' editor.

Turbo Pascal

File In Turbo Pascal Numero 1 (Con GoToXY E Write)

```
{
{      Questo file è stato creato da T.I.P., scritto da Fochi Michele      }
{
{
{      Le procedure "ClrScr", "Write" e "GoToXY", e le costanti
{      "TextAttr" e "LightGray" sono nella unit CRT.TPU.
{
{
{      Per eseguire il programma occorre quindi la seguente linea
{      di codice (nelle prime righe del programma):
{
{      Uses Crt;
{
}
```

Procedure P1;

Begin { P1 }

```
{ Cancello lo schermo }
TextAttr := LightGray;
ClrScr;
```

```
{ Ora viene riempito lo schermo }
```

```
GoToXY(01,01);  TextAttr := 014;
Write('+-----+');
GoToXY(01,02);  TextAttr := 014;
Write(' ');
GoToXY(02,02);  TextAttr := 015;
Write(' ');
```

```

GoToXY(80,02);  TextAttr := 014;
Write("");
GoToXY(01,03);  TextAttr := 014;
Write("");
GoToXY(02,03);  TextAttr := 015;
Write('');
GoToXY(80,03);  TextAttr := 014;
Write("");
GoToXY(01,04);  TextAttr := 014;
Write("");
GoToXY(02,04);  TextAttr := 015;
Write(' ..... ');
GoToXY(80,04);  TextAttr := 014;
Write("");
GoToXY(01,05);  TextAttr := 014;
Write("");
GoToXY(02,05);  TextAttr := 015;
Write(' ..... ');
GoToXY(80,05);  TextAttr := 014;
Write("");
GoToXY(01,06);  TextAttr := 014;
Write("");
GoToXY(02,06);  TextAttr := 015;
Write(' ...');
GoToXY(13,06);  TextAttr := 013;
Write('+-----+');
GoToXY(29,06);  TextAttr := 015;
Write('.....');
GoToXY(33,06);  TextAttr := 011;
Write('+-----+');
GoToXY(41,06);  TextAttr := 015;
Write('.....');
GoToXY(48,06);  TextAttr := 012;
Write('+-----+');
GoToXY(63,06);  TextAttr := 015;
Write('..... ');
GoToXY(80,06);  TextAttr := 014;
Write("");
GoToXY(01,07);  TextAttr := 014;
Write("");
GoToXY(02,07);  TextAttr := 015;
Write(' ...');
GoToXY(13,07);  TextAttr := 013;
Write("");
GoToXY(14,07);  TextAttr := 015;
Write('_____');
GoToXY(28,07);  TextAttr := 013;

```

```

Write("");
GoToXY(29,07);  TextAttr := 015;
Write('....');
GoToXY(33,07);  TextAttr := 011;
Write("");
GoToXY(34,07);  TextAttr := 015;
Write('_____');
GoToXY(40,07);  TextAttr := 011;
Write("");
GoToXY(41,07);  TextAttr := 015;
Write('.....');
GoToXY(48,07);  TextAttr := 012;
Write("");
GoToXY(49,07);  TextAttr := 015;
Write('_____');
GoToXY(62,07);  TextAttr := 012;
Write('++');
GoToXY(64,07);  TextAttr := 015;
Write('.....');
GoToXY(80,07);  TextAttr := 014;
Write("");
GoToXY(01,08);  TextAttr := 014;
Write("");
GoToXY(02,08);  TextAttr := 015;
Write('...');
GoToXY(13,08);  TextAttr := 013;
Write("");
GoToXY(14,08);  TextAttr := 015;
Write('__');
GoToXY(16,08);  TextAttr := 013;
Write('+-+');
GoToXY(20,08);  TextAttr := 015;
Write('__');
GoToXY(22,08);  TextAttr := 013;
Write('+-+');
GoToXY(26,08);  TextAttr := 015;
Write('__');
GoToXY(28,08);  TextAttr := 013;
Write("");
GoToXY(29,08);  TextAttr := 015;
Write('....');
GoToXY(33,08);  TextAttr := 011;
Write('+-+');
GoToXY(36,08);  TextAttr := 015;
Write('__');
GoToXY(38,08);  TextAttr := 011;
Write('+-+');

```



```

GoToXY(41,08);  TextAttr := 015;
Write('.....');
GoToXY(48,08);  TextAttr := 012;
Write('+--+');
GoToXY(51,08);  TextAttr := 015;
Write('___+-----+__');
GoToXY(63,08);  TextAttr := 012;
Write('++');
GoToXY(65,08);  TextAttr := 015;
Write('.....');
GoToXY(80,08);  TextAttr := 014;
Write('');
GoToXY(01,09);  TextAttr := 014;
Write('');
GoToXY(02,09);  TextAttr := 015;
Write('.....');
GoToXY(13,09);  TextAttr := 013;
Write('+--+');
GoToXY(17,09);  TextAttr := 015;
Write('..');
GoToXY(19,09);  TextAttr := 013;
Write('');
GoToXY(20,09);  TextAttr := 015;
Write('___');
GoToXY(22,09);  TextAttr := 013;
Write('');
GoToXY(23,09);  TextAttr := 015;
Write('..');
GoToXY(25,09);  TextAttr := 013;
Write('+--+');
GoToXY(29,09);  TextAttr := 015;
Write('.....');
GoToXY(35,09);  TextAttr := 011;
Write('');
GoToXY(36,09);  TextAttr := 015;
Write('___');
GoToXY(38,09);  TextAttr := 011;
Write('');
GoToXY(39,09);  TextAttr := 015;
Write('.....');
GoToXY(50,09);  TextAttr := 012;
Write('');
GoToXY(51,09);  TextAttr := 015;
Write('___|.....|___');
GoToXY(64,09);  TextAttr := 012;
Write('');
GoToXY(65,09);  TextAttr := 015;

```

```

Write('·····');
GoToXY(80,09); TextAttr := 014;
Write("");
GoToXY(01,10); TextAttr := 014;
Write("");
GoToXY(02,10); TextAttr := 015;
Write('·······');
GoToXY(19,10); TextAttr := 013;
Write("");
GoToXY(20,10); TextAttr := 015;
Write('__');
GoToXY(22,10); TextAttr := 013;
Write("");
GoToXY(23,10); TextAttr := 015;
Write('·········');
GoToXY(35,10); TextAttr := 011;
Write("");
GoToXY(36,10); TextAttr := 015;
Write('__');
GoToXY(38,10); TextAttr := 011;
Write("");
GoToXY(39,10); TextAttr := 015;
Write('·········');
GoToXY(50,10); TextAttr := 012;
Write("");
GoToXY(51,10); TextAttr := 015;
Write('__+-----+__');
GoToXY(63,10); TextAttr := 012;
Write('++');
GoToXY(65,10); TextAttr := 015;
Write('·····');
GoToXY(80,10); TextAttr := 014;
Write("");
GoToXY(01,11); TextAttr := 014;
Write("");
GoToXY(02,11); TextAttr := 015;
Write('·······');
GoToXY(19,11); TextAttr := 013;
Write("");
GoToXY(20,11); TextAttr := 015;
Write('__');
GoToXY(22,11); TextAttr := 013;
Write("");
GoToXY(23,11); TextAttr := 015;
Write('·········');
GoToXY(35,11); TextAttr := 011;
Write("");

```

```

GoToXY(36,11);  TextAttr := 015;
Write('___');
GoToXY(38,11);  TextAttr := 011;
Write('');
GoToXY(39,11);  TextAttr := 015;
Write('.....');
GoToXY(50,11);  TextAttr := 012;
Write('');
GoToXY(51,11);  TextAttr := 015;
Write('_____');
GoToXY(62,11);  TextAttr := 012;
Write('++');
GoToXY(64,11);  TextAttr := 015;
Write('.....');
GoToXY(80,11);  TextAttr := 014;
Write('');
GoToXY(01,12);  TextAttr := 014;
Write('');
GoToXY(02,12);  TextAttr := 015;
Write('.....');
GoToXY(19,12);  TextAttr := 013;
Write('');
GoToXY(20,12);  TextAttr := 015;
Write('___');
GoToXY(22,12);  TextAttr := 013;
Write('');
GoToXY(23,12);  TextAttr := 015;
Write('.....');
GoToXY(35,12);  TextAttr := 011;
Write('');
GoToXY(36,12);  TextAttr := 015;
Write('___');
GoToXY(38,12);  TextAttr := 011;
Write('');
GoToXY(39,12);  TextAttr := 015;
Write('.....');
GoToXY(50,12);  TextAttr := 012;
Write('');
GoToXY(51,12);  TextAttr := 015;
Write('___');
GoToXY(53,12);  TextAttr := 012;
Write('+-----+');
GoToXY(63,12);  TextAttr := 015;
Write('.....');
GoToXY(80,12);  TextAttr := 014;
Write('');
GoToXY(01,13);  TextAttr := 014;

```

```

Write("");
GoToXY(02,13);  TextAttr := 015;
Write('      .....');
GoToXY(17,13);  TextAttr := 013;
Write('+ - +');
GoToXY(20,13);  TextAttr := 015;
Write('__');
GoToXY(22,13);  TextAttr := 013;
Write('+ - +');
GoToXY(25,13);  TextAttr := 015;
Write('.....');
GoToXY(33,13);  TextAttr := 011;
Write('+ - +');
GoToXY(36,13);  TextAttr := 015;
Write('__');
GoToXY(38,13);  TextAttr := 011;
Write('+ - +');
GoToXY(41,13);  TextAttr := 015;
Write('.....');
GoToXY(48,13);  TextAttr := 012;
Write('+ - +');
GoToXY(51,13);  TextAttr := 015;
Write('__');
GoToXY(53,13);  TextAttr := 012;
Write('+ - +');
GoToXY(56,13);  TextAttr := 015;
Write('.....      ');
GoToXY(80,13);  TextAttr := 014;
Write("");
GoToXY(01,14);  TextAttr := 014;
Write("");
GoToXY(02,14);  TextAttr := 015;
Write('      .....');
GoToXY(17,14);  TextAttr := 013;
Write("");
GoToXY(18,14);  TextAttr := 015;
Write('_____');
GoToXY(24,14);  TextAttr := 013;
Write("");
GoToXY(25,14);  TextAttr := 015;
Write('·TEXT·');
GoToXY(33,14);  TextAttr := 011;
Write("");
GoToXY(34,14);  TextAttr := 015;
Write('_____');
GoToXY(40,14);  TextAttr := 011;
Write("");

```

117

```

Write('');
GoToXY(01,19);  TextAttr := 014;
Write('');
GoToXY(02,19);  TextAttr := 015;
Write('');
GoToXY(80,19);  TextAttr := 014;
Write('');
GoToXY(01,20);  TextAttr := 014;
Write('');
GoToXY(02,20);  TextAttr := 015;
Write('    FOCHI MICHELE & AMATULLI DAVIDE');
GoToXY(80,20);  TextAttr := 014;
Write('');
GoToXY(01,21);  TextAttr := 014;
Write('');
GoToXY(02,21);  TextAttr := 015;
Write('    VERSIONE 1.0');
GoToXY(80,21);  TextAttr := 014;
Write('');
GoToXY(01,22);  TextAttr := 014;
Write('');
GoToXY(02,22);  TextAttr := 015;
Write('');
GoToXY(80,22);  TextAttr := 014;
Write('');
GoToXY(01,23);  TextAttr := 014;
Write('');
GoToXY(02,23);  TextAttr := 015;
Write('');
GoToXY(80,23);  TextAttr := 014;
Write('');
GoToXY(01,24);  TextAttr := 014;
Write('+-----+');

End; { P1 }

```

File In Turbo Pascal Numero 2 (Senza GoToXY)

```

{
{      Questo file è stato creato da T.I.P., scritto da Fochi Michele      }
{
{
{      Le procedure "ClrScr", "Write" e "GoToXY", e le costanti
{      "TextAttr" e "LightGray" sono nella unit CRT.TPU.
{
{      Per eseguire il programma occorre quindi la seguente linea
{      di codice (nelle prime righe del programma):
{

```

```

{
{      Uses Crt;
}
}

```

Procedure P2;

Begin { P2 }

{ Cancellò lo schermo }

TextAttr := LightGray;

ClrScr;

{ Ora viene riempito lo schermo }

TextAttr := 014;

Write('+-----+');

TextAttr := 014;

Write(' ');

TextAttr := 015;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 015;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 015;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 015;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 014;

Write(' ');

TextAttr := 015;

Write(' ...');

TextAttr := 013;

Write('+-----+');

TextAttr := 015;

Write('...');

```

TextAttr := 011;
Write('+-----+');
TextAttr := 015;
Write('·····');
TextAttr := 012;
Write('+-----+');
TextAttr := 015;
Write('·····');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write('···');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('_____');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('···');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('_____');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('·····');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('_____');
TextAttr := 012;
Write('++');
TextAttr := 015;
Write('·····');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write('···');
TextAttr := 013;
Write("");
TextAttr := 015;

```



```

Write('___');
TextAttr := 013;
Write('+--+');
TextAttr := 015;
Write('___');
TextAttr := 013;
Write('+--+');
TextAttr := 015;
Write('___');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('...');
TextAttr := 011;
Write('+ -+');
TextAttr := 015;
Write('___');
TextAttr := 011;
Write('+ -+');
TextAttr := 015;
Write('.....');
TextAttr := 012;
Write('+ -+');
TextAttr := 015;
Write('___+-----+___');
TextAttr := 012;
Write('++');
TextAttr := 015;
Write('... ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' ...');
TextAttr := 013;
Write('+--+');
TextAttr := 015;
Write('..');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('___');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('..');

```

```

TextAttr := 013;
Write('+--+');
TextAttr := 015;
Write('·····');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('__');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('········');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('___!·····!___');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('····');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write('········');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('__');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('········');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('__');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('········');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('__+-----+__');
TextAttr := 012;

```

```

Write('++');
TextAttr := 015;
Write('.....');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' .....');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('__');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('.....');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('__');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('.....');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('_____');
TextAttr := 012;
Write('++');
TextAttr := 015;
Write('.....');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' .....');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('__');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('.....');

```

```

TextAttr := 011;
Write("");
TextAttr := 015;
Write('___');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('.....');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('___');
TextAttr := 012;
Write('+-----+');
TextAttr := 015;
Write('..... ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' .....');
TextAttr := 013;
Write('+--+');
TextAttr := 015;
Write('___');
TextAttr := 013;
Write('+--+');
TextAttr := 015;
Write('.....');
TextAttr := 011;
Write('+--+');
TextAttr := 015;
Write('___');
TextAttr := 011;
Write('+--+');
TextAttr := 015;
Write('.....');
TextAttr := 012;
Write('+--+');
TextAttr := 015;
Write('___');
TextAttr := 012;
Write('+--+');
TextAttr := 015;
Write('..... ');
TextAttr := 014;

```

```

Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' .....');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('_____');
TextAttr := 013;
Write("");
TextAttr := 015;
Write('·TEXT·');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('_____');
TextAttr := 011;
Write("");
TextAttr := 015;
Write('·IMAGE·');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('_____');
TextAttr := 012;
Write("");
TextAttr := 015;
Write('·PROCESSOR·');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' .....');
TextAttr := 013;
Write('+-----+');
TextAttr := 015;
Write('·····');
TextAttr := 011;
Write('+-----+');
TextAttr := 015;
Write('·····');
TextAttr := 012;
Write('+-----+');
TextAttr := 015;
Write('·····');
TextAttr := 012;
Write('+-----+');
TextAttr := 015;
Write('·····');

```

```

TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' ..... ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' ..... ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' FOCHI MICHELE & AMATULLI DAVIDE ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' VERSIONE 1.0 ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;
Write(' ');
TextAttr := 014;
Write("");
TextAttr := 014;
Write("");
TextAttr := 015;

```

```

Write('
TextAttr := 014;
Write(' ');
TextAttr := 014;
Write('+------+');

End; { P2 }

```

File In Turbo Pascal Numero 3 (Colori Ottimizzati)

```

{
{ Questo file è stato creato da T.I.P., scritto da Fochi Michele }
{
{
{ Le procedure "ClrScr", "Write" e "GoToXY", e le costanti }
{ "TextAttr" e "LightGray" sono nella unit CRT.TPU. }
{
{ Per eseguire il programma occorre quindi la seguente linea }
{ di codice (nelle prime righe del programma): }
{
{ Uses Crt; }

```

Procedure P3;

Begin { P3 }

```

{ Cancello lo schermo }
TextAttr := LightGray;
ClrScr;

```

{ Ora viene riempito lo schermo }

```

TextAttr := 011;
GoToXY(33,06);
Write('+------+');
GoToXY(33,07);
Write(' ');
GoToXY(40,07);
Write(' ');
GoToXY(33,08);
Write('+-+');
GoToXY(38,08);
Write('+-+');
GoToXY(35,09);
Write(' ');
GoToXY(38,09);

```

```

Write("");
GoToXY(35,10);
Write("");
GoToXY(38,10);
Write("");
GoToXY(35,11);
Write("");
GoToXY(38,11);
Write("");
GoToXY(35,12);
Write("");
GoToXY(38,12);
Write("");
GoToXY(33,13);
Write('+ - ');
GoToXY(38,13);
Write('+ - ');
GoToXY(33,14);
Write("");
GoToXY(40,14);
Write("");
GoToXY(33,15);
Write('+-----+');

```

```

TextAttr := 012;
GoToXY(48,06);
Write('+-----+');
GoToXY(48,07);
Write("");
GoToXY(62,07);
Write('++');
GoToXY(48,08);
Write('+ - ');
GoToXY(63,08);
Write('++');
GoToXY(50,09);
Write("");
GoToXY(64,09);
Write("");
GoToXY(50,10);
Write("");
GoToXY(63,10);
Write('++');
GoToXY(50,11);
Write("");
GoToXY(62,11);
Write('++');

```



```

GoToXY(50,12);
Write("");
GoToXY(53,12);
Write('+-----+');
GoToXY(48,13);
Write('+--+');
GoToXY(53,13);
Write('+--+');
GoToXY(48,14);
Write("");
GoToXY(55,14);
Write("");
GoToXY(48,15);
Write('+-----+');

```

```

TextAttr := 013;
GoToXY(13,06);
Write('+-----+');
GoToXY(13,07);
Write("");
GoToXY(28,07);
Write("");
GoToXY(13,08);
Write("");
GoToXY(16,08);
Write('+--+');
GoToXY(22,08);
Write('+--+');
GoToXY(28,08);
Write("");
GoToXY(13,09);
Write('+--+');
GoToXY(19,09);
Write("");
GoToXY(22,09);
Write("");
GoToXY(25,09);
Write('+--+');
GoToXY(19,10);
Write("");
GoToXY(22,10);
Write("");
GoToXY(19,11);
Write("");
GoToXY(22,11);
Write("");
GoToXY(19,12);

```

```

Write("");
GoToXY(22,12);
Write("");
GoToXY(17,13);
Write('+--+');
GoToXY(22,13);
Write('+--+');
GoToXY(17,14);
Write("");
GoToXY(24,14);
Write("");
GoToXY(17,15);
Write('+-----+');

TextAttr := 014;
GoToXY(01,01);
Write('+-----+');
GoToXY(01,02);
Write("");
GoToXY(80,02);
Write("");
GoToXY(01,03);
Write("");
GoToXY(80,03);
Write("");
GoToXY(01,04);
Write("");
GoToXY(80,04);
Write("");
GoToXY(01,05);
Write("");
GoToXY(80,05);
Write("");
GoToXY(01,06);
Write("");
GoToXY(80,06);
Write("");
GoToXY(01,07);
Write("");
GoToXY(80,07);
Write("");
GoToXY(01,08);
Write("");
GoToXY(80,08);
Write("");
GoToXY(01,09);
Write("");

```

```

GoToXY(80,09);
Write("");
GoToXY(01,10);
Write("");
GoToXY(80,10);
Write("");
GoToXY(01,11);
Write("");
GoToXY(80,11);
Write("");
GoToXY(01,12);
Write("");
GoToXY(80,12);
Write("");
GoToXY(01,13);
Write("");
GoToXY(80,13);
Write("");
GoToXY(01,14);
Write("");
GoToXY(80,14);
Write("");
GoToXY(01,15);
Write("");
GoToXY(80,15);
Write("");
GoToXY(01,16);
Write("");
GoToXY(80,16);
Write("");
GoToXY(01,17);
Write("");
GoToXY(80,17);
Write("");
GoToXY(01,18);
Write("");
GoToXY(80,18);
Write("");
GoToXY(01,19);
Write("");
GoToXY(80,19);
Write("");
GoToXY(01,20);
Write("");
GoToXY(80,20);
Write("");
GoToXY(01,21);

```

```

Write(_);
GoToXY(80,21);
Write(_);
GoToXY(01,22);
Write(_);
GoToXY(80,22);
Write(_);
GoToXY(01,23);
Write(_);
GoToXY(80,23);
Write(_);
GoToXY(01,24);
Write('+-----+');

```

```

TextAttr := 015;
GoToXY(02,02);
Write(' ');
GoToXY(02,03);
Write(' ');
GoToXY(02,04);
Write(' ..... ');
GoToXY(02,05);
Write(' ..... ');
GoToXY(02,06);
Write(' ...');
GoToXY(29,06);
Write('....');
GoToXY(41,06);
Write('.....');
GoToXY(63,06);
Write('..... ');
GoToXY(02,07);
Write(' ...');
GoToXY(14,07);
Write('_____');
GoToXY(29,07);
Write('....');
GoToXY(34,07);
Write('_____');
GoToXY(41,07);
Write('.....');
GoToXY(49,07);
Write('_____');
GoToXY(64,07);
Write('..... ');
GoToXY(02,08);
Write(' ...');

```

```

GoToXY(14,08);
Write('__');
GoToXY(20,08);
Write('__');
GoToXY(26,08);
Write('__');
GoToXY(29,08);
Write('...');
GoToXY(36,08);
Write('__');
GoToXY(41,08);
Write('.....');
GoToXY(51,08);
Write('__+-----+__');
GoToXY(65,08);
Write('... ');
GoToXY(02,09);
Write(' ...');
GoToXY(17,09);
Write('..');
GoToXY(20,09);
Write('__');
GoToXY(23,09);
Write('..');
GoToXY(29,09);
Write('.....');
GoToXY(36,09);
Write('__');
GoToXY(39,09);
Write('.....');
GoToXY(51,09);
Write('__|.....|__');
GoToXY(65,09);
Write('... ');
GoToXY(02,10);
Write(' .....');
GoToXY(20,10);
Write('__');
GoToXY(23,10);
Write('.....');
GoToXY(36,10);
Write('__');
GoToXY(39,10);
Write('.....');
GoToXY(51,10);
Write('__+-----+__');
GoToXY(65,10);

```

```

Write('·····');
GoToXY(02,11);
Write('·····');
GoToXY(20,11);
Write('__');
GoToXY(23,11);
Write('·····');
GoToXY(36,11);
Write('__');
GoToXY(39,11);
Write('·····');
GoToXY(51,11);
Write('_____');
GoToXY(64,11);
Write('·····');
GoToXY(02,12);
Write('·····');
GoToXY(20,12);
Write('__');
GoToXY(23,12);
Write('·····');
GoToXY(36,12);
Write('__');
GoToXY(39,12);
Write('·····');
GoToXY(51,12);
Write('__');
GoToXY(63,12);
Write('·····');
GoToXY(02,13);
Write('·····');
GoToXY(20,13);
Write('__');
GoToXY(25,13);
Write('·····');
GoToXY(36,13);
Write('__');
GoToXY(41,13);
Write('·····');
GoToXY(51,13);
Write('__');
GoToXY(56,13);
Write('·····');
GoToXY(02,14);
Write('·····');
GoToXY(18,14);
Write('_____');

```

File In Turbo Pascal Numero 4 (Con WriteStr)

135

```

{      Per eseguire il programma occorre quindi la seguente linea      }
{      di codice (nelle prime righe del programma):                     }
{                                                                           }
{      Uses Crt,Video;                                                    }

```

Procedure P4;

Begin { P4 }

{ Cancello lo schermo }

TextAttr := LightGray;

ClrScr;

{ Ora viene riempito lo schermo }

```

WriteStr(01,01,'+-----+',014);
WriteStr(01,02,',',014);
WriteStr(02,02,'',015);
WriteStr(80,02,',',014);
WriteStr(01,03,',',014);
WriteStr(02,03,'',015);
WriteStr(80,03,',',014);
WriteStr(01,04,',',014);
WriteStr(02,04,'.....',015);
WriteStr(80,04,',',014);
WriteStr(01,05,',',014);
WriteStr(02,05,'.....',015);
WriteStr(80,05,',',014);
WriteStr(01,06,',',014);
WriteStr(02,06,'... ',015);
WriteStr(13,06,'+-----+',013);
WriteStr(29,06,'.... ',015);
WriteStr(33,06,'+-----+',011);
WriteStr(41,06,'..... ',015);
WriteStr(48,06,'+-----+',012);
WriteStr(63,06,'..... ',015);
WriteStr(80,06,',',014);
WriteStr(01,07,',',014);
WriteStr(02,07,'... ',015);
WriteStr(13,07,',',013);
WriteStr(14,07,'_____',015);
WriteStr(28,07,',',013);
WriteStr(29,07,'.... ',015);
WriteStr(33,07,',',011);
WriteStr(34,07,'_____',015);
WriteStr(40,07,',',011);
WriteStr(41,07,'..... ',015);

```



```

WriteStr(48,07,"",012);
WriteStr(49,07,'_____',015);
WriteStr(62,07,'++',012);
WriteStr(64,07,'.....',015);
WriteStr(80,07,"",014);
WriteStr(01,08,"",014);
WriteStr(02,08,'... ',015);
WriteStr(13,08,"",013);
WriteStr(14,08,'__ ',015);
WriteStr(16,08,'+--+ ',013);
WriteStr(20,08,'__ ',015);
WriteStr(22,08,'+--+ ',013);
WriteStr(26,08,'__ ',015);
WriteStr(28,08,"",013);
WriteStr(29,08,'.....',015);
WriteStr(33,08,'+-+',011);
WriteStr(36,08,'__ ',015);
WriteStr(38,08,'+-+',011);
WriteStr(41,08,'.....',015);
WriteStr(48,08,'+-+',012);
WriteStr(51,08,'__+-----+__',015);
WriteStr(63,08,'++',012);
WriteStr(65,08,'.....',015);
WriteStr(80,08,"",014);
WriteStr(01,09,"",014);
WriteStr(02,09,'... ',015);
WriteStr(13,09,'+--+ ',013);
WriteStr(17,09,'.. ',015);
WriteStr(19,09,"",013);
WriteStr(20,09,'__ ',015);
WriteStr(22,09,"",013);
WriteStr(23,09,'.. ',015);
WriteStr(25,09,'+--+ ',013);
WriteStr(29,09,'.....',015);
WriteStr(35,09,"",011);
WriteStr(36,09,'__ ',015);
WriteStr(38,09,"",011);
WriteStr(39,09,'.....',015);
WriteStr(50,09,"",012);
WriteStr(51,09,'__|.....|__ ',015);
WriteStr(64,09,"",012);
WriteStr(65,09,'.....',015);
WriteStr(80,09,"",014);
WriteStr(01,10,"",014);
WriteStr(02,10,'.....',015);
WriteStr(19,10,"",013);
WriteStr(20,10,'__ ',015);

```

```

WriteStr(22,10,"",013);
WriteStr(23,10,'.....',015);
WriteStr(35,10,"",011);
WriteStr(36,10,'__',015);
WriteStr(38,10,"",011);
WriteStr(39,10,'.....',015);
WriteStr(50,10,"",012);
WriteStr(51,10,'__+-----+',015);
WriteStr(63,10,'++',012);
WriteStr(65,10,'.....',015);
WriteStr(80,10,"",014);
WriteStr(01,11,"",014);
WriteStr(02,11,'.....',015);
WriteStr(19,11,"",013);
WriteStr(20,11,'__',015);
WriteStr(22,11,"",013);
WriteStr(23,11,'.....',015);
WriteStr(35,11,"",011);
WriteStr(36,11,'__',015);
WriteStr(38,11,"",011);
WriteStr(39,11,'.....',015);
WriteStr(50,11,"",012);
WriteStr(51,11,'_____',015);
WriteStr(62,11,'++',012);
WriteStr(64,11,'.....',015);
WriteStr(80,11,"",014);
WriteStr(01,12,"",014);
WriteStr(02,12,'.....',015);
WriteStr(19,12,"",013);
WriteStr(20,12,'__',015);
WriteStr(22,12,"",013);
WriteStr(23,12,'.....',015);
WriteStr(35,12,"",011);
WriteStr(36,12,'__',015);
WriteStr(38,12,"",011);
WriteStr(39,12,'.....',015);
WriteStr(50,12,"",012);
WriteStr(51,12,'__',015);
WriteStr(53,12,'+-----+',012);
WriteStr(63,12,'.....',015);
WriteStr(80,12,"",014);
WriteStr(01,13,"",014);
WriteStr(02,13,'.....',015);
WriteStr(17,13,'+-+',013);
WriteStr(20,13,'__',015);
WriteStr(22,13,'+-+',013);
WriteStr(25,13,'.....',015);

```

```

WriteStr(33,13,'+-+',011);
WriteStr(36,13,'__',015);
WriteStr(38,13,'+-+',011);
WriteStr(41,13,'.....',015);
WriteStr(48,13,'+-+',012);
WriteStr(51,13,'__',015);
WriteStr(53,13,'+-+',012);
WriteStr(56,13,'.....',015);
WriteStr(80,13,'" ',014);
WriteStr(01,14,'" ',014);
WriteStr(02,14,'.....',015);
WriteStr(17,14,'" ',013);
WriteStr(18,14,'_____',015);
WriteStr(24,14,'" ',013);
WriteStr(25,14,'.TEXT...',015);
WriteStr(33,14,'" ',011);
WriteStr(34,14,'_____',015);
WriteStr(40,14,'" ',011);
WriteStr(41,14,'.IMAGE.',015);
WriteStr(48,14,'" ',012);
WriteStr(49,14,'_____',015);
WriteStr(55,14,'" ',012);
WriteStr(56,14,'.PROCESSOR...',015);
WriteStr(80,14,'" ',014);
WriteStr(01,15,'" ',014);
WriteStr(02,15,'.....',015);
WriteStr(17,15,'+-----+',013);
WriteStr(25,15,'.....',015);
WriteStr(33,15,'+-----+',011);
WriteStr(41,15,'.....',015);
WriteStr(48,15,'+-----+',012);
WriteStr(56,15,'.....',015);
WriteStr(80,15,'" ',014);
WriteStr(01,16,'" ',014);
WriteStr(02,16,'.....',015);
WriteStr(80,16,'" ',014);
WriteStr(01,17,'" ',014);
WriteStr(02,17,'.....',015);
WriteStr(80,17,'" ',014);
WriteStr(01,18,'" ',014);
WriteStr(02,18,'',015);
WriteStr(80,18,'" ',014);
WriteStr(01,19,'" ',014);
WriteStr(02,19,'',015);
WriteStr(80,19,'" ',014);
WriteStr(01,20,'" ',014);
WriteStr(02,20,' FOCHI MICHELE & AMATULLI DAVIDE',015);

```

```

WriteStr(80,20,"",014);
WriteStr(01,21,"",014);
WriteStr(02,21,'      VERSIONE 1.0                                ',015);
WriteStr(80,21,"",014);
WriteStr(01,22,"",014);
WriteStr(02,22,'                                ',015);
WriteStr(80,22,"",014);
WriteStr(01,23,"",014);
WriteStr(02,23,'                                ',015);
WriteStr(80,23,"",014);
WriteStr(01,24,'+-----+ ',014);

End; { P4 }

```

File In Turbo Pascal Numero 5 (Con Diretta)

```

{                                                                    }
{      Questo file è stato creato da T.I.P., scritto da Fochi Michele      }
{                                                                    }
{                                                                    }
{      La procedura "ClrScr" e le costanti "TextAttr" e "LightGray"      }
{      sono nella unit CRT.TPU.                                           }
{      La procedura "Diretta" è nella unit VIDEO.TPU.                   }
{                                                                    }
{      Per eseguire il programma occorre quindi la seguente linea      }
{      di codice (nelle prime righe del programma):                     }
{                                                                    }
{      Uses Crt,Video;                                                    }

```

Procedure P5;

Var St: String;

Begin { P5 }

```

{ Canello lo schermo }
TextAttr := LightGray;
ClrScr;

```

{ Ora viene riempito lo schermo }

```

St := '+-----+ ';
Diretta(01,01,St,014,00);
St := "";
Diretta(01,02,St,014,00);
St := '                                ';
Diretta(02,02,St,015,00);

```

```

St := '';
Diretta(80,02,St,014,00);
St := '';
Diretta(01,03,St,014,00);
St := '          ';
Diretta(02,03,St,015,00);
St := '';
Diretta(80,03,St,014,00);
St := '';
Diretta(01,04,St,014,00);
St := '          .....';
Diretta(02,04,St,015,00);
St := '';
Diretta(80,04,St,014,00);
St := '';
Diretta(01,05,St,014,00);
St := '          .....';
Diretta(02,05,St,015,00);
St := '';
Diretta(80,05,St,014,00);
St := '';
Diretta(01,06,St,014,00);
St := '    ...';
Diretta(02,06,St,015,00);
St := '+-----+';
Diretta(13,06,St,013,00);
St := '....';
Diretta(29,06,St,015,00);
St := '+-----+';
Diretta(33,06,St,011,00);
St := '.....';
Diretta(41,06,St,015,00);
St := '+-----+';
Diretta(48,06,St,012,00);
St := '.....';
Diretta(63,06,St,015,00);
St := '';
Diretta(80,06,St,014,00);
St := '';
Diretta(01,07,St,014,00);
St := '    ...';
Diretta(02,07,St,015,00);
St := '';
Diretta(13,07,St,013,00);
St := '_____';
Diretta(14,07,St,015,00);
St := '';

```

```

Diretta(28,07,St,013,00);
St := '....';
Diretta(29,07,St,015,00);
St := '|';
Diretta(33,07,St,011,00);
St := '_____';
Diretta(34,07,St,015,00);
St := '|';
Diretta(40,07,St,011,00);
St := '.....';
Diretta(41,07,St,015,00);
St := '|';
Diretta(48,07,St,012,00);
St := '_____';
Diretta(49,07,St,015,00);
St := '++';
Diretta(62,07,St,012,00);
St := '.....';
Diretta(64,07,St,015,00);
St := '|';
Diretta(80,07,St,014,00);
St := '|';
Diretta(01,08,St,014,00);
St := '...';
Diretta(02,08,St,015,00);
St := '|';
Diretta(13,08,St,013,00);
St := '___';
Diretta(14,08,St,015,00);
St := '+--+';
Diretta(16,08,St,013,00);
St := '___';
Diretta(20,08,St,015,00);
St := '+--+';
Diretta(22,08,St,013,00);
St := '___';
Diretta(26,08,St,015,00);
St := '|';
Diretta(28,08,St,013,00);
St := '....';
Diretta(29,08,St,015,00);
St := '+-+';
Diretta(33,08,St,011,00);
St := '___';
Diretta(36,08,St,015,00);
St := '+-+';
Diretta(38,08,St,011,00);

```

```

St := '.....';
Diretta(41,08,St,015,00);
St := '+-+';
Diretta(48,08,St,012,00);
St := '___+-----+___';
Diretta(51,08,St,015,00);
St := '++';
Diretta(63,08,St,012,00);
St := '....';
Diretta(65,08,St,015,00);
St := '"';
Diretta(80,08,St,014,00);
St := '"';
Diretta(01,09,St,014,00);
St := '...';
Diretta(02,09,St,015,00);
St := '+--+';
Diretta(13,09,St,013,00);
St := '..';
Diretta(17,09,St,015,00);
St := '"';
Diretta(19,09,St,013,00);
St := '___';
Diretta(20,09,St,015,00);
St := '"';
Diretta(22,09,St,013,00);
St := '..';
Diretta(23,09,St,015,00);
St := '+--+';
Diretta(25,09,St,013,00);
St := '.....';
Diretta(29,09,St,015,00);
St := '"';
Diretta(35,09,St,011,00);
St := '___';
Diretta(36,09,St,015,00);
St := '"';
Diretta(38,09,St,011,00);
St := '.....';
Diretta(39,09,St,015,00);
St := '"';
Diretta(50,09,St,012,00);
St := '___!.....!___';
Diretta(51,09,St,015,00);
St := '"';
Diretta(64,09,St,012,00);
St := '....';

```

```

Diretta(65,09,St,015,00);
St := '';
Diretta(80,09,St,014,00);
St := '';
Diretta(01,10,St,014,00);
St := '.....';
Diretta(02,10,St,015,00);
St := '';
Diretta(19,10,St,013,00);
St := '___';
Diretta(20,10,St,015,00);
St := '';
Diretta(22,10,St,013,00);
St := '.....';
Diretta(23,10,St,015,00);
St := '';
Diretta(35,10,St,011,00);
St := '___';
Diretta(36,10,St,015,00);
St := '';
Diretta(38,10,St,011,00);
St := '.....';
Diretta(39,10,St,015,00);
St := '';
Diretta(50,10,St,012,00);
St := '___+-----+___';
Diretta(51,10,St,015,00);
St := '++';
Diretta(63,10,St,012,00);
St := '....';
Diretta(65,10,St,015,00);
St := '';
Diretta(80,10,St,014,00);
St := '';
Diretta(01,11,St,014,00);
St := '.....';
Diretta(02,11,St,015,00);
St := '';
Diretta(19,11,St,013,00);
St := '___';
Diretta(20,11,St,015,00);
St := '';
Diretta(22,11,St,013,00);
St := '.....';
Diretta(23,11,St,015,00);
St := '';
Diretta(35,11,St,011,00);

```



```

St := '___';
Diretta(36,11,St,015,00);
St := '"';
Diretta(38,11,St,011,00);
St := '.....';
Diretta(39,11,St,015,00);
St := '"';
Diretta(50,11,St,012,00);
St := '_____';
Diretta(51,11,St,015,00);
St := '++';
Diretta(62,11,St,012,00);
St := '.....';
Diretta(64,11,St,015,00);
St := '"';
Diretta(80,11,St,014,00);
St := '"';
Diretta(01,12,St,014,00);
St := '.....';
Diretta(02,12,St,015,00);
St := '"';
Diretta(19,12,St,013,00);
St := '___';
Diretta(20,12,St,015,00);
St := '"';
Diretta(22,12,St,013,00);
St := '.....';
Diretta(23,12,St,015,00);
St := '"';
Diretta(35,12,St,011,00);
St := '___';
Diretta(36,12,St,015,00);
St := '"';
Diretta(38,12,St,011,00);
St := '.....';
Diretta(39,12,St,015,00);
St := '"';
Diretta(50,12,St,012,00);
St := '___';
Diretta(51,12,St,015,00);
St := '+-----+';
Diretta(53,12,St,012,00);
St := '.....';
Diretta(63,12,St,015,00);
St := '"';
Diretta(80,12,St,014,00);
St := '"';

```

```

Diretta(01,13,St,014,00);
St := '.....';
Diretta(02,13,St,015,00);
St := '+-+';
Diretta(17,13,St,013,00);
St := '___';
Diretta(20,13,St,015,00);
St := '+-+';
Diretta(22,13,St,013,00);
St := '.....';
Diretta(25,13,St,015,00);
St := '+-+';
Diretta(33,13,St,011,00);
St := '___';
Diretta(36,13,St,015,00);
St := '+-+';
Diretta(38,13,St,011,00);
St := '.....';
Diretta(41,13,St,015,00);
St := '+-+';
Diretta(48,13,St,012,00);
St := '___';
Diretta(51,13,St,015,00);
St := '+-+';
Diretta(53,13,St,012,00);
St := '.....';
Diretta(56,13,St,015,00);
St := '"';
Diretta(80,13,St,014,00);
St := '"';
Diretta(01,14,St,014,00);
St := '.....';
Diretta(02,14,St,015,00);
St := '"';
Diretta(17,14,St,013,00);
St := '_____';
Diretta(18,14,St,015,00);
St := '"';
Diretta(24,14,St,013,00);
St := 'TEXT...';
Diretta(25,14,St,015,00);
St := '"';
Diretta(33,14,St,011,00);
St := '_____';
Diretta(34,14,St,015,00);
St := '"';
Diretta(40,14,St,011,00);

```

```

St := 'IMAGE';
Diretta(41,14,St,015,00);
St := '';
Diretta(48,14,St,012,00);
St := '_____';
Diretta(49,14,St,015,00);
St := '';
Diretta(55,14,St,012,00);
St := 'PROCESSOR...';
Diretta(56,14,St,015,00);
St := '';
Diretta(80,14,St,014,00);
St := '';
Diretta(01,15,St,014,00);
St := '.....';
Diretta(02,15,St,015,00);
St := '+-----+';
Diretta(17,15,St,013,00);
St := '.....';
Diretta(25,15,St,015,00);
St := '+-----+';
Diretta(33,15,St,011,00);
St := '.....';
Diretta(41,15,St,015,00);
St := '+-----+';
Diretta(48,15,St,012,00);
St := '.....';
Diretta(56,15,St,015,00);
St := '';
Diretta(80,15,St,014,00);
St := '';
Diretta(01,16,St,014,00);
St := '.....';
Diretta(02,16,St,015,00);
St := '';
Diretta(80,16,St,014,00);
St := '';
Diretta(01,17,St,014,00);
St := '.....';
Diretta(02,17,St,015,00);
St := '';
Diretta(80,17,St,014,00);
St := '';
Diretta(01,18,St,014,00);
St := ' ';
Diretta(02,18,St,015,00);
St := '';

```

```

Diretta(80,18,St,014,00);
St := '';
Diretta(01,19,St,014,00);
St := '          ';
Diretta(02,19,St,015,00);
St := '';
Diretta(80,19,St,014,00);
St := '';
Diretta(01,20,St,014,00);
St := '      FOCHI MICHELE & AMATULLI DAVIDE          ';
Diretta(02,20,St,015,00);
St := '';
Diretta(80,20,St,014,00);
St := '';
Diretta(01,21,St,014,00);
St := '      VERSIONE 1.0          ';
Diretta(02,21,St,015,00);
St := '';
Diretta(80,21,St,014,00);
St := '';
Diretta(01,22,St,014,00);
St := '          ';
Diretta(02,22,St,015,00);
St := '';
Diretta(80,22,St,014,00);
St := '';
Diretta(01,23,St,014,00);
St := '          ';
Diretta(02,23,St,015,00);
St := '';
Diretta(80,23,St,014,00);
St := '+-----+';
Diretta(01,24,St,014,00);

End; { P5 }

```

File In Turbo Pascal Numero 6 (Memoria Video)

```

{
{      Questo file è stato creato da T.I.P., scritto da Fochi Michele      }
{
{
{      La procedura "ClrScr" e le costanti "TextAttr" e "LightGray"      }
{      sono nella unit CRT.TPU.                                          }
{
{      Per eseguire il programma occorre quindi la seguente linea      }
{      di codice (nelle prime righe del programma):                      }

```



```

Var Fisico: ^RecPage; { Inizio dello schermo fisico }

Begin { P6 }

{ Cancello lo schermo }
TextAttr := LightGray;
ClrScr;

{ Calcolo dell' inizio della memoria video }
If (Mem[$0000:$0499] = 7)
  Then
    Fisico := PTR($B000,$0000)
  Else
    Fisico := PTR($B800,$0000);

{ Il contenuto del record maschera va ora in memoria video }
Fisico^ := Screen;

End; { P6 }

```

File In Turbo Pascal Numero 7 (File Maschera)

```

{
{      Questo file è stato creato da T.I.P., scritto da Fochi Michele      }
{
{
{
{      La procedura "ClrScr" e le costanti "TextAttr" e "LightGray"
{      sono nella unit CRT.TPU.
{
{
{      Per eseguire il programma occorre quindi la seguente linea
{      di codice (nelle prime righe del programma):
{
{
{      Uses Crt;

```

Procedure P7;

```

Type RecPage= Array [1..24,1..80] Of
  Record
    Ch: Char;
    At: Byte;
  End; { RecPage }

```

```

Rec=  Record
  Header: String[007];
  Page:  RecPage;

```



```

        End; { Rec }

Var MSKFile: File Of      { File strutturato }
    Rec;
    MSKRec: ^Rec;         { Record del file }
    Fisico: ^RecPage;     { Inizio dello schermo fisico }

Begin { P7 }

{ Cancello lo schermo }
TextAttr := LightGray;
ClrScr;

{ Alloco nello Heap il record MSKRec (3848 Bytes) }
New(MSKRec);

{ Assegnazione del file maschera }
Assign(MSKFile,'P7');

{$I-} Reset(MSKFile); {$I+}

{ Se il file esiste viene letto }
If (IOResult = 0)
Then
    Read(MSKFile,MSKRec^);

{ Calcolo dell' inizio della memoria video }
If (Mem[$0000:$0499] = 7)
Then
    Fisico := PTR($B000,$0000)

Else
    Fisico := PTR($B800,$0000);

{ Il contenuto del record maschera va ora in memoria video }
Fisico^ := MSKRec^.Page;

{ Libero la memoria dal record maschera (3848 Bytes) }
Dispose(MSKRec);

End; { P7 }

```

Turbo C

File In Turbo C Numero 1 (Con GoToXY E CPrintf)

```
/* */
/* Questo file è stato creato da T.I.P., scritto da Fochi Michele */
/* */
/* */
/* Le procedure "clrscr()", "textattr()", "gotoxy()" e */
/* "cprintf()" e la costante "LIGHTGRAY" sono nella unit CONIO.H */
/* */
/* Per eseguire il programma occorre quindi la seguente linea */
/* di codice (nelle prime righe del programma): */
/* */
/* #include <conio.h> */
```

void C1()

```
{ /* C1 */
```

```
/* Cancello lo schermo */
```

```
textattr(LIGHTGRAY);
```

```
clrscr();
```

```
/* Ora viene riempito lo schermo */
```

```
gotoxy(1,1);    textattr(14);
cprintf("+-----+");
gotoxy(1,2);    textattr(14);
cprintf("|");
gotoxy(2,2);    textattr(15);
cprintf("                ");
gotoxy(80,2);   textattr(14);
cprintf("|");
gotoxy(1,3);    textattr(14);
cprintf("|");
gotoxy(2,3);    textattr(15);
cprintf("                ");
gotoxy(80,3);   textattr(14);
cprintf("|");
gotoxy(1,4);    textattr(14);
cprintf("|");
gotoxy(2,4);    textattr(15);
cprintf(".....");
gotoxy(80,4);   textattr(14);
cprintf("|");
gotoxy(1,5);    textattr(14);
cprintf("|");
```

```

gotoxy(2,5);      textattr(15);
cprintf(" ..... ");
gotoxy(80,5);     textattr(14);
cprintf("|");
gotoxy(1,6);      textattr(14);
cprintf("|");
gotoxy(2,6);      textattr(15);
cprintf(" ...");
gotoxy(13,6);     textattr(13);
cprintf("+-----+");
gotoxy(29,6);     textattr(15);
cprintf("....");
gotoxy(33,6);     textattr(11);
cprintf("+-----+");
gotoxy(41,6);     textattr(15);
cprintf(".....");
gotoxy(48,6);     textattr(12);
cprintf("+-----+");
gotoxy(63,6);     textattr(15);
cprintf("..... ");
gotoxy(80,6);     textattr(14);
cprintf("|");
gotoxy(1,7);      textattr(14);
cprintf("|");
gotoxy(2,7);      textattr(15);
cprintf(" ...");
gotoxy(13,7);     textattr(13);
cprintf("|");
gotoxy(14,7);     textattr(15);
cprintf("_____");
gotoxy(28,7);     textattr(13);
cprintf("|");
gotoxy(29,7);     textattr(15);
cprintf("....");
gotoxy(33,7);     textattr(11);
cprintf("|");
gotoxy(34,7);     textattr(15);
cprintf("_____");
gotoxy(40,7);     textattr(11);
cprintf("|");
gotoxy(41,7);     textattr(15);
cprintf(".....");
gotoxy(48,7);     textattr(12);
cprintf("|");
gotoxy(49,7);     textattr(15);
cprintf("_____");
gotoxy(62,7);     textattr(12);

```

```

cprintf("++");
gotoxy(64,7);    textattr(15);
cprintf(".....");
gotoxy(80,7);    textattr(14);
cprintf("");
gotoxy(1,8);     textattr(14);
cprintf("");
gotoxy(2,8);     textattr(15);
cprintf("    ...");
gotoxy(13,8);    textattr(13);
cprintf("");
gotoxy(14,8);    textattr(15);
cprintf("  ");
gotoxy(16,8);    textattr(13);
cprintf("+++");
gotoxy(20,8);    textattr(15);
cprintf("  ");
gotoxy(22,8);    textattr(13);
cprintf("+++");
gotoxy(26,8);    textattr(15);
cprintf("  ");
gotoxy(28,8);    textattr(13);
cprintf("");
gotoxy(29,8);    textattr(15);
cprintf("....");
gotoxy(33,8);    textattr(11);
cprintf("+-");
gotoxy(36,8);    textattr(15);
cprintf("  ");
gotoxy(38,8);    textattr(11);
cprintf("+-");
gotoxy(41,8);    textattr(15);
cprintf(".....");
gotoxy(48,8);    textattr(12);
cprintf("+-");
gotoxy(51,8);    textattr(15);
cprintf("  +-----+ ");
gotoxy(63,8);    textattr(12);
cprintf("++");
gotoxy(65,8);    textattr(15);
cprintf("....");
gotoxy(80,8);    textattr(14);
cprintf("");
gotoxy(1,9);     textattr(14);
cprintf("");
gotoxy(2,9);     textattr(15);
cprintf("    ...");

```

```

gotoxy(13,9);    textattr(13);
cprintf("++");
gotoxy(17,9);    textattr(15);
cprintf("·");
gotoxy(19,9);    textattr(13);
cprintf("");
gotoxy(20,9);    textattr(15);
cprintf("__");
gotoxy(22,9);    textattr(13);
cprintf("");
gotoxy(23,9);    textattr(15);
cprintf("·");
gotoxy(25,9);    textattr(13);
cprintf("++");
gotoxy(29,9);    textattr(15);
cprintf(".....");
gotoxy(35,9);    textattr(11);
cprintf("");
gotoxy(36,9);    textattr(15);
cprintf("__");
gotoxy(38,9);    textattr(11);
cprintf("");
gotoxy(39,9);    textattr(15);
cprintf(".....");
gotoxy(50,9);    textattr(12);
cprintf("");
gotoxy(51,9);    textattr(15);
cprintf("__'|.....'|__");
gotoxy(64,9);    textattr(12);
cprintf("");
gotoxy(65,9);    textattr(15);
cprintf("....");
gotoxy(80,9);    textattr(14);
cprintf("");
gotoxy(1,10);    textattr(14);
cprintf("");
gotoxy(2,10);    textattr(15);
cprintf(".....");
gotoxy(19,10);   textattr(13);
cprintf("");
gotoxy(20,10);   textattr(15);
cprintf("__");
gotoxy(22,10);   textattr(13);
cprintf("");
gotoxy(23,10);   textattr(15);
cprintf(".....");
gotoxy(35,10);   textattr(11);

```

```

cprintf("");
gotoxy(36,10);    textattr(15);
cprintf("___");
gotoxy(38,10);    textattr(11);
cprintf("");
gotoxy(39,10);    textattr(15);
cprintf(".....");
gotoxy(50,10);    textattr(12);
cprintf("");
gotoxy(51,10);    textattr(15);
cprintf("__+-----+__");
gotoxy(63,10);    textattr(12);
cprintf("++");
gotoxy(65,10);    textattr(15);
cprintf("....");
gotoxy(80,10);    textattr(14);
cprintf("");
gotoxy(1,11);     textattr(14);
cprintf("");
gotoxy(2,11);     textattr(15);
cprintf(".....");
gotoxy(19,11);    textattr(13);
cprintf("");
gotoxy(20,11);    textattr(15);
cprintf("___");
gotoxy(22,11);    textattr(13);
cprintf("");
gotoxy(23,11);    textattr(15);
cprintf(".....");
gotoxy(35,11);    textattr(11);
cprintf("");
gotoxy(36,11);    textattr(15);
cprintf("___");
gotoxy(38,11);    textattr(11);
cprintf("");
gotoxy(39,11);    textattr(15);
cprintf(".....");
gotoxy(50,11);    textattr(12);
cprintf("");
gotoxy(51,11);    textattr(15);
cprintf("_____");
gotoxy(62,11);    textattr(12);
cprintf("++");
gotoxy(64,11);    textattr(15);
cprintf("....");
gotoxy(80,11);    textattr(14);
cprintf("");

```

```

gotoxy(1,12);    textattr(14);
cprintf("");
gotoxy(2,12);    textattr(15);
cprintf("      .....");
gotoxy(19,12);   textattr(13);
cprintf("");
gotoxy(20,12);   textattr(15);
cprintf("___");
gotoxy(22,12);   textattr(13);
cprintf("");
gotoxy(23,12);   textattr(15);
cprintf(".....");
gotoxy(35,12);   textattr(11);
cprintf("");
gotoxy(36,12);   textattr(15);
cprintf("___");
gotoxy(38,12);   textattr(11);
cprintf("");
gotoxy(39,12);   textattr(15);
cprintf(".....");
gotoxy(50,12);   textattr(12);
cprintf("");
gotoxy(51,12);   textattr(15);
cprintf("___");
gotoxy(53,12);   textattr(12);
cprintf("+-----+");
gotoxy(63,12);   textattr(15);
cprintf(".....");
gotoxy(80,12);   textattr(14);
cprintf("");
gotoxy(1,13);    textattr(14);
cprintf("");
gotoxy(2,13);    textattr(15);
cprintf("      .....");
gotoxy(17,13);   textattr(13);
cprintf("+--+");
gotoxy(20,13);   textattr(15);
cprintf("___");
gotoxy(22,13);   textattr(13);
cprintf("+--+");
gotoxy(25,13);   textattr(15);
cprintf(".....");
gotoxy(33,13);   textattr(11);
cprintf("+--+");
gotoxy(36,13);   textattr(15);
cprintf("___");
gotoxy(38,13);   textattr(11);

```

```

cprintf("+ - +");
gotoxy(41,13);    textattr(15);
cprintf(".....");
gotoxy(48,13);    textattr(12);
cprintf("+ - +");
gotoxy(51,13);    textattr(15);
cprintf(" _ ");
gotoxy(53,13);    textattr(12);
cprintf("+ - +");
gotoxy(56,13);    textattr(15);
cprintf(".....");
gotoxy(80,13);    textattr(14);
cprintf("|");
gotoxy(1,14);     textattr(14);
cprintf("|");
gotoxy(2,14);     textattr(15);
cprintf(" .....");
gotoxy(17,14);    textattr(13);
cprintf("|");
gotoxy(18,14);    textattr(15);
cprintf("_____");
gotoxy(24,14);    textattr(13);
cprintf("|");
gotoxy(25,14);    textattr(15);
cprintf("·TEXT··");
gotoxy(33,14);    textattr(11);
cprintf("|");
gotoxy(34,14);    textattr(15);
cprintf("_____");
gotoxy(40,14);    textattr(11);
cprintf("|");
gotoxy(41,14);    textattr(15);
cprintf("·IMAGE·");
gotoxy(48,14);    textattr(12);
cprintf("|");
gotoxy(49,14);    textattr(15);
cprintf("_____");
gotoxy(55,14);    textattr(12);
cprintf("|");
gotoxy(56,14);    textattr(15);
cprintf("·PROCESSOR··");
gotoxy(80,14);    textattr(14);
cprintf("|");
gotoxy(1,15);     textattr(14);
cprintf("|");
gotoxy(2,15);     textattr(15);
cprintf(" .....");

```



```

gotoxy(17,15);    textattr(13);
cprintf("+-----+");
gotoxy(25,15);    textattr(15);
cprintf(".....");
gotoxy(33,15);    textattr(11);
cprintf("+-----+");
gotoxy(41,15);    textattr(15);
cprintf(".....");
gotoxy(48,15);    textattr(12);
cprintf("+-----+");
gotoxy(56,15);    textattr(15);
cprintf(".....");
gotoxy(80,15);    textattr(14);
cprintf("");
gotoxy(1,16);     textattr(14);
cprintf("");
gotoxy(2,16);     textattr(15);
cprintf(".....");
gotoxy(80,16);    textattr(14);
cprintf("");
gotoxy(1,17);     textattr(14);
cprintf("");
gotoxy(2,17);     textattr(15);
cprintf(".....");
gotoxy(80,17);    textattr(14);
cprintf("");
gotoxy(1,18);     textattr(14);
cprintf("");
gotoxy(2,18);     textattr(15);
cprintf(".....");
gotoxy(80,18);    textattr(14);
cprintf("");
gotoxy(1,19);     textattr(14);
cprintf("");
gotoxy(2,19);     textattr(15);
cprintf(".....");
gotoxy(80,19);    textattr(14);
cprintf("");
gotoxy(1,20);     textattr(14);
cprintf("");
gotoxy(2,20);     textattr(15);
cprintf("FOCHI MICHELE & AMATULLI DAVIDE");
gotoxy(80,20);    textattr(14);
cprintf("");
gotoxy(1,21);     textattr(14);
cprintf("");
gotoxy(2,21);     textattr(15);

```

```

cprintf("    VERSIONE 1.0");
gotoxy(80,21);    textattr(14);
cprintf("");
gotoxy(1,22);    textattr(14);
cprintf("");
gotoxy(2,22);    textattr(15);
cprintf("
");
gotoxy(80,22);    textattr(14);
cprintf("");
gotoxy(1,23);    textattr(14);
cprintf("");
gotoxy(2,23);    textattr(15);
cprintf("
");
gotoxy(80,23);    textattr(14);
cprintf("");
gotoxy(1,24);    textattr(14);
cprintf("+-----+");

} /* C1 */

```

File In Turbo C Numero 2 (Senza GoToXY)

```

/*
/*      Questo file è stato creato da T.I.P., scritto da Fochi Michele
/*
/*
/*      Le procedure "clrscr()", "textattr()", "gotoxy()" e
/*      "cprintf()" e la costante "LIGHTGRAY" sono nella unit CONIO.H
/*
/*      Per eseguire il programma occorre quindi la seguente linea
/*      di codice (nelle prime righe del programma):
/*
/*      #include <conio.h>

```

void C2()

```

{ /* C2 */

/* Cancello lo schermo */
textattr(LIGHTGRAY);
clrscr();

/* Ora viene riempito lo schermo */

textattr(14);
cprintf("+-----+");
textattr(14);

```

```

cprintf("");
textattr(15);
cprintf(" ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" ..... ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" ..... ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" ...");
textattr(13);
cprintf("+-----+");
textattr(15);
cprintf("....");
textattr(11);
cprintf("+-----+");
textattr(15);
cprintf(".....");
textattr(12);
cprintf("+-----+");
textattr(15);
cprintf("..... ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" ...");
textattr(13);
cprintf("");

```

```

textattr(15);
cprintf("_____");
textattr(13);
cprintf("|");
textattr(15);
cprintf("....");
textattr(11);
cprintf("|");
textattr(15);
cprintf("_____");
textattr(11);
cprintf("|");
textattr(15);
cprintf(".....");
textattr(12);
cprintf("|");
textattr(15);
cprintf("_____");
textattr(12);
cprintf("++");
textattr(15);
cprintf(".....");
textattr(14);
cprintf("|");
textattr(14);
cprintf("|");
textattr(15);
cprintf("    ...");
textattr(13);
cprintf("|");
textattr(15);
cprintf("___");
textattr(13);
cprintf("+-+");
textattr(15);
cprintf("___");
textattr(13);
cprintf("+-+");
textattr(15);
cprintf("___");
textattr(13);
cprintf("|");
textattr(15);
cprintf("....");
textattr(11);
cprintf("+-");
textattr(15);

```

```

cprintf("__");
textattr(11);
cprintf("+--+");
textattr(15);
cprintf(".....");
textattr(12);
cprintf("+--+");
textattr(15);
cprintf("__+-----+__");
textattr(12);
cprintf("++");
textattr(15);
cprintf("....");
textattr(14);
cprintf("|");
textattr(14);
cprintf("|");
textattr(15);
cprintf("...");
textattr(13);
cprintf("+-+");
textattr(15);
cprintf("·");
textattr(13);
cprintf("|");
textattr(15);
cprintf("__");
textattr(13);
cprintf("|");
textattr(15);
cprintf("·");
textattr(13);
cprintf("+-+");
textattr(15);
cprintf(".....");
textattr(11);
cprintf("|");
textattr(15);
cprintf("__");
textattr(11);
cprintf("|");
textattr(15);
cprintf(".....");
textattr(12);
cprintf("|");
textattr(15);
cprintf("__!.....!__");

```

```

textattr(12);
cprintf("");
textattr(15);
cprintf("···· ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" ······");
textattr(13);
cprintf("");
textattr(15);
cprintf(" _");
textattr(13);
cprintf("");
textattr(15);
cprintf("········");
textattr(11);
cprintf("");
textattr(15);
cprintf(" _");
textattr(11);
cprintf("");
textattr(15);
cprintf("········");
textattr(12);
cprintf("");
textattr(15);
cprintf(" _+-----+ _");
textattr(12);
cprintf("++");
textattr(15);
cprintf("···· ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" ······");
textattr(13);
cprintf("");
textattr(15);
cprintf(" _");
textattr(13);
cprintf("");
textattr(15);

```

```

cprintf(".....");
textattr(11);
cprintf("");
textattr(15);
cprintf("__");
textattr(11);
cprintf("");
textattr(15);
cprintf(".....");
textattr(12);
cprintf("");
textattr(15);
cprintf("_____");
textattr(12);
cprintf("++");
textattr(15);
cprintf(".....");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" .....");
textattr(13);
cprintf("");
textattr(15);
cprintf("__");
textattr(13);
cprintf("");
textattr(15);
cprintf(".....");
textattr(11);
cprintf("");
textattr(15);
cprintf("__");
textattr(11);
cprintf("");
textattr(15);
cprintf(".....");
textattr(12);
cprintf("");
textattr(15);
cprintf("__");
textattr(12);
cprintf("+-----+");
textattr(15);
cprintf(".....");

```

```

textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" .....");
textattr(13);
cprintf("+ -");
textattr(15);
cprintf(" _");
textattr(13);
cprintf("+ -");
textattr(15);
cprintf(" .....");
textattr(11);
cprintf("+ -");
textattr(15);
cprintf(" _");
textattr(11);
cprintf("+ -");
textattr(15);
cprintf(" .....");
textattr(12);
cprintf("+ -");
textattr(15);
cprintf(" _");
textattr(12);
cprintf("+ -");
textattr(15);
cprintf(" ..... ");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf(" .....");
textattr(13);
cprintf("");
textattr(15);
cprintf(" _____");
textattr(13);
cprintf("");
textattr(15);
cprintf("·TEXT··");
textattr(11);
cprintf("");
textattr(15);

```



```

cprintf("_____");
textattr(11);
cprintf("");
textattr(15);
cprintf("·IMAGE·");
textattr(12);
cprintf("");
textattr(15);
cprintf("_____");
textattr(12);
cprintf("");
textattr(15);
cprintf("·PROCESSOR····");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf("·····");
textattr(13);
cprintf("+-----+");
textattr(15);
cprintf("·····");
textattr(11);
cprintf("+-----+");
textattr(15);
cprintf("·····");
textattr(12);
cprintf("+-----+");
textattr(15);
cprintf("··········");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf("······························");
textattr(14);
cprintf("");
textattr(14);
cprintf("");
textattr(15);
cprintf("······························");
textattr(14);
cprintf("");
textattr(14);
cprintf("");

```

```

textattr(15);
printf("                                ");
textattr(14);
printf("");
textattr(14);
printf("");
textattr(15);
printf("                                ");
textattr(14);
printf("");
textattr(14);
printf("");
textattr(15);
printf("    FOCHI MICHELE & AMATULLI DAVIDE    ");
textattr(14);
printf("");
textattr(14);
printf("");
textattr(15);
printf("    VERSIONE 1.0                                ");
textattr(14);
printf("");
textattr(14);
printf("");
textattr(15);
printf("                                ");
textattr(14);
printf("");
textattr(14);
printf("");
textattr(15);
printf("                                ");
textattr(14);
printf("");
textattr(14);
printf("+++++");
} /* C2 */

```

File In Turbo C Numero 3 (Colori Ottimizzati)

```

/*                                */
/*    Questo file è stato creato da T.I.P., scritto da Fochi Michele    */
/*                                */
/*                                */
/*    Le procedure "clrscr()", "textattr()", "gotoxy()" e                */
/*    "cprintf()" e la costante "LIGHTGRAY" sono nella unit CONIO.H      */

```

```

/* */
/*      Per eseguire il programma occorre quindi la seguente linea      */
/*      di codice (nelle prime righe del programma):                     */
/* */
/*      #include <conio.h>                                                 */

```

```
void C3()
```

```
{ /* C3 */
```

```

/* Cancello lo schermo */
textattr(LIGHTGRAY);
clrscr();

```

```
/* Ora viene riempito lo schermo */
```

```

textattr(11);
gotoxy(33,6);
cprintf("+-----+");
gotoxy(33,7);
cprintf("|");
gotoxy(40,7);
cprintf("|");
gotoxy(33,8);
cprintf("+--+");
gotoxy(38,8);
cprintf("+--+");
gotoxy(35,9);
cprintf("|");
gotoxy(38,9);
cprintf("|");
gotoxy(35,10);
cprintf("|");
gotoxy(38,10);
cprintf("|");
gotoxy(35,11);
cprintf("|");
gotoxy(38,11);
cprintf("|");
gotoxy(35,12);
cprintf("|");
gotoxy(38,12);
cprintf("|");
gotoxy(33,13);
cprintf("+--+");
gotoxy(38,13);

```

```

cprintf("+ - +");
gotoxy(33,14);
cprintf("|");
gotoxy(40,14);
cprintf("|");
gotoxy(33,15);
cprintf("+-----+");

textattr(12);
gotoxy(48,6);
cprintf("+-----+");
gotoxy(48,7);
cprintf("|");
gotoxy(62,7);
cprintf("+ +");
gotoxy(48,8);
cprintf("+ - +");
gotoxy(63,8);
cprintf("+ +");
gotoxy(50,9);
cprintf("|");
gotoxy(64,9);
cprintf("|");
gotoxy(50,10);
cprintf("|");
gotoxy(63,10);
cprintf("+ +");
gotoxy(50,11);
cprintf("|");
gotoxy(62,11);
cprintf("+ +");
gotoxy(50,12);
cprintf("|");
gotoxy(53,12);
cprintf("+-----+");
gotoxy(48,13);
cprintf("+ - +");
gotoxy(53,13);
cprintf("+ - +");
gotoxy(48,14);
cprintf("|");
gotoxy(55,14);
cprintf("|");
gotoxy(48,15);
cprintf("+-----+");

```

```

textattr(13);

```

```

gotoxy(13,6);
cprintf("+-----+");
gotoxy(13,7);
cprintf("|");
gotoxy(28,7);
cprintf("|");
gotoxy(13,8);
cprintf("|");
gotoxy(16,8);
cprintf("+-+");
gotoxy(22,8);
cprintf("+-+");
gotoxy(28,8);
cprintf("|");
gotoxy(13,9);
cprintf("+-+");
gotoxy(19,9);
cprintf("|");
gotoxy(22,9);
cprintf("|");
gotoxy(25,9);
cprintf("+-+");
gotoxy(19,10);
cprintf("|");
gotoxy(22,10);
cprintf("|");
gotoxy(19,11);
cprintf("|");
gotoxy(22,11);
cprintf("|");
gotoxy(19,12);
cprintf("|");
gotoxy(22,12);
cprintf("|");
gotoxy(17,13);
cprintf("+-");
gotoxy(22,13);
cprintf("+-");
gotoxy(17,14);
cprintf("|");
gotoxy(24,14);
cprintf("|");
gotoxy(17,15);
cprintf("+-----+");

textattr(14);
gotoxy(1,1);

```

```

cprintf("+-----+");
gotoxy(1,2);
cprintf("|");
gotoxy(80,2);
cprintf("|");
gotoxy(1,3);
cprintf("|");
gotoxy(80,3);
cprintf("|");
gotoxy(1,4);
cprintf("|");
gotoxy(80,4);
cprintf("|");
gotoxy(1,5);
cprintf("|");
gotoxy(80,5);
cprintf("|");
gotoxy(1,6);
cprintf("|");
gotoxy(80,6);
cprintf("|");
gotoxy(1,7);
cprintf("|");
gotoxy(80,7);
cprintf("|");
gotoxy(1,8);
cprintf("|");
gotoxy(80,8);
cprintf("|");
gotoxy(1,9);
cprintf("|");
gotoxy(80,9);
cprintf("|");
gotoxy(1,10);
cprintf("|");
gotoxy(80,10);
cprintf("|");
gotoxy(1,11);
cprintf("|");
gotoxy(80,11);
cprintf("|");
gotoxy(1,12);
cprintf("|");
gotoxy(80,12);
cprintf("|");
gotoxy(1,13);
cprintf("|");

```

```

gotoxy(80,13);
cprintf("");
gotoxy(1,14);
cprintf("");
gotoxy(80,14);
cprintf("");
gotoxy(1,15);
cprintf("");
gotoxy(80,15);
cprintf("");
gotoxy(1,16);
cprintf("");
gotoxy(80,16);
cprintf("");
gotoxy(1,17);
cprintf("");
gotoxy(80,17);
cprintf("");
gotoxy(1,18);
cprintf("");
gotoxy(80,18);
cprintf("");
gotoxy(1,19);
cprintf("");
gotoxy(80,19);
cprintf("");
gotoxy(1,20);
cprintf("");
gotoxy(80,20);
cprintf("");
gotoxy(1,21);
cprintf("");
gotoxy(80,21);
cprintf("");
gotoxy(1,22);
cprintf("");
gotoxy(80,22);
cprintf("");
gotoxy(1,23);
cprintf("");
gotoxy(80,23);
cprintf("");
gotoxy(1,24);
cprintf("+-----+");

textattr(15);
gotoxy(2,2);

```

```

cprintf("                                ");
gotoxy(2,3);
cprintf("                                ");
gotoxy(2,4);
cprintf(" ..... ");
gotoxy(2,5);
cprintf(" ..... ");
gotoxy(2,6);
cprintf("    ...");
gotoxy(29,6);
cprintf("....");
gotoxy(41,6);
cprintf(".....");
gotoxy(63,6);
cprintf("..... ");
gotoxy(2,7);
cprintf("    ...");
gotoxy(14,7);
cprintf("_____");
gotoxy(29,7);
cprintf("....");
gotoxy(34,7);
cprintf("_____");
gotoxy(41,7);
cprintf(".....");
gotoxy(49,7);
cprintf("_____");
gotoxy(64,7);
cprintf("..... ");
gotoxy(2,8);
cprintf("    ...");
gotoxy(14,8);
cprintf("_");
gotoxy(20,8);
cprintf("_");
gotoxy(26,8);
cprintf("_");
gotoxy(29,8);
cprintf("....");
gotoxy(36,8);
cprintf("_");
gotoxy(41,8);
cprintf(".....");
gotoxy(51,8);
cprintf("_+-----+_");
gotoxy(65,8);
cprintf("..... ");

```



```

gotoxy(2,9);
cprintf("    ...");
gotoxy(17,9);
cprintf("·");
gotoxy(20,9);
cprintf("___");
gotoxy(23,9);
cprintf("·");
gotoxy(29,9);
cprintf(".....");
gotoxy(36,9);
cprintf("___");
gotoxy(39,9);
cprintf(".....");
gotoxy(51,9);
cprintf("  '.....'  ");
gotoxy(65,9);
cprintf(".....");
gotoxy(2,10);
cprintf("    .....");
gotoxy(20,10);
cprintf("___");
gotoxy(23,10);
cprintf(".....");
gotoxy(36,10);
cprintf("___");
gotoxy(39,10);
cprintf(".....");
gotoxy(51,10);
cprintf("  +-----+  ");
gotoxy(65,10);
cprintf(".....");
gotoxy(2,11);
cprintf("    .....");
gotoxy(20,11);
cprintf("___");
gotoxy(23,11);
cprintf(".....");
gotoxy(36,11);
cprintf("___");
gotoxy(39,11);
cprintf(".....");
gotoxy(51,11);
cprintf("_____");
gotoxy(64,11);
cprintf(".....");
gotoxy(2,12);

```

```

cprintf("      .....");
gotoxy(20,12);
cprintf("_");
gotoxy(23,12);
cprintf(".....");
gotoxy(36,12);
cprintf("_");
gotoxy(39,12);
cprintf(".....");
gotoxy(51,12);
cprintf("_");
gotoxy(63,12);
cprintf(".....");
gotoxy(2,13);
cprintf("      .....");
gotoxy(20,13);
cprintf("_");
gotoxy(25,13);
cprintf(".....");
gotoxy(36,13);
cprintf("_");
gotoxy(41,13);
cprintf(".....");
gotoxy(51,13);
cprintf("_");
gotoxy(56,13);
cprintf(".....");
gotoxy(2,14);
cprintf("      .....");
gotoxy(18,14);
cprintf("_____");
gotoxy(25,14);
cprintf("·TEXT·");
gotoxy(34,14);
cprintf("_____");
gotoxy(41,14);
cprintf("·IMAGE·");
gotoxy(49,14);
cprintf("_____");
gotoxy(56,14);
cprintf("·PROCESSOR·");
gotoxy(2,15);
cprintf("      .....");
gotoxy(25,15);
cprintf(".....");
gotoxy(41,15);
cprintf(".....");

```

```

gotoxy(56,15);
cprintf(".....");
gotoxy(2,16);
cprintf(".....");
gotoxy(2,17);
cprintf(".....");
gotoxy(2,18);
cprintf("");
gotoxy(2,19);
cprintf("");
gotoxy(2,20);
cprintf("    FOCHI MICHELE & AMATULLI DAVIDE");
gotoxy(2,21);
cprintf("    VERSIONE 1.0");
gotoxy(2,22);
cprintf("");
gotoxy(2,23);
cprintf("");

} /* C3 */

```

File In Turbo C Numero 4 (Con DIRETTA)

```

/*
/*      Questo file è stato creato da T.I.P., scritto da Fochi Michele
/*
/*
/*      La procedura "Diretta" è nella unit VIDEO.H
/*      Le procedure "clrscr()" e "textattr()" e la costante
/*      "LIGHTGRAY" sono nella unit CONIO.H
/*
/*      Per eseguire il programma occorrono quindi le seguenti linee
/*      di codice (nelle prime righe del programma):
/*
/*      #include <conio.h>
/*      #include "video.h"
*/
*/

```

void C4()

```

{ /* C4 */

/* Cancello lo schermo */
textattr(LIGHTGRAY);
clrscr();

/* Ora viene riempito lo schermo */

```

```

Diretta(1,1,"+-----+",14);
Diretta(1,2,"|",14);
Diretta(2,2,"",15);
Diretta(80,2,"|",14);
Diretta(1,3,"|",14);
Diretta(2,3,"",15);
Diretta(80,3,"|",14);
Diretta(1,4,"|",14);
Diretta(2,4,".....",15);
Diretta(80,4,"|",14);
Diretta(1,5,"|",14);
Diretta(2,5,".....",15);
Diretta(80,5,"|",14);
Diretta(1,6,"|",14);
Diretta(2,6,"...",15);
Diretta(13,6,"+-----+",13);
Diretta(29,6,"....",15);
Diretta(33,6,"+-----+",11);
Diretta(41,6,".....",15);
Diretta(48,6,"+-----+",12);
Diretta(63,6,".....",15);
Diretta(80,6,"|",14);
Diretta(1,7,"|",14);
Diretta(2,7,"...",15);
Diretta(13,7,"|",13);
Diretta(14,7,"_____",15);
Diretta(28,7,"|",13);
Diretta(29,7,"....",15);
Diretta(33,7,"|",11);
Diretta(34,7,"_____",15);
Diretta(40,7,"|",11);
Diretta(41,7,".....",15);
Diretta(48,7,"|",12);
Diretta(49,7,"_____",15);
Diretta(62,7,"++",12);
Diretta(64,7,".....",15);
Diretta(80,7,"|",14);
Diretta(1,8,"|",14);
Diretta(2,8,"...",15);
Diretta(13,8,"|",13);
Diretta(14,8,"_",15);
Diretta(16,8,"+--+",13);
Diretta(20,8,"_",15);
Diretta(22,8,"+--+",13);
Diretta(26,8,"_",15);
Diretta(28,8,"|",13);
Diretta(29,8,"....",15);

```

Diretta(33,8,"+-",11);
 Diretta(36,8,"__",15);
 Diretta(38,8,"+-",11);
 Diretta(41,8,".....",15);
 Diretta(48,8,"+-",12);
 Diretta(51,8,"__+-----+__",15);
 Diretta(63,8,"++",12);
 Diretta(65,8,"....",15);
 Diretta(80,8,"|",14);
 Diretta(1,9,"|",14);
 Diretta(2,9,".....",15);
 Diretta(13,9,"+-+",13);
 Diretta(17,9,"..",15);
 Diretta(19,9,"|",13);
 Diretta(20,9,"__",15);
 Diretta(22,9,"|",13);
 Diretta(23,9,"..",15);
 Diretta(25,9,"+-+",13);
 Diretta(29,9,".....",15);
 Diretta(35,9,"|",11);
 Diretta(36,9,"__",15);
 Diretta(38,9,"|",11);
 Diretta(39,9,".....",15);
 Diretta(50,9,"|",12);
 Diretta(51,9,"_|.....|__",15);
 Diretta(64,9,"|",12);
 Diretta(65,9,"....",15);
 Diretta(80,9,"|",14);
 Diretta(1,10,"|",14);
 Diretta(2,10,".....",15);
 Diretta(19,10,"|",13);
 Diretta(20,10,"__",15);
 Diretta(22,10,"|",13);
 Diretta(23,10,".....",15);
 Diretta(35,10,"|",11);
 Diretta(36,10,"__",15);
 Diretta(38,10,"|",11);
 Diretta(39,10,".....",15);
 Diretta(50,10,"|",12);
 Diretta(51,10,"__+-----+__",15);
 Diretta(63,10,"++",12);
 Diretta(65,10,"....",15);
 Diretta(80,10,"|",14);
 Diretta(1,11,"|",14);
 Diretta(2,11,".....",15);
 Diretta(19,11,"|",13);
 Diretta(20,11,"__",15);

```

Diretta(22,11,"",13);
Diretta(23,11,".....",15);
Diretta(35,11,"",11);
Diretta(36,11,"_",15);
Diretta(38,11,"",11);
Diretta(39,11,".....",15);
Diretta(50,11,"",12);
Diretta(51,11,"_____",15);
Diretta(62,11,"++",12);
Diretta(64,11,".....",15);
Diretta(80,11,"",14);
Diretta(1,12,"",14);
Diretta(2,12,".....",15);
Diretta(19,12,"",13);
Diretta(20,12,"_",15);
Diretta(22,12,"",13);
Diretta(23,12,".....",15);
Diretta(35,12,"",11);
Diretta(36,12,"_",15);
Diretta(38,12,"",11);
Diretta(39,12,".....",15);
Diretta(50,12,"",12);
Diretta(51,12,"_",15);
Diretta(53,12,"+-----+",12);
Diretta(63,12,".....",15);
Diretta(80,12,"",14);
Diretta(1,13,"",14);
Diretta(2,13,".....",15);
Diretta(17,13,"+-",13);
Diretta(20,13,"_",15);
Diretta(22,13,"+-",13);
Diretta(25,13,".....",15);
Diretta(33,13,"+-",11);
Diretta(36,13,"_",15);
Diretta(38,13,"+-",11);
Diretta(41,13,".....",15);
Diretta(48,13,"+-",12);
Diretta(51,13,"_",15);
Diretta(53,13,"+-",12);
Diretta(56,13,".....",15);
Diretta(80,13,"",14);
Diretta(1,14,"",14);
Diretta(2,14,".....",15);
Diretta(17,14,"",13);
Diretta(18,14,"_____",15);
Diretta(24,14,"",13);
Diretta(25,14,"·TEXT··",15);

```

```

Diretta(33,14,"",11);
Diretta(34,14,"_____",15);
Diretta(40,14,"",11);
Diretta(41,14,"·IMAGE·",15);
Diretta(48,14,"",12);
Diretta(49,14,"_____",15);
Diretta(55,14,"",12);
Diretta(56,14,"·PROCESSOR····",15);
Diretta(80,14,"",14);
Diretta(1,15,"",14);
Diretta(2,15,"·····",15);
Diretta(17,15,"+-----+",13);
Diretta(25,15,"·····",15);
Diretta(33,15,"+-----+",11);
Diretta(41,15,"·····",15);
Diretta(48,15,"+-----+",12);
Diretta(56,15,"··········",15);
Diretta(80,15,"",14);
Diretta(1,16,"",14);
Diretta(2,16,"······························",15);
Diretta(80,16,"",14);
Diretta(1,17,"",14);
Diretta(2,17,"······························",15);
Diretta(80,17,"",14);
Diretta(1,18,"",14);
Diretta(2,18,"······························",15);
Diretta(80,18,"",14);
Diretta(1,19,"",14);
Diretta(2,19,"······························",15);
Diretta(80,19,"",14);
Diretta(1,20,"",14);
Diretta(2,20,"FOCHI MICHELE & AMATULLI DAVIDE",15);
Diretta(80,20,"",14);
Diretta(1,21,"",14);
Diretta(2,21,"VERSIONE 1.0",15);
Diretta(80,21,"",14);
Diretta(1,22,"",14);
Diretta(2,22,"······························",15);
Diretta(80,22,"",14);
Diretta(1,23,"",14);
Diretta(2,23,"······························",15);
Diretta(80,23,"",14);
Diretta(1,24,"+-----+-----+-----+-----+-----+-----+-----+-----+-----+",14);

} /* C4 */

```

Data Base

File In Data Base Numero 1 (A Colori)

*

* Questo file è stato creato da T.I.P., scritto da Fochi Michele

*

@ 0,0 CLEAR

SET COLOR TO W/N

SET COLOR TO GR+/N

@ 00,00 SAY "+-----+"

SET COLOR TO GR+/N

@ 01,00 SAY ""

SET COLOR TO W+/N

@ 01,01 SAY " "

SET COLOR TO GR+/N

@ 01,79 SAY ""

SET COLOR TO GR+/N

@ 02,00 SAY ""

SET COLOR TO W+/N

@ 02,01 SAY " "

SET COLOR TO GR+/N

@ 02,79 SAY ""

SET COLOR TO GR+/N

@ 03,00 SAY ""

SET COLOR TO W+/N

@ 03,01 SAY " "

SET COLOR TO GR+/N

@ 03,79 SAY ""

SET COLOR TO GR+/N

@ 04,00 SAY ""

SET COLOR TO W+/N

@ 04,01 SAY " "

SET COLOR TO GR+/N

@ 04,79 SAY ""

SET COLOR TO GR+/N

@ 05,00 SAY ""

SET COLOR TO W+/N

@ 05,01 SAY " ..."

SET COLOR TO RB+/N

@ 05,12 SAY "+-----+"

SET COLOR TO W+/N

@ 05,28 SAY "...."

SET COLOR TO BG+/N


```

@ 05,32 SAY "+-----+"
SET COLOR TO W+/N
@ 05,40 SAY "....."
SET COLOR TO R+/N
@ 05,47 SAY "+-----+"
SET COLOR TO W+/N
@ 05,62 SAY "....."
SET COLOR TO GR+/N
@ 05,79 SAY ""
SET COLOR TO GR+/N
@ 06,00 SAY ""
SET COLOR TO W+/N
@ 06,01 SAY "... "
SET COLOR TO RB+/N
@ 06,12 SAY ""
SET COLOR TO W+/N
@ 06,13 SAY "_____ "
SET COLOR TO RB+/N
@ 06,27 SAY ""
SET COLOR TO W+/N
@ 06,28 SAY "...."
SET COLOR TO BG+/N
@ 06,32 SAY ""
SET COLOR TO W+/N
@ 06,33 SAY "_____"
SET COLOR TO BG+/N
@ 06,39 SAY ""
SET COLOR TO W+/N
@ 06,40 SAY "....."
SET COLOR TO R+/N
@ 06,47 SAY ""
SET COLOR TO W+/N
@ 06,48 SAY "_____ "
SET COLOR TO R+/N
@ 06,61 SAY "++"
SET COLOR TO W+/N
@ 06,63 SAY "....."
SET COLOR TO GR+/N
@ 06,79 SAY ""
SET COLOR TO GR+/N
@ 07,00 SAY ""
SET COLOR TO W+/N
@ 07,01 SAY "... "
SET COLOR TO RB+/N
@ 07,12 SAY ""
SET COLOR TO W+/N
@ 07,13 SAY "___"

```

SET COLOR TO RB+/N
 @ 07,15 SAY "+--+"
 SET COLOR TO W+/N
 @ 07,19 SAY "___"
 SET COLOR TO RB+/N
 @ 07,21 SAY "+--+"
 SET COLOR TO W+/N
 @ 07,25 SAY "___"
 SET COLOR TO RB+/N
 @ 07,27 SAY "||"
 SET COLOR TO W+/N
 @ 07,28 SAY "...."
 SET COLOR TO BG+/N
 @ 07,32 SAY "+-+"
 SET COLOR TO W+/N
 @ 07,35 SAY "___"
 SET COLOR TO BG+/N
 @ 07,37 SAY "+-+"
 SET COLOR TO W+/N
 @ 07,40 SAY "....."
 SET COLOR TO R+/N
 @ 07,47 SAY "+-+"
 SET COLOR TO W+/N
 @ 07,50 SAY "___+-----+___"
 SET COLOR TO R+/N
 @ 07,62 SAY "++"
 SET COLOR TO W+/N
 @ 07,64 SAY "...." "
 SET COLOR TO GR+/N
 @ 07,79 SAY "||"
 SET COLOR TO GR+/N
 @ 08,00 SAY "||"
 SET COLOR TO W+/N
 @ 08,01 SAY "..." "
 SET COLOR TO RB+/N
 @ 08,12 SAY "+--+"
 SET COLOR TO W+/N
 @ 08,16 SAY "..." "
 SET COLOR TO RB+/N
 @ 08,18 SAY "||"
 SET COLOR TO W+/N
 @ 08,19 SAY "___"
 SET COLOR TO RB+/N
 @ 08,21 SAY "||"
 SET COLOR TO W+/N
 @ 08,22 SAY "..." "
 SET COLOR TO RB+/N

```

@ 08,24 SAY "+--+"
SET COLOR TO W+/N
@ 08,28 SAY "....."
SET COLOR TO BG+/N
@ 08,34 SAY ""
SET COLOR TO W+/N
@ 08,35 SAY " _ "
SET COLOR TO BG+/N
@ 08,37 SAY ""
SET COLOR TO W+/N
@ 08,38 SAY "....."
SET COLOR TO R+/N
@ 08,49 SAY ""
SET COLOR TO W+/N
@ 08,50 SAY " _!.....! _ "
SET COLOR TO R+/N
@ 08,63 SAY ""
SET COLOR TO W+/N
@ 08,64 SAY ".... "
SET COLOR TO GR+/N
@ 08,79 SAY ""
SET COLOR TO GR+/N
@ 09,00 SAY ""
SET COLOR TO W+/N
@ 09,01 SAY " ..... "
SET COLOR TO RB+/N
@ 09,18 SAY ""
SET COLOR TO W+/N
@ 09,19 SAY " _ "
SET COLOR TO RB+/N
@ 09,21 SAY ""
SET COLOR TO W+/N
@ 09,22 SAY "....."
SET COLOR TO BG+/N
@ 09,34 SAY ""
SET COLOR TO W+/N
@ 09,35 SAY " _ "
SET COLOR TO BG+/N
@ 09,37 SAY ""
SET COLOR TO W+/N
@ 09,38 SAY "....."
SET COLOR TO R+/N
@ 09,49 SAY ""
SET COLOR TO W+/N
@ 09,50 SAY " _+-----+ _ "
SET COLOR TO R+/N
@ 09,62 SAY "++"

```

```

SET COLOR TO W+/N
@ 09,64 SAY "...."
SET COLOR TO GR+/N
@ 09,79 SAY ""
SET COLOR TO GR+/N
@ 10,00 SAY ""
SET COLOR TO W+/N
@ 10,01 SAY "....."
SET COLOR TO RB+/N
@ 10,18 SAY ""
SET COLOR TO W+/N
@ 10,19 SAY "___"
SET COLOR TO RB+/N
@ 10,21 SAY ""
SET COLOR TO W+/N
@ 10,22 SAY "....."
SET COLOR TO BG+/N
@ 10,34 SAY ""
SET COLOR TO W+/N
@ 10,35 SAY "___"
SET COLOR TO BG+/N
@ 10,37 SAY ""
SET COLOR TO W+/N
@ 10,38 SAY "....."
SET COLOR TO R+/N
@ 10,49 SAY ""
SET COLOR TO W+/N
@ 10,50 SAY "_____"
SET COLOR TO R+/N
@ 10,61 SAY "++"
SET COLOR TO W+/N
@ 10,63 SAY "...."
SET COLOR TO GR+/N
@ 10,79 SAY ""
SET COLOR TO GR+/N
@ 11,00 SAY ""
SET COLOR TO W+/N
@ 11,01 SAY "....."
SET COLOR TO RB+/N
@ 11,18 SAY ""
SET COLOR TO W+/N
@ 11,19 SAY "___"
SET COLOR TO RB+/N
@ 11,21 SAY ""
SET COLOR TO W+/N
@ 11,22 SAY "....."
SET COLOR TO BG+/N

```

```

@ 11,34 SAY ""
SET COLOR TO W+/N
@ 11,35 SAY " _ "
SET COLOR TO BG+/N
@ 11,37 SAY ""
SET COLOR TO W+/N
@ 11,38 SAY "....."
SET COLOR TO R+/N
@ 11,49 SAY ""
SET COLOR TO W+/N
@ 11,50 SAY " _ "
SET COLOR TO R+/N
@ 11,52 SAY "+-----+"
SET COLOR TO W+/N
@ 11,62 SAY "....."
SET COLOR TO GR+/N
@ 11,79 SAY ""
SET COLOR TO GR+/N
@ 12,00 SAY ""
SET COLOR TO W+/N
@ 12,01 SAY "....."
SET COLOR TO RB+/N
@ 12,16 SAY "+-+"
SET COLOR TO W+/N
@ 12,19 SAY " _ "
SET COLOR TO RB+/N
@ 12,21 SAY "+-+"
SET COLOR TO W+/N
@ 12,24 SAY "....."
SET COLOR TO BG+/N
@ 12,32 SAY "+-+"
SET COLOR TO W+/N
@ 12,35 SAY " _ "
SET COLOR TO BG+/N
@ 12,37 SAY "+-+"
SET COLOR TO W+/N
@ 12,40 SAY "....."
SET COLOR TO R+/N
@ 12,47 SAY "+-+"
SET COLOR TO W+/N
@ 12,50 SAY " _ "
SET COLOR TO R+/N
@ 12,52 SAY "+-+"
SET COLOR TO W+/N
@ 12,55 SAY "....."
SET COLOR TO GR+/N
@ 12,79 SAY ""

```

```

SET COLOR TO GR+/N
@ 13,00 SAY ""
SET COLOR TO W+/N
@ 13,01 SAY "....."
SET COLOR TO RB+/N
@ 13,16 SAY ""
SET COLOR TO W+/N
@ 13,17 SAY "_____"
SET COLOR TO RB+/N
@ 13,23 SAY ""
SET COLOR TO W+/N
@ 13,24 SAY "·TEXT··"
SET COLOR TO BG+/N
@ 13,32 SAY ""
SET COLOR TO W+/N
@ 13,33 SAY "_____"
SET COLOR TO BG+/N
@ 13,39 SAY ""
SET COLOR TO W+/N
@ 13,40 SAY "·IMAGE·"
SET COLOR TO R+/N
@ 13,47 SAY ""
SET COLOR TO W+/N
@ 13,48 SAY "_____"
SET COLOR TO R+/N
@ 13,54 SAY ""
SET COLOR TO W+/N
@ 13,55 SAY "·PROCESSOR··"
SET COLOR TO GR+/N
@ 13,79 SAY ""
SET COLOR TO GR+/N
@ 14,00 SAY ""
SET COLOR TO W+/N
@ 14,01 SAY "....."
SET COLOR TO RB+/N
@ 14,16 SAY "+-----+"
SET COLOR TO W+/N
@ 14,24 SAY "....."
SET COLOR TO BG+/N
@ 14,32 SAY "+-----+"
SET COLOR TO W+/N
@ 14,40 SAY "....."
SET COLOR TO R+/N
@ 14,47 SAY "+-----+"
SET COLOR TO W+/N
@ 14,55 SAY "....."
SET COLOR TO GR+/N

```

```

@ 14,79 SAY ""
SET COLOR TO GR+/N
@ 15,00 SAY ""
SET COLOR TO W+/N
@ 15,01 SAY " ..... "
SET COLOR TO GR+/N
@ 15,79 SAY ""
SET COLOR TO GR+/N
@ 16,00 SAY ""
SET COLOR TO W+/N
@ 16,01 SAY " ..... "
SET COLOR TO GR+/N
@ 16,79 SAY ""
SET COLOR TO GR+/N
@ 17,00 SAY ""
SET COLOR TO W+/N
@ 17,01 SAY " "
SET COLOR TO GR+/N
@ 17,79 SAY ""
SET COLOR TO GR+/N
@ 18,00 SAY ""
SET COLOR TO W+/N
@ 18,01 SAY " "
SET COLOR TO GR+/N
@ 18,79 SAY ""
SET COLOR TO GR+/N
@ 19,00 SAY ""
SET COLOR TO W+/N
@ 19,01 SAY " FOCHI MICHELE & AMATULLI DAVIDE "
SET COLOR TO GR+/N
@ 19,79 SAY ""
SET COLOR TO GR+/N
@ 20,00 SAY ""
SET COLOR TO W+/N
@ 20,01 SAY " VERSIONE 1.0 "
SET COLOR TO GR+/N
@ 20,79 SAY ""
SET COLOR TO GR+/N
@ 21,00 SAY ""
SET COLOR TO W+/N
@ 21,01 SAY " "
SET COLOR TO GR+/N
@ 21,79 SAY ""
SET COLOR TO GR+/N
@ 22,00 SAY ""
SET COLOR TO W+/N
@ 22,01 SAY " "

```

```

SET COLOR TO GR+/N
@ 22,79 SAY ""
|
SET COLOR TO GR+/N
@ 23,00 SAY "+-----+"
*
*       Fine del programma
*

```

File In Data Base Numero 2 (A Colori Ottimizzato)

```

*
*       Questo file è stato creato da T.I.P., scritto da Fochi Michele
*

```

```

@ 0,0 CLEAR

```

```

SET COLOR TO BG+/N
@ 05,32 SAY "+-----+"
@ 06,32 SAY ""
|
@ 06,39 SAY ""
|
@ 07,32 SAY "+-+"
@ 07,37 SAY "+-+"
@ 08,34 SAY ""
|
@ 08,37 SAY ""
|
@ 09,34 SAY ""
|
@ 09,37 SAY ""
|
@ 10,34 SAY ""
|
@ 10,37 SAY ""
|
@ 11,34 SAY ""
|
@ 11,37 SAY ""
|
@ 12,32 SAY "+-+"
@ 12,37 SAY "+-+"
@ 13,32 SAY ""
|
@ 13,39 SAY ""
|
@ 14,32 SAY "+-----+"
SET COLOR TO R+/N
@ 05,47 SAY "+-----+"
@ 06,47 SAY ""
|
@ 06,61 SAY "++"
@ 07,47 SAY "+-+"
@ 07,62 SAY "++"
@ 08,49 SAY ""
|
@ 08,63 SAY ""
|
@ 09,49 SAY ""
|
@ 09,62 SAY "++"
@ 10,49 SAY ""
|
@ 10,61 SAY "++"

```



```

@ 11,49 SAY ""
@ 11,52 SAY "+-----+"
@ 12,47 SAY "+-+"
@ 12,52 SAY "+-+"
@ 13,47 SAY ""
@ 13,54 SAY ""
@ 14,47 SAY "+-----+"
SET COLOR TO RB+/N
@ 05,12 SAY "+-----+"
@ 06,12 SAY ""
@ 06,27 SAY ""
@ 07,12 SAY ""
@ 07,15 SAY "+--+"
@ 07,21 SAY "+--+"
@ 07,27 SAY ""
@ 08,12 SAY "+--+"
@ 08,18 SAY ""
@ 08,21 SAY ""
@ 08,24 SAY "+--+"
@ 09,18 SAY ""
@ 09,21 SAY ""
@ 10,18 SAY ""
@ 10,21 SAY ""
@ 11,18 SAY ""
@ 11,21 SAY ""
@ 12,16 SAY "+-+"
@ 12,21 SAY "+-+"
@ 13,16 SAY ""
@ 13,23 SAY ""
@ 14,16 SAY "+-----+"
SET COLOR TO GR+/N
@ 00,00 SAY "+-----+"
@ 01,00 SAY ""
@ 01,79 SAY ""
@ 02,00 SAY ""
@ 02,79 SAY ""
@ 03,00 SAY ""
@ 03,79 SAY ""
@ 04,00 SAY ""
@ 04,79 SAY ""
@ 05,00 SAY ""
@ 05,79 SAY ""
@ 06,00 SAY ""
@ 06,79 SAY ""
@ 07,00 SAY ""
@ 07,79 SAY ""
@ 08,00 SAY ""

```

```

@ 08,79 SAY ""
@ 09,00 SAY ""
@ 09,79 SAY ""
@ 10,00 SAY ""
@ 10,79 SAY ""
@ 11,00 SAY ""
@ 11,79 SAY ""
@ 12,00 SAY ""
@ 12,79 SAY ""
@ 13,00 SAY ""
@ 13,79 SAY ""
@ 14,00 SAY ""
@ 14,79 SAY ""
@ 15,00 SAY ""
@ 15,79 SAY ""
@ 16,00 SAY ""
@ 16,79 SAY ""
@ 17,00 SAY ""
@ 17,79 SAY ""
@ 18,00 SAY ""
@ 18,79 SAY ""
@ 19,00 SAY ""
@ 19,79 SAY ""
@ 20,00 SAY ""
@ 20,79 SAY ""
@ 21,00 SAY ""
@ 21,79 SAY ""
@ 22,00 SAY ""
@ 22,79 SAY ""
@ 23,00 SAY "+-----+"
SET COLOR TO W+/N
@ 01,01 SAY " "
@ 02,01 SAY " "
@ 03,01 SAY " ..... "
@ 04,01 SAY " ..... "
@ 05,01 SAY " ..."
@ 05,28 SAY "...."
@ 05,40 SAY "....."
@ 05,62 SAY "..... "
@ 06,01 SAY " ..."
@ 06,13 SAY " _____ "
@ 06,28 SAY "...."
@ 06,33 SAY " _____ "
@ 06,40 SAY "....."
@ 06,48 SAY " _____ "
@ 06,63 SAY "..... "
@ 07,01 SAY " ..."

```

@ 07,13 SAY " _ "
 @ 07,19 SAY " _ "
 @ 07,25 SAY " _ "
 @ 07,28 SAY "...."
 @ 07,35 SAY " _ "
 @ 07,40 SAY "....."
 @ 07,50 SAY " _+-----+ _ "
 @ 07,64 SAY ".... " "
 @ 08,01 SAY " ..."
 @ 08,16 SAY "..
 @ 08,19 SAY " _ "
 @ 08,22 SAY "..
 @ 08,28 SAY "....."
 @ 08,35 SAY " _ "
 @ 08,38 SAY "....."
 @ 08,50 SAY " _!.....! _ "
 @ 08,64 SAY ".... " "
 @ 09,01 SAY ""
 @ 09,19 SAY " _ "
 @ 09,22 SAY "....."
 @ 09,35 SAY " _ "
 @ 09,38 SAY "....."
 @ 09,50 SAY " _+-----+ _ "
 @ 09,64 SAY ".... " "
 @ 10,01 SAY ""
 @ 10,19 SAY " _ "
 @ 10,22 SAY "....."
 @ 10,35 SAY " _ "
 @ 10,38 SAY "....."
 @ 10,50 SAY " _____ "
 @ 10,63 SAY ".... " "
 @ 11,01 SAY ""
 @ 11,19 SAY " _ "
 @ 11,22 SAY "....."
 @ 11,35 SAY " _ "
 @ 11,38 SAY "....."
 @ 11,50 SAY " _ "
 @ 11,62 SAY ".... " "
 @ 12,01 SAY ""
 @ 12,19 SAY " _ "
 @ 12,24 SAY "....."
 @ 12,35 SAY " _ "
 @ 12,40 SAY "....."
 @ 12,50 SAY " _ "
 @ 12,55 SAY "..... " "
 @ 13,01 SAY ""
 @ 13,17 SAY " _____ "

```

@ 13,24 SAY ".TEXT..."
@ 13,33 SAY "_____"
@ 13,40 SAY ".IMAGE..."
@ 13,48 SAY "_____"
@ 13,55 SAY ".PROCESSOR..."
@ 14,01 SAY "....."
@ 14,24 SAY "....."
@ 14,40 SAY "....."
@ 14,55 SAY "....."
@ 15,01 SAY "....."
@ 16,01 SAY "....."
@ 17,01 SAY "....."
@ 18,01 SAY "....."
@ 19,01 SAY "FOCHI MICHELE & AMATULLI DAVIDE"
@ 20,01 SAY "VERSIONE 1.0"
@ 21,01 SAY "....."
@ 22,01 SAY "....."
*
*   Fine del programma
*

```

File In Data Base Numero 3 (In Bianco E Nero)

```

*
*   Questo file è stato creato da T.I.P., scritto da Fochi Michele
*

```

```

@ 0,0 CLEAR

```

```

@ 00,00 SAY "+-----+"
@ 01,00 SAY "      '"
@ 02,00 SAY "      '"
@ 03,00 SAY ".....'"
@ 04,00 SAY ".....'"
@ 05,00 SAY ".....'"
@ 06,00 SAY ".....'"
@ 07,00 SAY ".....'"
@ 08,00 SAY ".....'"
@ 09,00 SAY ".....'"
@ 10,00 SAY ".....'"
@ 11,00 SAY ".....'"
@ 12,00 SAY ".....'"
@ 13,00 SAY ".....'TEXT...'IMAGE...'PROCESSOR..."
@ 14,00 SAY ".....'"
@ 15,00 SAY ".....'"
@ 16,00 SAY ".....'"

```

```

@ 17,00 SAY ""
@ 18,00 SAY ""
@ 19,00 SAY ""      FOCHI MICHELE & AMATULLI DAVIDE
@ 20,00 SAY ""      VERSIONE 1.0
@ 21,00 SAY ""
@ 22,00 SAY ""
@ 23,00 SAY "+-----+"
*
*      Fine del programma
*

```

GW-Basic

File In GW-Basic Numero 1 (Con Locate)

```

10 REM
11 REM Questo file è stato creato da T.I.P., scritto da Fochi Michele
12 REM
30 REM
40 REM Cancello lo schermo
50 COLOR 15,00,00
60 CLS
70 REM
80 REM Ora viene riempito lo schermo
90 REM
100 LOCATE 01,01
110 COLOR 14,00,00
120 PRINT "+-----+"
130 LOCATE 02,01
140 COLOR 14,00,00
150 PRINT ""
160 LOCATE 02,02
170 COLOR 15,00,00
180 PRINT "
190 LOCATE 02,80
200 COLOR 14,00,00
210 PRINT ""
220 LOCATE 03,01
230 COLOR 14,00,00
240 PRINT ""
250 LOCATE 03,02
260 COLOR 15,00,00
270 PRINT "
280 LOCATE 03,80
290 COLOR 14,00,00

```

```

300 PRINT ""
310 LOCATE 04,01
320 COLOR 14,00,00
330 PRINT ""
340 LOCATE 04,02
350 COLOR 15,00,00
360 PRINT " ..... "
370 LOCATE 04,80
380 COLOR 14,00,00
390 PRINT ""
400 LOCATE 05,01
410 COLOR 14,00,00
420 PRINT ""
430 LOCATE 05,02
440 COLOR 15,00,00
450 PRINT " ..... "
460 LOCATE 05,80
470 COLOR 14,00,00
480 PRINT ""
490 LOCATE 06,01
500 COLOR 14,00,00
510 PRINT ""
520 LOCATE 06,02
530 COLOR 15,00,00
540 PRINT " ..."
550 LOCATE 06,13
560 COLOR 13,00,00
570 PRINT "+-----+"
580 LOCATE 06,29
590 COLOR 15,00,00
600 PRINT "...."
610 LOCATE 06,33
620 COLOR 11,00,00
630 PRINT "+-----+"
640 LOCATE 06,41
650 COLOR 15,00,00
660 PRINT "....."
670 LOCATE 06,48
680 COLOR 12,00,00
690 PRINT "+-----+"
700 LOCATE 06,63
710 COLOR 15,00,00
720 PRINT "..... "
730 LOCATE 06,80
740 COLOR 14,00,00
750 PRINT ""
760 LOCATE 07,01

```

```

770 COLOR 14,00,00
780 PRINT ""
790 LOCATE 07,02
800 COLOR 15,00,00
810 PRINT "    ..."
820 LOCATE 07,13
830 COLOR 13,00,00
840 PRINT ""
850 LOCATE 07,14
860 COLOR 15,00,00
870 PRINT "_____ "
880 LOCATE 07,28
890 COLOR 13,00,00
900 PRINT ""
910 LOCATE 07,29
920 COLOR 15,00,00
930 PRINT "...."
940 LOCATE 07,33
950 COLOR 11,00,00
960 PRINT ""
970 LOCATE 07,34
980 COLOR 15,00,00
990 PRINT "_____ "
1000 LOCATE 07,40
1010 COLOR 11,00,00
1020 PRINT ""
1030 LOCATE 07,41
1040 COLOR 15,00,00
1050 PRINT "....."
1060 LOCATE 07,48
1070 COLOR 12,00,00
1080 PRINT ""
1090 LOCATE 07,49
1100 COLOR 15,00,00
1110 PRINT "_____ "
1120 LOCATE 07,62
1130 COLOR 12,00,00
1140 PRINT "++"
1150 LOCATE 07,64
1160 COLOR 15,00,00
1170 PRINT "..... "
1180 LOCATE 07,80
1190 COLOR 14,00,00
1200 PRINT ""
1210 LOCATE 08,01
1220 COLOR 14,00,00
1230 PRINT ""

```

1240 LOCATE 08,02
1250 COLOR 15,00,00
1260 PRINT " ..."
1270 LOCATE 08,13
1280 COLOR 13,00,00
1290 PRINT ""
1300 LOCATE 08,14
1310 COLOR 15,00,00
1320 PRINT " _ "
1330 LOCATE 08,16
1340 COLOR 13,00,00
1350 PRINT "+--+"
1360 LOCATE 08,20
1370 COLOR 15,00,00
1380 PRINT " _ "
1390 LOCATE 08,22
1400 COLOR 13,00,00
1410 PRINT "+--+"
1420 LOCATE 08,26
1430 COLOR 15,00,00
1440 PRINT " _ "
1450 LOCATE 08,28
1460 COLOR 13,00,00
1470 PRINT ""
1480 LOCATE 08,29
1490 COLOR 15,00,00
1500 PRINT "...."
1510 LOCATE 08,33
1520 COLOR 11,00,00
1530 PRINT "+-+"
1540 LOCATE 08,36
1550 COLOR 15,00,00
1560 PRINT " _ "
1570 LOCATE 08,38
1580 COLOR 11,00,00
1590 PRINT "+-+"
1600 LOCATE 08,41
1610 COLOR 15,00,00
1620 PRINT "....."
1630 LOCATE 08,48
1640 COLOR 12,00,00
1650 PRINT "+-+"
1660 LOCATE 08,51
1670 COLOR 15,00,00
1680 PRINT " _ +-----+ _ "
1690 LOCATE 08,63
1700 COLOR 12,00,00


```

1710 PRINT "++"
1720 LOCATE 08,65
1730 COLOR 15,00,00
1740 PRINT "...."
1750 LOCATE 08,80
1760 COLOR 14,00,00
1770 PRINT ""
1780 LOCATE 09,01
1790 COLOR 14,00,00
1800 PRINT ""
1810 LOCATE 09,02
1820 COLOR 15,00,00
1830 PRINT "... "
1840 LOCATE 09,13
1850 COLOR 13,00,00
1860 PRINT "+--+"
1870 LOCATE 09,17
1880 COLOR 15,00,00
1890 PRINT ".."
1900 LOCATE 09,19
1910 COLOR 13,00,00
1920 PRINT ""
1930 LOCATE 09,20
1940 COLOR 15,00,00
1950 PRINT " _ "
1960 LOCATE 09,22
1970 COLOR 13,00,00
1980 PRINT ""
1990 LOCATE 09,23
2000 COLOR 15,00,00
2010 PRINT ".."
2020 LOCATE 09,25
2030 COLOR 13,00,00
2040 PRINT "+--+"
2050 LOCATE 09,29
2060 COLOR 15,00,00
2070 PRINT "....."
2080 LOCATE 09,35
2090 COLOR 11,00,00
2100 PRINT ""
2110 LOCATE 09,36
2120 COLOR 15,00,00
2130 PRINT " _ "
2140 LOCATE 09,38
2150 COLOR 11,00,00
2160 PRINT ""
2170 LOCATE 09,39

```

```

2180 COLOR 15,00,00
2190 PRINT "....."
2200 LOCATE 09,50
2210 COLOR 12,00,00
2220 PRINT ""
2230 LOCATE 09,51
2240 COLOR 15,00,00
2250 PRINT " _'|.....'| _"
2260 LOCATE 09,64
2270 COLOR 12,00,00
2280 PRINT ""
2290 LOCATE 09,65
2300 COLOR 15,00,00
2310 PRINT ".... "
2320 LOCATE 09,80
2330 COLOR 14,00,00
2340 PRINT ""
2350 LOCATE 10,01
2360 COLOR 14,00,00
2370 PRINT ""
2380 LOCATE 10,02
2390 COLOR 15,00,00
2400 PRINT " ..... "
2410 LOCATE 10,19
2420 COLOR 13,00,00
2430 PRINT ""
2440 LOCATE 10,20
2450 COLOR 15,00,00
2460 PRINT " _ "
2470 LOCATE 10,22
2480 COLOR 13,00,00
2490 PRINT ""
2500 LOCATE 10,23
2510 COLOR 15,00,00
2520 PRINT "....."
2530 LOCATE 10,35
2540 COLOR 11,00,00
2550 PRINT ""
2560 LOCATE 10,36
2570 COLOR 15,00,00
2580 PRINT " _ "
2590 LOCATE 10,38
2600 COLOR 11,00,00
2610 PRINT ""
2620 LOCATE 10,39
2630 COLOR 15,00,00
2640 PRINT "....."

```

```

2650 LOCATE 10,50
2660 COLOR 12,00,00
2670 PRINT ""
2680 LOCATE 10,51
2690 COLOR 15,00,00
2700 PRINT " _+-----+ _"
2710 LOCATE 10,63
2720 COLOR 12,00,00
2730 PRINT "++"
2740 LOCATE 10,65
2750 COLOR 15,00,00
2760 PRINT "...."
2770 LOCATE 10,80
2780 COLOR 14,00,00
2790 PRINT ""
2800 LOCATE 11,01
2810 COLOR 14,00,00
2820 PRINT ""
2830 LOCATE 11,02
2840 COLOR 15,00,00
2850 PRINT " ....."
2860 LOCATE 11,19
2870 COLOR 13,00,00
2880 PRINT ""
2890 LOCATE 11,20
2900 COLOR 15,00,00
2910 PRINT " _"
2920 LOCATE 11,22
2930 COLOR 13,00,00
2940 PRINT ""
2950 LOCATE 11,23
2960 COLOR 15,00,00
2970 PRINT "....."
2980 LOCATE 11,35
2990 COLOR 11,00,00
3000 PRINT ""
3010 LOCATE 11,36
3020 COLOR 15,00,00
3030 PRINT " _"
3040 LOCATE 11,38
3050 COLOR 11,00,00
3060 PRINT ""
3070 LOCATE 11,39
3080 COLOR 15,00,00
3090 PRINT "....."
3100 LOCATE 11,50
3110 COLOR 12,00,00

```

```

3120 PRINT ""
3130 LOCATE 11,51
3140 COLOR 15,00,00
3150 PRINT "_____ "
3160 LOCATE 11,62
3170 COLOR 12,00,00
3180 PRINT "++"
3190 LOCATE 11,64
3200 COLOR 15,00,00
3210 PRINT "..... "
3220 LOCATE 11,80
3230 COLOR 14,00,00
3240 PRINT ""
3250 LOCATE 12,01
3260 COLOR 14,00,00
3270 PRINT ""
3280 LOCATE 12,02
3290 COLOR 15,00,00
3300 PRINT "....."
3310 LOCATE 12,19
3320 COLOR 13,00,00
3330 PRINT ""
3340 LOCATE 12,20
3350 COLOR 15,00,00
3360 PRINT " _ "
3370 LOCATE 12,22
3380 COLOR 13,00,00
3390 PRINT ""
3400 LOCATE 12,23
3410 COLOR 15,00,00
3420 PRINT "....."
3430 LOCATE 12,35
3440 COLOR 11,00,00
3450 PRINT ""
3460 LOCATE 12,36
3470 COLOR 15,00,00
3480 PRINT " _ "
3490 LOCATE 12,38
3500 COLOR 11,00,00
3510 PRINT ""
3520 LOCATE 12,39
3530 COLOR 15,00,00
3540 PRINT "....."
3550 LOCATE 12,50
3560 COLOR 12,00,00
3570 PRINT ""
3580 LOCATE 12,51

```

```

3590 COLOR 15,00,00
3600 PRINT "___"
3610 LOCATE 12,53
3620 COLOR 12,00,00
3630 PRINT "+-----+"
3640 LOCATE 12,63
3650 COLOR 15,00,00
3660 PRINT "....."
3670 LOCATE 12,80
3680 COLOR 14,00,00
3690 PRINT "||"
3700 LOCATE 13,01
3710 COLOR 14,00,00
3720 PRINT "||"
3730 LOCATE 13,02
3740 COLOR 15,00,00
3750 PRINT "....."
3760 LOCATE 13,17
3770 COLOR 13,00,00
3780 PRINT "+-+"
3790 LOCATE 13,20
3800 COLOR 15,00,00
3810 PRINT "___"
3820 LOCATE 13,22
3830 COLOR 13,00,00
3840 PRINT "+-+"
3850 LOCATE 13,25
3860 COLOR 15,00,00
3870 PRINT "....."
3880 LOCATE 13,33
3890 COLOR 11,00,00
3900 PRINT "+-+"
3910 LOCATE 13,36
3920 COLOR 15,00,00
3930 PRINT "___"
3940 LOCATE 13,38
3950 COLOR 11,00,00
3960 PRINT "+-+"
3970 LOCATE 13,41
3980 COLOR 15,00,00
3990 PRINT "....."
4000 LOCATE 13,48
4010 COLOR 12,00,00
4020 PRINT "+-+"
4030 LOCATE 13,51
4040 COLOR 15,00,00
4050 PRINT "___"

```

```

4060 LOCATE 13,53
4070 COLOR 12,00,00
4080 PRINT "+-+"
4090 LOCATE 13,56
4100 COLOR 15,00,00
4110 PRINT "....."      "
4120 LOCATE 13,80
4130 COLOR 14,00,00
4140 PRINT ""
4150 LOCATE 14,01
4160 COLOR 14,00,00
4170 PRINT ""
4180 LOCATE 14,02
4190 COLOR 15,00,00
4200 PRINT "      ...."
4210 LOCATE 14,17
4220 COLOR 13,00,00
4230 PRINT ""
4240 LOCATE 14,18
4250 COLOR 15,00,00
4260 PRINT "_____"
4270 LOCATE 14,24
4280 COLOR 13,00,00
4290 PRINT ""
4300 LOCATE 14,25
4310 COLOR 15,00,00
4320 PRINT "·TEXT·..."
4330 LOCATE 14,33
4340 COLOR 11,00,00
4350 PRINT ""
4360 LOCATE 14,34
4370 COLOR 15,00,00
4380 PRINT "_____"
4390 LOCATE 14,40
4400 COLOR 11,00,00
4410 PRINT ""
4420 LOCATE 14,41
4430 COLOR 15,00,00
4440 PRINT "·IMAGE·"
4450 LOCATE 14,48
4460 COLOR 12,00,00
4470 PRINT ""
4480 LOCATE 14,49
4490 COLOR 15,00,00
4500 PRINT "_____"
4510 LOCATE 14,55
4520 COLOR 12,00,00

```

```

4530 PRINT ""
4540 LOCATE 14,56
4550 COLOR 15,00,00
4560 PRINT "·PROCESSOR·"
4570 LOCATE 14,80
4580 COLOR 14,00,00
4590 PRINT ""
4600 LOCATE 15,01
4610 COLOR 14,00,00
4620 PRINT ""
4630 LOCATE 15,02
4640 COLOR 15,00,00
4650 PRINT "      ....."
4660 LOCATE 15,17
4670 COLOR 13,00,00
4680 PRINT "+-----+"
4690 LOCATE 15,25
4700 COLOR 15,00,00
4710 PRINT "....."
4720 LOCATE 15,33
4730 COLOR 11,00,00
4740 PRINT "+-----+"
4750 LOCATE 15,41
4760 COLOR 15,00,00
4770 PRINT "....."
4780 LOCATE 15,48
4790 COLOR 12,00,00
4800 PRINT "+-----+"
4810 LOCATE 15,56
4820 COLOR 15,00,00
4830 PRINT "....."
4840 LOCATE 15,80
4850 COLOR 14,00,00
4860 PRINT ""
4870 LOCATE 16,01
4880 COLOR 14,00,00
4890 PRINT ""
4900 LOCATE 16,02
4910 COLOR 15,00,00
4920 PRINT "      ..... "
4930 LOCATE 16,80
4940 COLOR 14,00,00
4950 PRINT ""
4960 LOCATE 17,01
4970 COLOR 14,00,00
4980 PRINT ""
4990 LOCATE 17,02

```

5000 COLOR 15,00,00		
5010 PRINT "		"
5020 LOCATE 17,80		
5030 COLOR 14,00,00		
5040 PRINT ""		
5050 LOCATE 18,01		
5060 COLOR 14,00,00		
5070 PRINT ""		
5080 LOCATE 18,02		
5090 COLOR 15,00,00		
5100 PRINT "	"	
5110 LOCATE 18,80		
5120 COLOR 14,00,00		
5130 PRINT ""		
5140 LOCATE 19,01		
5150 COLOR 14,00,00		
5160 PRINT ""		
5170 LOCATE 19,02		
5180 COLOR 15,00,00		
5190 PRINT "	"	
5200 LOCATE 19,80		
5210 COLOR 14,00,00		
5220 PRINT ""		
5230 LOCATE 20,01		
5240 COLOR 14,00,00		
5250 PRINT ""		
5260 LOCATE 20,02		
5270 COLOR 15,00,00		
5280 PRINT " FOCHI MICHELE & AMATULLI DAVIDE		"
5290 LOCATE 20,80		
5300 COLOR 14,00,00		
5310 PRINT ""		
5320 LOCATE 21,01		
5330 COLOR 14,00,00		
5340 PRINT ""		
5350 LOCATE 21,02		
5360 COLOR 15,00,00		
5370 PRINT " VERSIONE 1.0	"	
5380 LOCATE 21,80		
5390 COLOR 14,00,00		
5400 PRINT ""		
5410 LOCATE 22,01		
5420 COLOR 14,00,00		
5430 PRINT ""		
5440 LOCATE 22,02		
5450 COLOR 15,00,00		
5460 PRINT "	"	


```

5470 LOCATE 22,80
5480 COLOR 14,00,00
5490 PRINT ""
5500 LOCATE 23,01
5510 COLOR 14,00,00
5520 PRINT ""
5530 LOCATE 23,02
5540 COLOR 15,00,00
5550 PRINT "
5560 LOCATE 23,80
5570 COLOR 14,00,00
5580 PRINT ""
5590 LOCATE 24,01
5600 COLOR 14,00,00
5610 PRINT "+-----+"
5620 REM
5630 REM Fine del programma
5640 REM

```

File In GW-Basic Numero 2 (Senza Locate)

```

10 REM
11 REM Questo file è stato creato da T.I.P., scritto da Fochi Michele
12 REM
30 REM
40 REM Cancello lo schermo
50 COLOR 15,00,00
60 CLS
70 REM
80 REM Ora viene riempito lo schermo
90 REM
100 COLOR 14,00,00
110 PRINT "+-----+";
120 COLOR 14,00,00
130 PRINT "|";
140 COLOR 15,00,00
150 PRINT "
160 COLOR 14,00,00
170 PRINT "|";
180 COLOR 14,00,00
190 PRINT "|";
200 COLOR 15,00,00
210 PRINT "
220 COLOR 14,00,00
230 PRINT "|";
240 COLOR 14,00,00
250 PRINT "|";

```

```

260 COLOR 15,00,00
270 PRINT " ..... ";
280 COLOR 14,00,00
290 PRINT " ";
300 COLOR 14,00,00
310 PRINT " ";
320 COLOR 15,00,00
330 PRINT " ..... ";
340 COLOR 14,00,00
350 PRINT " ";
360 COLOR 14,00,00
370 PRINT " ";
380 COLOR 15,00,00
390 PRINT " ...";
400 COLOR 13,00,00
410 PRINT "+-----+";
420 COLOR 15,00,00
430 PRINT "....";
440 COLOR 11,00,00
450 PRINT "+-----+";
460 COLOR 15,00,00
470 PRINT ".....";
480 COLOR 12,00,00
490 PRINT "+-----+";
500 COLOR 15,00,00
510 PRINT "..... ";
520 COLOR 14,00,00
530 PRINT " ";
540 COLOR 14,00,00
550 PRINT " ";
560 COLOR 15,00,00
570 PRINT " ...";
580 COLOR 13,00,00
590 PRINT " ";
600 COLOR 15,00,00
610 PRINT " _____ ";
620 COLOR 13,00,00
630 PRINT " ";
640 COLOR 15,00,00
650 PRINT "....";
660 COLOR 11,00,00
670 PRINT " ";
680 COLOR 15,00,00
690 PRINT " _____ ";
700 COLOR 11,00,00
710 PRINT " ";
720 COLOR 15,00,00

```

```

730 PRINT ".....";
740 COLOR 12,00,00
750 PRINT "'";
760 COLOR 15,00,00
770 PRINT "_____";
780 COLOR 12,00,00
790 PRINT "++";
800 COLOR 15,00,00
810 PRINT ".....";
820 COLOR 14,00,00
830 PRINT "'";
840 COLOR 14,00,00
850 PRINT "'";
860 COLOR 15,00,00
870 PRINT "...";
880 COLOR 13,00,00
890 PRINT "'";
900 COLOR 15,00,00
910 PRINT "___";
920 COLOR 13,00,00
930 PRINT "+--+";
940 COLOR 15,00,00
950 PRINT "___";
960 COLOR 13,00,00
970 PRINT "+--+";
980 COLOR 15,00,00
990 PRINT "___";
1000 COLOR 13,00,00
1010 PRINT "'";
1020 COLOR 15,00,00
1030 PRINT ".....";
1040 COLOR 11,00,00
1050 PRINT "+-+";
1060 COLOR 15,00,00
1070 PRINT "___";
1080 COLOR 11,00,00
1090 PRINT "+-+";
1100 COLOR 15,00,00
1110 PRINT ".....";
1120 COLOR 12,00,00
1130 PRINT "+-+";
1140 COLOR 15,00,00
1150 PRINT "___+-----+___";
1160 COLOR 12,00,00
1170 PRINT "++";
1180 COLOR 15,00,00
1190 PRINT ".....";

```

```

1200 COLOR 14,00,00
1210 PRINT "''";
1220 COLOR 14,00,00
1230 PRINT "''";
1240 COLOR 15,00,00
1250 PRINT "    ...";
1260 COLOR 13,00,00
1270 PRINT "+--+";
1280 COLOR 15,00,00
1290 PRINT "...";
1300 COLOR 13,00,00
1310 PRINT "''";
1320 COLOR 15,00,00
1330 PRINT "___";
1340 COLOR 13,00,00
1350 PRINT "''";
1360 COLOR 15,00,00
1370 PRINT "...";
1380 COLOR 13,00,00
1390 PRINT "+--+";
1400 COLOR 15,00,00
1410 PRINT ".....";
1420 COLOR 11,00,00
1430 PRINT "''";
1440 COLOR 15,00,00
1450 PRINT "___";
1460 COLOR 11,00,00
1470 PRINT "''";
1480 COLOR 15,00,00
1490 PRINT ".....";
1500 COLOR 12,00,00
1510 PRINT "''";
1520 COLOR 15,00,00
1530 PRINT "___!.....!___";
1540 COLOR 12,00,00
1550 PRINT "''";
1560 COLOR 15,00,00
1570 PRINT "....  ";
1580 COLOR 14,00,00
1590 PRINT "''";
1600 COLOR 14,00,00
1610 PRINT "''";
1620 COLOR 15,00,00
1630 PRINT "    .....";
1640 COLOR 13,00,00
1650 PRINT "''";
1660 COLOR 15,00,00

```

```

1670 PRINT "___";
1680 COLOR 13,00,00
1690 PRINT "|";
1700 COLOR 15,00,00
1710 PRINT ".....";
1720 COLOR 11,00,00
1730 PRINT "|";
1740 COLOR 15,00,00
1750 PRINT "___";
1760 COLOR 11,00,00
1770 PRINT "|";
1780 COLOR 15,00,00
1790 PRINT ".....";
1800 COLOR 12,00,00
1810 PRINT "|";
1820 COLOR 15,00,00
1830 PRINT "___+-----+___";
1840 COLOR 12,00,00
1850 PRINT "+ +";
1860 COLOR 15,00,00
1870 PRINT "....    ";
1880 COLOR 14,00,00
1890 PRINT "|";
1900 COLOR 14,00,00
1910 PRINT "|";
1920 COLOR 15,00,00
1930 PRINT "    .....";
1940 COLOR 13,00,00
1950 PRINT "|";
1960 COLOR 15,00,00
1970 PRINT "___";
1980 COLOR 13,00,00
1990 PRINT "|";
2000 COLOR 15,00,00
2010 PRINT ".....";
2020 COLOR 11,00,00
2030 PRINT "|";
2040 COLOR 15,00,00
2050 PRINT "___";
2060 COLOR 11,00,00
2070 PRINT "|";
2080 COLOR 15,00,00
2090 PRINT ".....";
2100 COLOR 12,00,00
2110 PRINT "|";
2120 COLOR 15,00,00
2130 PRINT "_____";

```

```

2140 COLOR 12,00,00
2150 PRINT "+-";
2160 COLOR 15,00,00
2170 PRINT ".....";
2180 COLOR 14,00,00
2190 PRINT "|";
2200 COLOR 14,00,00
2210 PRINT "|";
2220 COLOR 15,00,00
2230 PRINT " .....";
2240 COLOR 13,00,00
2250 PRINT "|";
2260 COLOR 15,00,00
2270 PRINT " _";
2280 COLOR 13,00,00
2290 PRINT "|";
2300 COLOR 15,00,00
2310 PRINT ".....";
2320 COLOR 11,00,00
2330 PRINT "|";
2340 COLOR 15,00,00
2350 PRINT " _";
2360 COLOR 11,00,00
2370 PRINT "|";
2380 COLOR 15,00,00
2390 PRINT ".....";
2400 COLOR 12,00,00
2410 PRINT "|";
2420 COLOR 15,00,00
2430 PRINT " _";
2440 COLOR 12,00,00
2450 PRINT "+-----+";
2460 COLOR 15,00,00
2470 PRINT ".....";
2480 COLOR 14,00,00
2490 PRINT "|";
2500 COLOR 14,00,00
2510 PRINT "|";
2520 COLOR 15,00,00
2530 PRINT " .....";
2540 COLOR 13,00,00
2550 PRINT "+-+";
2560 COLOR 15,00,00
2570 PRINT " _";
2580 COLOR 13,00,00
2590 PRINT "+-+";
2600 COLOR 15,00,00

```

```

2610 PRINT ".....";
2620 COLOR 11,00,00
2630 PRINT "+-+";
2640 COLOR 15,00,00
2650 PRINT "___";
2660 COLOR 11,00,00
2670 PRINT "+-+";
2680 COLOR 15,00,00
2690 PRINT ".....";
2700 COLOR 12,00,00
2710 PRINT "+-+";
2720 COLOR 15,00,00
2730 PRINT "___";
2740 COLOR 12,00,00
2750 PRINT "+-+";
2760 COLOR 15,00,00
2770 PRINT ".....";
2780 COLOR 14,00,00
2790 PRINT "|";
2800 COLOR 14,00,00
2810 PRINT "|";
2820 COLOR 15,00,00
2830 PRINT ".....";
2840 COLOR 13,00,00
2850 PRINT "|";
2860 COLOR 15,00,00
2870 PRINT "_____";
2880 COLOR 13,00,00
2890 PRINT "|";
2900 COLOR 15,00,00
2910 PRINT "·TEXT·";
2920 COLOR 11,00,00
2930 PRINT "|";
2940 COLOR 15,00,00
2950 PRINT "_____";
2960 COLOR 11,00,00
2970 PRINT "|";
2980 COLOR 15,00,00
2990 PRINT "·IMAGE·";
3000 COLOR 12,00,00
3010 PRINT "|";
3020 COLOR 15,00,00
3030 PRINT "_____";
3040 COLOR 12,00,00
3050 PRINT "|";
3060 COLOR 15,00,00
3070 PRINT "·PROCESSOR·";

```

```

3080 COLOR 14,00,00
3090 PRINT "|";
3100 COLOR 14,00,00
3110 PRINT "|";
3120 COLOR 15,00,00
3130 PRINT " .....";
3140 COLOR 13,00,00
3150 PRINT "+-----+";
3160 COLOR 15,00,00
3170 PRINT ".....";
3180 COLOR 11,00,00
3190 PRINT "+-----+";
3200 COLOR 15,00,00
3210 PRINT ".....";
3220 COLOR 12,00,00
3230 PRINT "+-----+";
3240 COLOR 15,00,00
3250 PRINT ".....";
3260 COLOR 14,00,00
3270 PRINT "|";
3280 COLOR 14,00,00
3290 PRINT "|";
3300 COLOR 15,00,00
3310 PRINT " .....";
3320 COLOR 14,00,00
3330 PRINT "|";
3340 COLOR 14,00,00
3350 PRINT "|";
3360 COLOR 15,00,00
3370 PRINT " .....";
3380 COLOR 14,00,00
3390 PRINT "|";
3400 COLOR 14,00,00
3410 PRINT "|";
3420 COLOR 15,00,00
3430 PRINT " ";
3440 COLOR 14,00,00
3450 PRINT "|";
3460 COLOR 14,00,00
3470 PRINT "|";
3480 COLOR 15,00,00
3490 PRINT " ";
3500 COLOR 14,00,00
3510 PRINT "|";
3520 COLOR 14,00,00
3530 PRINT "|";
3540 COLOR 15,00,00

```



```

3550 PRINT "      FOCHI MICHELE & AMATULLI DAVIDE      ";
3560 COLOR 14,00,00
3570 PRINT """;
3580 COLOR 14,00,00
3590 PRINT """;
3600 COLOR 15,00,00
3610 PRINT "      VERSIONE 1.0      ";
3620 COLOR 14,00,00
3630 PRINT """;
3640 COLOR 14,00,00
3650 PRINT """;
3660 COLOR 15,00,00
3670 PRINT "      ";
3680 COLOR 14,00,00
3690 PRINT """;
3700 COLOR 14,00,00
3710 PRINT """;
3720 COLOR 15,00,00
3730 PRINT "      ";
3740 COLOR 14,00,00
3750 PRINT """;
3760 COLOR 14,00,00
3770 PRINT "+-----+";
3780 REM
3790 REM Fine del programma
3800 REM

```

Sequenze Escape ANSI

File In Sequenze Escape ANSI Numero 1 (Con Posizionamento)

```

[1;1H[0;1;33;40m+-----
+[2;1H[0;1;33;40m|[2;2H[0;1;37;40m
[2;80H[0;1;33;40m|[3;1H[0;1;33;40m|[3;2H[0;1;37;40m
[3;80H[0;1;33;40m|[4;1H[0;1;33;40m|[4;2H[0;1;37;40m
.....
[4;80H[0;1;33;40m|[5;1H[0;1;33;40m|[5;2H[0;1;37;40m
.....
[5;80H[0;1;33;40m|[6;1H[0;1;33;40m|[6;2H[0;1;37;40m      ...[6;13H[0;1;35;40m+-----
+[6;29H[0;1;37;40m...[6;33H[0;1;36;40m+-----
+[6;41H[0;1;37;40m...[6;48H[0;1;31;40m+-----+[6;63H[0;1;37;40m...
[6;80H[0;1;33;40m|[7;1H[0;1;33;40m|[7;2H[0;1;37;40m
...[7;13H[0;1;35;40m|[7;14H[0;1;37;40m_____ [7;28H[0;1;35;40m|[7;29H[0;1;37;40m

```

m···[7;33H[0;1;36;40m|[7;34H[0;1;37;40m_____[7;40H[0;1;36;40m|[7;41H[0;1;37;40m·····
[7;48H[0;1;31;40m|[7;49H[0;1;37;40m_____ [7;62H[0;1;31;40m+
+[7;64H[0;1;37;40m····· [7;80H[0;1;33;40m|[8;1H[0;1;33;40m|[8;2H[0;1;37;40m
··[8;13H[0;1;35;40m|[8;14H[0;1;37;40m__ [8;16H[0;1;35;40m+--
+[8;20H[0;1;37;40m__ [8;22H[0;1;35;40m+--
+[8;26H[0;1;37;40m__ [8;28H[0;1;35;40m|[8;29H[0;1;37;40m····[8;33H[0;1;36;40m+-
+[8;36H[0;1;37;40m__ [8;38H[0;1;36;40m+--+ [8;41H[0;1;37;40m·····[8;48H[0;1;31;40m+-
+[8;51H[0;1;37;40m__ +-----+ __ [8;63H[0;1;31;40m++ [8;65H[0;1;37;40m····
[8;80H[0;1;33;40m|[9;1H[0;1;33;40m|[9;2H[0;1;37;40m ···[9;13H[0;1;35;40m+--
+[9;17H[0;1;37;40m·· [9;19H[0;1;35;40m|[9;20H[0;1;37;40m__ [9;22H[0;1;35;40m|[9;23H[0;1;3
7;40m·· [9;25H[0;1;35;40m+--
+[9;29H[0;1;37;40m····· [9;35H[0;1;36;40m|[9;36H[0;1;37;40m__ [9;38H[0;1;36;40m|[9;39H[0;
1;37;40m········ [9;50H[0;1;31;40m|[9;51H[0;1;37;40m__ '·····'
__ [9;64H[0;1;31;40m|[9;65H[0;1;37;40m····
[9;80H[0;1;33;40m|[10;1H[0;1;33;40m|[10;2H[0;1;37;40m
······ [10;19H[0;1;35;40m|[10;20H[0;1;37;40m__ [10;22H[0;1;35;40m|[10;23H[0;1;37;40m····
····· [10;35H[0;1;36;40m|[10;36H[0;1;37;40m__ [10;38H[0;1;36;40m|[10;39H[0;1;37;40m·····
··· [10;50H[0;1;31;40m|[10;51H[0;1;37;40m__ +-----+ __ [10;63H[0;1;31;40m+
+[10;65H[0;1;37;40m···· [10;80H[0;1;33;40m|[11;1H[0;1;33;40m|[11;2H[0;1;37;40m
······ [11;19H[0;1;35;40m|[11;20H[0;1;37;40m__ [11;22H[0;1;35;40m|[11;23H[0;1;37;40m····
····· [11;35H[0;1;36;40m|[11;36H[0;1;37;40m__ [11;38H[0;1;36;40m|[11;39H[0;1;37;40m·····
··· [11;50H[0;1;31;40m|[11;51H[0;1;37;40m_____ [11;62H[0;1;31;40m+
+[11;64H[0;1;37;40m···· [11;80H[0;1;33;40m|[12;1H[0;1;33;40m|[12;2H[0;1;37;40m
······ [12;19H[0;1;35;40m|[12;20H[0;1;37;40m__ [12;22H[0;1;35;40m|[12;23H[0;1;37;40m····
····· [12;35H[0;1;36;40m|[12;36H[0;1;37;40m__ [12;38H[0;1;36;40m|[12;39H[0;1;37;40m·····
··· [12;50H[0;1;31;40m|[12;51H[0;1;37;40m__ [12;53H[0;1;31;40m+-----
+[12;63H[0;1;37;40m···· [12;80H[0;1;33;40m|[13;1H[0;1;33;40m|[13;2H[0;1;37;40m
····· [13;17H[0;1;35;40m+--+ [13;20H[0;1;37;40m__ [13;22H[0;1;35;40m+-
+[13;25H[0;1;37;40m······ [13;33H[0;1;36;40m+-
+[13;36H[0;1;37;40m__ [13;38H[0;1;36;40m+-
+[13;41H[0;1;37;40m······ [13;48H[0;1;31;40m+-
+[13;51H[0;1;37;40m__ [13;53H[0;1;31;40m+--+ [13;56H[0;1;37;40m········
[13;80H[0;1;33;40m|[14;1H[0;1;33;40m|[14;2H[0;1;37;40m
····· [14;17H[0;1;35;40m|[14;18H[0;1;37;40m_____ [14;24H[0;1;35;40m|[14;25H[0;1;37;40m·
TEXT·· [14;33H[0;1;36;40m|[14;34H[0;1;37;40m_____ [14;40H[0;1;36;40m|[14;41H[0;1;37;4
0m·IMAGE· [14;48H[0;1;31;40m|[14;49H[0;1;37;40m_____ [14;55H[0;1;31;40m|[14;56H[0;1;3
7;40m·PROCESSOR·· [14;80H[0;1;33;40m|[15;1H[0;1;33;40m|[15;2H[0;1;37;40m
····· [15;17H[0;1;35;40m+-----+ [15;25H[0;1;37;40m······ [15;33H[0;1;36;40m+-----
+[15;41H[0;1;37;40m······ [15;48H[0;1;31;40m+-----+ [15;56H[0;1;37;40m········
[15;80H[0;1;33;40m|[16;1H[0;1;33;40m|[16;2H[0;1;37;40m
······························
[16;80H[0;1;33;40m|[17;1H[0;1;33;40m|[17;2H[0;1;37;40m
······························
[17;80H[0;1;33;40m|[18;1H[0;1;33;40m|[18;2H[0;1;37;40m
[18;80H[0;1;33;40m|[19;1H[0;1;33;40m|[19;2H[0;1;37;40m
[19;80H[0;1;33;40m|[20;1H[0;1;33;40m|[20;2H[0;1;37;40m

FOCHI MICHELE &
DAVIDE

[20;80H[0;1;33;40m'[21;1H[0;1;33;40m'[21;2H[0;1;37;40m
[21;80H[0;1;33;40m'[22;1H[0;1;33;40m'[22;2H[0;1;37;40m
[22;80H[0;1;33;40m'[23;1H[0;1;33;40m'[23;2H[0;1;37;40m
[23;80H[0;1;33;40m'[24;1H[0;1;33;40m+-----
-----+[23;1H[0m

VERSIONE 1.0

File In Sequenze Escape ANSI Numero 2 (Senza Posizionamento)

[0;1;33;40m+-----
+[0;1;33;40m'[0;1;37;40m
[0;1;33;40m'[0;1;33;40m'[0;1;37;40m
[0;1;33;40m'[0;1;33;40m'[0;1;37;40m
[0;1;33;40m'[0;1;33;40m'[0;1;37;40m
[0;1;33;40m'[0;1;33;40m'[0;1;37;40m[0;1;35;40m+-----
+[0;1;37;40m····[0;1;36;40m+-----+[0;1;37;40m····[0;1;31;40m+-----
+[0;1;37;40m···· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m
··[0;1;35;40m'[0;1;37;40m [0;1;35;40m'[0;1;37;40m··[0;1;36;40m'[0;1;37;40m
m [0;1;36;40m'[0;1;37;40m····[0;1;31;40m'[0;1;37;40m [0;1;31;40m+
+[0;1;37;40m···· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m
··[0;1;35;40m'[0;1;37;40m [0;1;35;40m+--+ [0;1;37;40m [0;1;35;40m+--
+[0;1;37;40m [0;1;35;40m'[0;1;37;40m····[0;1;36;40m+--+ [0;1;37;40m [0;1;36;40m+--
+[0;1;37;40m····[0;1;31;40m+--+ [0;1;37;40m [0;1;31;40m+--+ [0;1;37;40m····
[0;1;33;40m'[0;1;33;40m'[0;1;37;40m[0;1;35;40m+--
+[0;1;37;40m··[0;1;35;40m'[0;1;37;40m [0;1;35;40m'[0;1;37;40m··[0;1;35;40m+--
+[0;1;37;40m····[0;1;36;40m'[0;1;37;40m [0;1;36;40m'[0;1;37;40m····[0;1;31;40m'[0;1;
;37;40m [0;1;31;40m'[0;1;37;40m···· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m
····[0;1;35;40m'[0;1;37;40m [0;1;35;40m'[0;1;37;40m····[0;1;36;40m'[0;1;37;40m
 [0;1;36;40m'[0;1;37;40m····[0;1;31;40m'[0;1;37;40m +-----+ [0;1;31;40m+
+[0;1;37;40m···· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m
····[0;1;35;40m'[0;1;37;40m [0;1;35;40m'[0;1;37;40m····[0;1;36;40m'[0;1;37;40m
 [0;1;36;40m'[0;1;37;40m····[0;1;31;40m'[0;1;37;40m [0;1;31;40m+
+[0;1;37;40m···· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m
····[0;1;35;40m'[0;1;37;40m [0;1;35;40m'[0;1;37;40m····[0;1;36;40m'[0;1;37;40m
 [0;1;36;40m'[0;1;37;40m····[0;1;31;40m'[0;1;37;40m [0;1;31;40m+-----
+[0;1;37;40m···· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m[0;1;35;40m+--
+[0;1;37;40m [0;1;35;40m+--+ [0;1;37;40m····[0;1;36;40m+--+ [0;1;37;40m [0;1;36;40m+--
+[0;1;37;40m····[0;1;31;40m+--+ [0;1;37;40m [0;1;31;40m+--+ [0;1;37;40m····
[0;1;33;40m'[0;1;33;40m'[0;1;37;40m
····[0;1;35;40m'[0;1;37;40m [0;1;35;40m'[0;1;37;40m·TEXT··[0;1;36;40m'[0;1;37;40m
m [0;1;36;40m'[0;1;37;40m·IMAGE·[0;1;31;40m'[0;1;37;40m [0;1;31;40m'[0;1;37;
;40m·PROCESSOR·· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m[0;1;35;40m+-----
+[0;1;37;40m····[0;1;36;40m+-----+[0;1;37;40m····[0;1;31;40m+-----
+[0;1;37;40m···· [0;1;33;40m'[0;1;33;40m'[0;1;37;40m
..... [0;1;33;40m'[0;1;33;40m'[0;1;37;40m
..... [0;1;33;40m'[0;1;33;40m'[0;1;37;40m

```
[0;1;33;40m|[0;1;33;40m|[0;1;37;40m
[0;1;33;40m|[0;1;33;40m|[0;1;37;40m
[0;1;33;40m|[0;1;33;40m|[0;1;37;40m
[0;1;33;40m|[0;1;33;40m|[0;1;37;40m
[0;1;33;40m|[0;1;33;40m|[0;1;37;40m
[0;1;33;40m|[0;1;33;40m+-----
+[23;1H[0m
```

FOCHI MICHELE & AMATULLI DAVIDE
VERSIONE 1.0

Le conversioni sono tutte corrette e i programmi non danno errori in compilazione/esecuzione.

G.I.P.

Verrà preso come modello il seguente disegno:

File In Turbo Pascal

```
{=====
}
{
{      QUESTO FILE E' STATO CREATO DAL PROGRAMMA G.I.P.      }
{
{      G.I.P è uno dei 3 programmi della tesina per l' anno scolastico 1992/93      }
{      del gruppo Fochi Michele e Amatulli Davide.      }
{
{      I programmi sono stati scritti da Fochi Michele in Turbo Pascal.      }
{
{=====
}
```

Program Immagine;

Uses Crt, Graph;

```
Var   Gd:      Integer;
      Gm:      Integer;
      Poly:    Array [1..100] Of PointType;
      ArcC:    ArcCoordsType;
      FillPatt: FillPatternType;
      I:       Byte;
```

```
Begin {* MAIN *}
Gd := Detect;
Gm := Detect;
InitGraph(Gd,Gm,");
```

For i := Black To White Do

SetPalette(i,i);

SetRGBPalette(Black, 000,000,000);
SetRGBPalette(Blue, 000,000,042);
SetRGBPalette(Green, 000,042,000);
SetRGBPalette(Cyan, 000,042,042);
SetRGBPalette(Red, 042,000,000);
SetRGBPalette(Magenta, 042,000,042);
SetRGBPalette(Brown, 042,021,000);
SetRGBPalette(LightGray, 042,042,042);
SetRGBPalette(DarkGray, 021,021,021);
SetRGBPalette(LightBlue, 021,021,063);
SetRGBPalette(LightGreen, 021,063,021);
SetRGBPalette(LightCyan, 021,063,063);
SetRGBPalette(LightRed, 063,021,021);
SetRGBPalette(LightMagenta, 063,021,063);
SetRGBPalette(Yellow, 063,063,021);
SetRGBPalette(White, 063,063,063);

SetBkColor(Black);
SetFillStyle(SolidFill,Cyan);
Bar(0,0,GetMaxX,GetMaxY);
SetLineStyle(SolidLn,0,NormWidth);
SetColor(White);
SetFillStyle(SolidFill,Black);
Bar(73,106,364,312);
Rectangle(73,106,364,312);
Poly[1].X := 73;
Poly[1].Y := 106;
Poly[2].X := 115;
Poly[2].Y := 72;
Poly[3].X := 406;
Poly[3].Y := 72;
Poly[4].X := 364;
Poly[4].Y := 106;
FillPoly(4,Poly);
Poly[1].X := 406;
Poly[1].Y := 72;
Poly[2].X := 406;
Poly[2].Y := 278;
Poly[3].X := 364;
Poly[3].Y := 312;
FillPoly(4,Poly);
SetFillStyle(SolidFill,LightGray);
Poly[1].X := 117;

```

Poly[1].Y := 289;
Poly[2].X := 204;
Poly[2].Y := 289;
Poly[3].X := 204;
Poly[3].Y := 201;
Poly[4].X := 182;
Poly[4].Y := 201;
Poly[5].X := 118;
Poly[5].Y := 269;
Poly[6].X := 118;
Poly[6].Y := 288;
Poly[7].X := 118;
Poly[7].Y := 288;
Poly[8].X := 118;
Poly[8].Y := 288;
Poly[9].X := 118;
Poly[9].Y := 288;
FillPoly(9,Poly);
SetTextStyle(DefaultFont,HorizDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(126,193,'< FWD <');
SetTextStyle(DefaultFont,HorizDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(128,206,'> REVE >');
Bar(73,119,364,165);
Rectangle(73,119,364,165);
SetFillStyle(SolidFill,Red);
FloodFill(211,143,15);
Poly[1].X := 364;
Poly[1].Y := 118;
Poly[2].X := 372;
Poly[2].Y := 113;
Poly[3].X := 372;
Poly[3].Y := 136;
Poly[4].X := 379;
Poly[4].Y := 140;
Poly[5].X := 379;
Poly[5].Y := 298;
Poly[6].X := 364;
Poly[6].Y := 311;
Poly[7].X := 364;
Poly[7].Y := 120;
Poly[8].X := 364;
Poly[8].Y := 120;
FillPoly(8,Poly);
Bar(73,313,364,298);
Rectangle(73,313,364,298);

```

```

SetTextStyle(DefaultFont,HorizDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(167,156,'STEREO CASSETTE PLAYER');
SetTextStyle(TriplexFont,HorizDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(136,137,'auto reverse');
Line(196,144,358,144);
Line(196,139,358,139);
SetTextStyle(SmallFont,VertDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(222,244,'GRAPHIC EQUALIZER');
SetFillStyle(SolidFill,Green);
Bar(239,263,299,274);
Rectangle(239,263,299,274);
Bar(238,222,300,233);
Rectangle(238,222,300,233);
Bar(238,240,299,253);
Rectangle(238,240,299,253);
Bar(237,260,299,274);
Rectangle(237,260,299,274);
SetTextStyle(DefaultFont,HorizDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(328,227,'120Hz');
SetTextStyle(DefaultFont,HorizDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(326,247,'1KHz');
SetTextStyle(DefaultFont,HorizDir,1);
SetTextJustify(CenterText,CenterText);
OutTextXY(326,268,'8KHz');
Ellipse(151,237,220,46,43,46);
Ellipse(150,238,221,50,28,28);
Ellipse(150,238,218,56,16,16);
SetFillStyle(SolidFill,Black);
FloodFill(159,272,15);
FloodFill(153,239,15);
SetFillStyle(SolidFill,White);
FloodFill(154,258,15);
FillEllipse(393,149,5,7);
SetColor(Black);
SetFillStyle(SolidFill,LightGray);
FloodFill(395,150,0);
SetColor(White);
SetFillStyle(SolidFill,Black);
FillEllipse(393,149,2,4);
SetFillStyle(SolidFill,LightGray);
Poly[1].X := 388;
Poly[1].Y := 221;

```

```

Poly[2].X := 388;
Poly[2].Y := 252;
Poly[3].X := 394;
Poly[3].Y := 246;
Poly[4].X := 394;
Poly[4].Y := 214;
Poly[5].X := 389;
Poly[5].Y := 221;
Poly[6].X := 389;
Poly[6].Y := 221;
Poly[7].X := 389;
Poly[7].Y := 221;
FillPoly(7,Poly);
Bar(323,65,362,84);
Rectangle(323,65,362,84);
Poly[1].X := 323;
Poly[1].Y := 65;
Poly[2].X := 330;
Poly[2].Y := 60;
Poly[3].X := 369;
Poly[3].Y := 60;
Poly[4].X := 362;
Poly[4].Y := 65;
FillPoly(4,Poly);
Poly[1].X := 369;
Poly[1].Y := 60;
Poly[2].X := 369;
Poly[2].Y := 79;
Poly[3].X := 362;
Poly[3].Y := 84;
FillPoly(4,Poly);
Bar(306,75,347,95);
Rectangle(306,75,347,95);
Poly[1].X := 306;
Poly[1].Y := 75;
Poly[2].X := 313;
Poly[2].Y := 70;
Poly[3].X := 354;
Poly[3].Y := 70;
Poly[4].X := 347;
Poly[4].Y := 75;
FillPoly(4,Poly);
Poly[1].X := 354;
Poly[1].Y := 70;
Poly[2].X := 354;
Poly[2].Y := 90;
Poly[3].X := 347;

```



```

Poly[3].Y := 95;
FillPoly(4,Poly);
Bar(223,72,254,87);
Rectangle(223,72,254,87);
Poly[1].X := 223;
Poly[1].Y := 72;
Poly[2].X := 229;
Poly[2].Y := 68;
Poly[3].X := 260;
Poly[3].Y := 68;
Poly[4].X := 254;
Poly[4].Y := 72;
FillPoly(4,Poly);
Poly[1].X := 260;
Poly[1].Y := 68;
Poly[2].X := 260;
Poly[2].Y := 83;
Poly[3].X := 254;
Poly[3].Y := 87;
FillPoly(4,Poly);
SetFillStyle(SolidFill,Red);
FillEllipse(131,85,6,3);
SetFillStyle(SolidFill,LightRed);
FillEllipse(156,85,6,3);
Line(99,92,291,92);
Line(94,95,285,95);
Line(89,98,279,98);
Line(85,101,273,101);
SetColor(Green);
SetFillStyle(SolidFill,Black);
Bar(249,265,286,269);
Rectangle(249,265,286,269);
Bar(249,264,285,270);
Rectangle(249,264,285,270);
Bar(249,243,285,250);
Rectangle(249,243,285,250);
Bar(249,224,284,230);
Rectangle(249,224,284,230);
SetColor(Yellow);
Line(223,202,345,202);
Line(223,286,347,286);

ReadLn;
GraphDefaults;
CloseGraph;
End. { * MAIN *}

```

File In Turbo C

```
/
/*=====*/
/*
/*      QUESTO FILE E' STATO CREATO DAL PROGRAMMA G.I.P.
/*
/*      G.I.P è uno dei 3 programmi della tesina per l' anno scolastico 1992/93
/*      del gruppo Fochi Michele e Amatulli Davide.
/*
/*      I programmi sono stati scritti da Fochi Michele in Turbo Pascal.
/*
/*=====*/
#include <conio.h>
#include <graphics.h>

void main()

{ /* MAIN */
  int          Gd;
  int          Gm;
  int          Poly[200];
  struct arcctype ArcC;
  char          FillPatt[8];
  char          i;

  Gd = DETECT;
  Gm = DETECT;
  initgraph(&Gd,&Gm,"");

  for (i=BLACK; i<=WHITE; i++)
    setpalette(i,i);

  setrgbpalette(BLACK,    0,0,0);
  setrgbpalette(BLUE,    0,0,42);
  setrgbpalette(GREEN,   0,42,0);
  setrgbpalette(CYAN,    0,42,42);
  setrgbpalette(RED,     42,0,0);
  setrgbpalette(MAGENTA,  42,0,42);
  setrgbpalette(BROWN,   42,21,0);
  setrgbpalette(LIGHTGRAY, 42,42,42);
  setrgbpalette(DARKGRAY, 21,21,21);
  setrgbpalette(LIGHTBLUE, 21,21,63);
  setrgbpalette(LIGHTGREEN, 21,63,21);
  setrgbpalette(LIGHTCYAN, 21,63,63);
```

```

setrgbpalette(LIGHTRED, 63,21,21);
setrgbpalette(LIGHTMAGENTA, 63,21,63);
setrgbpalette(YELLOW, 63,63,21);
setrgbpalette(WHITE, 63,63,63);

```

```

setbkcolor(BLACK);
setfillstyle(SOLID_FILL,CYAN);
bar(0,0,getmaxx(),getmaxy());
setlinestyle(SOLID_LINE,0,1);
setcolor(WHITE);
setfillstyle(SOLID_FILL,BLACK);
bar(73,106,364,312);
rectangle(73,106,364,312);
Poly[0] = 73;
Poly[1] = 106;
Poly[2] = 115;
Poly[3] = 72;
Poly[4] = 406;
Poly[5] = 72;
Poly[6] = 364;
Poly[7] = 106;
fillpoly(4,Poly);
Poly[0] = 406;
Poly[1] = 72;
Poly[2] = 406;
Poly[3] = 278;
Poly[4] = 364;
Poly[5] = 312;
fillpoly(4,Poly);
setfillstyle(SOLID_FILL,LIGHTGRAY);
Poly[0] = 117;
Poly[1] = 289;
Poly[2] = 204;
Poly[3] = 289;
Poly[4] = 204;
Poly[5] = 201;
Poly[6] = 182;
Poly[7] = 201;
Poly[8] = 118;
Poly[9] = 269;
Poly[10] = 118;
Poly[11] = 288;
Poly[12] = 118;
Poly[13] = 288;
Poly[14] = 118;
Poly[15] = 288;
Poly[16] = 118;

```

```

Poly[17] = 288;
fillpoly(9,Poly);
settextstyle(DEFAULT_FONT,HORIZ_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(126,193,"< FWD <");
settextstyle(DEFAULT_FONT,HORIZ_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(128,206,"> REVE >");
bar(73,119,364,165);
rectangle(73,119,364,165);
setfillstyle(SOLID_FILL,RED);
floodfill(211,143,15);
Poly[0] = 364;
Poly[1] = 118;
Poly[2] = 372;
Poly[3] = 113;
Poly[4] = 372;
Poly[5] = 136;
Poly[6] = 379;
Poly[7] = 140;
Poly[8] = 379;
Poly[9] = 298;
Poly[10] = 364;
Poly[11] = 311;
Poly[12] = 364;
Poly[13] = 120;
Poly[14] = 364;
Poly[15] = 120;
fillpoly(8,Poly);
bar(73,313,364,298);
rectangle(73,313,364,298);
settextstyle(DEFAULT_FONT,HORIZ_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(167,156,"STEREO CASSETTE PLAYER");
settextstyle(TRIPLEX_FONT,HORIZ_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(136,137,"auto reverse");
line(196,144,358,144);
line(196,139,358,139);
settextstyle(SMALL_FONT,VERT_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(222,244,"GRAPHIC EQUALIZER");
setfillstyle(SOLID_FILL,GREEN);
bar(239,263,299,274);
rectangle(239,263,299,274);
bar(238,222,300,233);
rectangle(238,222,300,233);

```

```

bar(238,240,299,253);
rectangle(238,240,299,253);
bar(237,260,299,274);
rectangle(237,260,299,274);
settextstyle(DEFAULT_FONT,HORIZ_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(328,227,"120Hz");
settextstyle(DEFAULT_FONT,HORIZ_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(326,247,"1KHz");
settextstyle(DEFAULT_FONT,HORIZ_DIR,1);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(326,268,"8KHz");
ellipse(151,237,220,46,43,46);
ellipse(150,238,221,50,28,28);
ellipse(150,238,218,56,16,16);
setfillstyle(SOLID_FILL,BLACK);
floodfill(159,272,15);
floodfill(153,239,15);
setfillstyle(SOLID_FILL,WHITE);
floodfill(154,258,15);
fillellipse(393,149,5,7);
setcolor(BLACK);
setfillstyle(SOLID_FILL,LIGHTGRAY);
floodfill(395,150,0);
setcolor(WHITE);
setfillstyle(SOLID_FILL,BLACK);
fillellipse(393,149,2,4);
setfillstyle(SOLID_FILL,LIGHTGRAY);
Poly[0] = 388;
Poly[1] = 221;
Poly[2] = 388;
Poly[3] = 252;
Poly[4] = 394;
Poly[5] = 246;
Poly[6] = 394;
Poly[7] = 214;
Poly[8] = 389;
Poly[9] = 221;
Poly[10] = 389;
Poly[11] = 221;
Poly[12] = 389;
Poly[13] = 221;
fillpoly(7,Poly);
bar(323,65,362,84);
rectangle(323,65,362,84);
Poly[0] = 323;

```

```

Poly[1] = 65;
Poly[2] = 330;
Poly[3] = 60;
Poly[4] = 369;
Poly[5] = 60;
Poly[6] = 362;
Poly[7] = 65;
fillpoly(4,Poly);
Poly[0] = 369;
Poly[1] = 60;
Poly[2] = 369;
Poly[3] = 79;
Poly[4] = 362;
Poly[5] = 84;
fillpoly(4,Poly);
bar(306,75,347,95);
rectangle(306,75,347,95);
Poly[0] = 306;
Poly[1] = 75;
Poly[2] = 313;
Poly[3] = 70;
Poly[4] = 354;
Poly[5] = 70;
Poly[6] = 347;
Poly[7] = 75;
fillpoly(4,Poly);
Poly[0] = 354;
Poly[1] = 70;
Poly[2] = 354;
Poly[3] = 90;
Poly[4] = 347;
Poly[5] = 95;
fillpoly(4,Poly);
bar(223,72,254,87);
rectangle(223,72,254,87);
Poly[0] = 223;
Poly[1] = 72;
Poly[2] = 229;
Poly[3] = 68;
Poly[4] = 260;
Poly[5] = 68;
Poly[6] = 254;
Poly[7] = 72;
fillpoly(4,Poly);
Poly[0] = 260;
Poly[1] = 68;
Poly[2] = 260;

```

```

Poly[3] = 83;
Poly[4] = 254;
Poly[5] = 87;
fillpoly(4,Poly);
setfillstyle(SOLID_FILL,RED);
fillellipse(131,85,6,3);
setfillstyle(SOLID_FILL,LIGHTRED);
fillellipse(156,85,6,3);
line(99,92,291,92);
line(94,95,285,95);
line(89,98,279,98);
line(85,101,273,101);
setcolor(GREEN);
setfillstyle(SOLID_FILL,BLACK);
bar(249,265,286,269);
rectangle(249,265,286,269);
bar(249,264,285,270);
rectangle(249,264,285,270);
bar(249,243,285,250);
rectangle(249,243,285,250);
bar(249,224,284,230);
rectangle(249,224,284,230);
FillPatt[0] = 0;
FillPatt[1] = 84;
FillPatt[2] = 170;
FillPatt[3] = 84;
FillPatt[4] = 170;
FillPatt[5] = 64;
FillPatt[6] = 0;
FillPatt[7] = 0;
FillPatt[0] = 0;
FillPatt[1] = 24;
FillPatt[2] = 36;
FillPatt[3] = 66;
FillPatt[4] = 36;
FillPatt[5] = 24;
FillPatt[6] = 60;
FillPatt[7] = 0;
FillPatt[0] = 129;
FillPatt[1] = 60;
FillPatt[2] = 66;
FillPatt[3] = 66;
FillPatt[4] = 66;
FillPatt[5] = 66;
FillPatt[6] = 60;
FillPatt[7] = 129;
setlinestyle(SOLID_LINE,0,1);

```

```

setcolor(WHITE);
setfillstyle(USER_FILL,CYAN);
setfillpattern(FillPatt,CYAN);
floodfill(287,350,15);
FillPatt[0] = 238;
FillPatt[1] = 221;
FillPatt[2] = 187;
FillPatt[3] = 119;
FillPatt[4] = 238;
FillPatt[5] = 221;
FillPatt[6] = 187;
FillPatt[7] = 119;
setlinestyle(SOLID_LINE,0,1);
setcolor(WHITE);
setfillstyle(USER_FILL,LIGHTGRAY);
setfillpattern(FillPatt,LIGHTGRAY);
floodfill(184,281,15);
setlinestyle(SOLID_LINE,0,3);
line(256,226,256,229);
line(265,245,265,249);
line(275,266,275,269);
setcolor(YELLOW);
line(223,202,345,202);
line(223,286,347,286);

while (!kbhit()) /* attesa... */
graphdefaults();
closegraph();
} /* MAIN */

```

G.I.P.

Verrà preso come modello il seguente disegno:

File In Turbo Pascal

```

{
    Generazione di uno sprite con il programma Sprite Manager.
    E' possibile modificare il file a seconda delle proprie
    esigenze.
    SPRITE MANAGER v5.0 - FOCHI MICHELE

    ***** DATI DELLO SPRITE *****

```


$$\}$$

Type RecNomeSprite= Record

(X: 45;

[illegible]

242

```

Var   NomeSprite:      Pointer;      { Puntatore allo sprite }
      GraphDriver:     Integer;      { Driver grafico      }
      GraphMode:       Integer;      { Modo grafico        }

Begin { * Main Program *}
GraphDriver := Detect;
GraphMode := Detect;
InitGraph(GraphDriver,GraphMode,"");
NomeSprite := @ConstNomeSprite;
With ConstNomeSprite Do
  PutImage(((GetMaxX-X) Div 2,(GetMaxY-Y) Div 2,NomeSprite^,NormalPut);
ReadLn;
CloseGraph;
End. { * Main Program *}

```

Le conversioni sono tutte corrette e i programmi non danno errori in compilazione, esecuzione o interpretazione.